
P R O J E C T S T U D Y H A N D B O O K

从零读懂 nsF5

隐写项目

信息隐藏 × 机器学习 × 工程实践 —— 项目学习手册

建议周期：6–12 个月（含 12 周快速入门） · 每周 6–8 小时
适合：刚接触 Python / 未系统学过信息安全与机器学习的同学
2026 年 10 月 · 项目 v1.9.x · 配套代码 F:\Steganography

目 录

导读 · 这本手册怎么用	1
0.1 读者对象与起点	2
0.2 本手册的阅读约定	2
0.3 12 周快速入门路线总览（6–12 个月深入版见附录 F）	3
0.4 学完以后，你应该能做到	4
0.5 项目地图：先认识你要学的代码	5
0.6 学习小贴士	6
0.7 v1.4.0 更新说明（2026-09-06）	7
第 1 章 · 数字图像与二进制（第 1 周）	8
1.1 一张数字图像，其实就是一堆数字	9
1.2 二进制、字节与位	13
1.3 为什么图像里“有空地”可以藏东西	15
1.4 位平面：把一张图拆成 8 层	17
1.5 回到项目：先认识 cover 与 stego	20
1.6 小结与自我检查	21
第 1½ 章 · JPEG 是什么，DCT 系数是什么？（压缩域隐写的战场）	23
1.5.1 JPEG 是什么：一台“有损压缩机”	24
1.5.2 DCT 系数是什么：8×8 块的“配方表”	25
1.5.3 在系数上做 nsF5：载体、湿点与减幅	29
1.5.4 在真 JPEG 上跑一遍	31
1.5.5 回到项目看代码	33
第 2 章 · 准备工具：Python、NumPy 与图像读写（第 2 周）	35
2.1 安装环境	36
2.2 用 NumPy 思考图像	36
2.3 图像读写：认识 image_io.py	39
2.4 第一次端到端运行：run_e2e.py	41
2.5 常见报错速查	42

第3章 · LSB 隐写与它的统计“指纹” (第3–4周)	45
3.1 加密 vs 隐写：先回答第一个问题	46
3.2 最小可运行的 LSB 隐写	46
3.3 为什么会露馅：卡方检验	49
3.4 RS 分析：LSB 的“结构塌缩”	52
3.5 项目的综合判读：analyze()	54
3.6 动手实验：藏一句英文，再抓它出来	55
3.7 小结与自我检查	56
第4章 · 矩阵编码与 F5：少改一点，藏得更多 (第5周)	58
4.1 从“每像素 1 位”到“平均每次改动藏 p 位”	59
4.2 汉明码速成：校验矩阵与伴随式	60
4.3 手算一个 $p=3$ 的例子	62
4.4 F5：矩阵编码 + JPEG 系数减幅	64
4.5 回到项目：MatrixEmbedding 与效率曲线	65
4.6 小结与自我检查	66
第5章 · nsF5 与湿纸编码：项目的核心算法 (第6周)	68
5.1 先把问题拧清楚：F5 的收缩	68
5.2 湿纸编码：先划湿点，再解方程	70
5.3 读懂项目：nsF5Pixel._embed()	72
5.4 再看三个求解函数	73
5.5 验证实验：效率与往返	73
5.6 小结与自我检查	74
第6章 · 口令、SHA-256 与“键控”安全设计 (第6–7周)	76
6.1 如果嵌入位置是固定的，会怎样？	77
6.2 两条种子规则：先认图，再找正文	77
6.3 确定性置乱：splitmix64 + Fisher–Yates	79
6.4 篡改感知：到底能感知什么	81
6.5 动手实验：口令错误与像素篡改	82
6.6 小结与自我检查	83
第7章 · 机器学习基础 (写给第一次学的人) (第7–8周)	85

7.1 隐写检测，本质上是一个二分类问题.....	86
7.2 数据：样本、特征与标签.....	86
7.3 模型怎么“学”：损失、梯度与逻辑回归.....	86
7.4 过拟合、交叉验证与数据泄漏.....	88
7.5 评估指标：别只看准确率.....	90
7.6 小结与自我检查.....	91
第 8 章 · 用机器学习做隐写检测（第 8–9 周）	93
8.1 先回答一个反直觉的问题：为什么不用 CNN 直接看像素？	94
8.2 认识 11 个特征.....	95
8.3 数据怎么来：make_dataset.py 的“1+6”设计	97
8.4 训练脚本怎么读：train_model.py	97
8.5 怎么读结果：ROC、AUC 与按档检出率.....	98
8.6 灵敏度：在 GUI 里如何工作.....	100
8.7 动手实验：从训练到单图判定.....	100
8.8 v1.4.0 更新：SRM、143 维特征与双版本模型（2026-09）	101
8.9 小结与自我检查.....	104
第 9 章 · 工程化：C++、GPU、GUI 与测试（第 10 周）	106
9.1 系统架构：分层而不耦合.....	107
9.2 C++ 加速：为什么是“热路径”，以及如何保证没错.....	107
9.3 GPU 版：把 11 维特征“批量向量化”.....	109
9.4 GUI：把研究工具变成可教学的应用	110
9.5 测试与 CI：让重构不翻车	110
9.6 代码阅读路线图（按需选用）	112
9.7 v1.4.0 更新：SRM、多源数据与模型工程（2026-09）	112
第 10 章 · 综合实践：从“读懂”到“做出来”（第 11–12 周）	115
10.1 方向 A：复现并批判性验证（最稳）	115
10.2 方向 B：功能扩展（最有成就感）	117
10.3 方向 C：教学演示再包装.....	118
10.4 文档与代码对照阅读表.....	118
10.5 汇报/答辩常见问题与应答要点.....	119

10.6 交付清单（最后一周）	120
第 11 章 · 边界、伦理与继续前进（穿插学习）	122
11.1 法律与伦理边界	122
11.2 项目已知局限（看懂边界才算真正读懂）	123
11.3 现代信息隐藏与隐写分析地图	125
11.4 结语	126
第 12 章 · 批判性思维（复盘指南）	128
12.1 一切以实验结果为准：实验是怎么骗人的	129
12.2 敢于质疑设计，但要建立在思考之上	131
12.3 质疑之后，要敢于并善于给出自己的方案	132
12.4 区分“结论”与“证据”	133
12.5 承认“我不知道”是一种能力	134
12.6 警惕“看起来合理的叙事”	134
12.7 把“复现”当作批判性思维的第一课	135
12.8 实验日志是批判性思维的载体	136
12.9 章末练习：给一个结论做体检	137
附录 A · 术语中英对照	139
附录 B · 常用命令速查	144
附录 C · 12 周打卡检查表	146
附录 D · 概念 → 代码定位表	148
附录 E · 推荐资源	150
书籍（中文先入门）	150
核心论文（按阅读顺序）	150
在线资料与工具	150
附录 H · 命令行工具与安装版（v1.9.0）	151
H.1 三种安装方式	151
H.2 命令行三步（与 GUI 一一对应）	151
H.3 新手引导路径	151
H.4 与手册正文的对应	152
附录 J · 章末练习提示与答案	153

J.1 第 1 章	153
J.2 第 1½ 章	153
J.3 第 2 章	153
J.4 第 3 章	154
J.5 第 4 章	154
J.6 第 5 章	155
J.7 第 6 章	155
J.8 第 7 章	156
J.9 第 8 章	157
J.10 第 9 章	157
J.11 第 10 章	157
J.12 第 11 章	158
附录 K · 真实数据速查（供实验对账）	159
K.1 嵌入实测（256×256 演示图，2026-10-04）	159
K.2 满容量效率扫描（随机噪声图，compute_comparison(6) 实测）	159
K.3 检测主表（BOSSbase 1.01，按源图 holdout，摘自 docs/RESULTS.md）	160
K.4 语料与协议要点	160
K.5 篡改与哈希实测	160
K.6 LSB 嵌入率扫描（第 3 章，ch03_results.csv）	161
附录 L · 配套工具 ImageMatrix 使用指南	162
L.1 它是什么，为什么配它	162
L.2 获取与安装	162
L.3 界面与核心操作	163
L.4 五种矩阵格式速查	164
L.5 命令行（批处理与自检）	164
L.6 与手册各章的对应	165
L.7 一个完整的动手流程	165

导读 · 这本手册怎么用

本章导航 · 先看问题，再读正文

核心问题 这本手册凭什么值得你花 12 周？

读完你能回答

- 说清本手册的写法：**先摸实物、再讲原理**，每条原理都配一个可以亲手复算的例子；
- 复述三条学习纪律：每个抽象先摸实物、每条原理配可复算算例、每章逼问“自己实现会在哪踩坑”；
- 遇到“书上数字与你的运行结果不一致”时，按**先怀疑环境、再怀疑理解、最后才是怀疑书**的顺序排查；
- 会用五种标记块（新手提示 / 避坑提醒 / 动手做 / 想一想 / 回到项目看代码）与附录 J / K / L。

读法建议：第一遍只跑代码让现象出现，第二遍精读复算，第三遍合上书做练习。

一本好书和一个好课堂的区别，往往不在讲什么，而在按什么顺序讲、在每个新概念出现之前你手里已经握着什么。计算机系统领域的经典教材 CSAPP（《深入理解计算机系统》）之所以被反复推荐，靠的是三条朴素的纪律：每讲一个抽象，都先让你在更低的层次上摸过它的实物；每讲一条原理，都配一个可以亲手复算的数字例子；每讲完一章，都逼你回答“如果让你自己实现，会在哪里踩坑”。本手册把这三条纪律搬进了隐写学习——每个抽象都先落到看得见摸得着的实物上，每条原理都配一个能亲手复算的例子，每章结束都逼问一次“坑在哪”。

本手册的另一条原则是：**所有数字必须可以复算**。你在书中看到的每个算例——汉明码的伴随式、splitmix64 的输出、卡方统计量、JPEG 常数块的 DC 值——都是用项目源代码现场算出来的，并标注了出处；附录 K 汇总了项目真实的实验数据。遇到书上数字与你的运行结果不一致时，先怀疑环境，再怀疑理解，最后才是怀疑书。

想一想 |

翻开任何一本教材，你习惯先看目录还是先跑代码？CSAPP 的作者建议：先跑一个最小的可运行程序，再回头读原理。本手册的每一章都按这个顺序组织——“动手做”方块永远出现在原理讲解之前或之中。回想你上一次“只看不做”的学习，它最后留下了多少可以调用的知识？

这是一份“跟着代码学”的手册。它不代替教科书，而是把 F:\Steganography 项目背后的信息隐藏与机器学习知识，按零基础本科生可以接受的顺序拆开：先用直觉和例子讲清楚“为什么”，再带你看看项目里“怎么实现”，最后让你亲手跑实验、改代码。

0.1 读者对象与起点

你只需要具备这些前提，其余内容手册会逐步补齐：

会用鼠标操作电脑、能安装软件（Windows 环境）；

上过大学数学公共课或愿意遇到公式时“先跳过、再回头”；

有一点编程基础更好，但没有也没关系——第 2 章会带你把 Python 用到够用；

至少有 12 周（约 3 个月）的耐心，每周 6–8 小时；想深入请预留 6–12 个月。

新手提示 | 如果你连 Python 都没装过，不要慌：前两周就是专门为这种情况设计的。你不需要成为程序员，只需要成为“能读懂并修改这个项目的人”。

0.2 本手册的阅读约定

五种方块之外，本手册还大量使用三种“页面上的工具”，先在这里统一交代。一是**算例（Worked Example）**：完整走一遍数字的手算过程，你应当拿草稿纸跟着算，算对了再往下走；二是**章末练习**：每章最后 3–5 题，分“手算”与“上机”两类，答案提示集中在附录 J——先自己想十分钟再看提示；三是**真实数据标注**：凡是来自项目真实运行结果的数字，都在括号里注明出处（如“实测 2026-10-04”），这是本手册与你读过的多数中文教材最大的不同——我们不允许任何“编出来好看”的数字。

手册里反复出现五种小方块，含义如下：

标记	含义
新手提示	把容易卡住的概念换成大白话或类比，让你顺利往下走
避坑提醒	初学者最容易踩的坑：数值类型、参数不一致、误读统计量等

标记	含义
动手做	必须亲自动手的小实验，只读不练等于没学
想一想	不需要标准答案的思考题，用来检验自己是否真的理解
回到项目看代码	指出应当打开哪个文件、读哪一段，建立“概念→代码”的映射

关于配图。 全书正文配图由配套项目 **StegoLab** 生成：它把每一章的概念做成可复现的图，并把每章“动手做”做成可运行的脚本。每张图的图注都写成问答式——**先点出“这张图回答的是哪个问题”，再说它展示了什么**，方便你把图和正文里那句话直接对上。配图的数据来源登记在 StegoLab 的 figure_manifest.json 中，重跑脚本应得到逐字节一致的图；来自 ImageMatrix 仓库的界面截图，则是零代码教具 ImageMatrix 的实机画面。正文里出现的关键数字，都能在附录 K 查到出处。

0.3 12 周快速入门路线总览（6–12 个月深入版见附录 F）

路线遵循“载体基础 → 隐写算法 → 隐写分析 → 机器学习 → 工程加速 → 综合 实践”的自然顺序。每一周都对应可运行的代码或可复现的实验：

周次	主题	对应章节	动手成果
第 1 周	数字图像与二进制	第 1 章	能亲手查看一张图的像素与位平面
第 1 周（补充）	JPEG 与 DCT 系数	第 1½ 章	能解释量化表如何决定隐写容量
第 2 周	Python/NumPy/Pillow	第 2 章	跑通 GUI 或 run_e2e.py，能读写图像数组
第 3–4 周	LSB 隐写与盲分析	第 3 章	藏一句话→用卡方/RS 检测到它
第 5 周	矩阵编码与 F5	第 4 章	用汉明码做到“改动更少、藏得更多”

周次	主题	对应章节	动手成果
第 6 周	nsF5、湿纸与哈希键控	第 5–6 章	读懂 ns5_core.py，解释湿纸求解
第 7 周	机器学习基础	第 7 章	能讲清特征/标签/过拟合/AUC
第 8–9 周	机器学习隐写检测	第 8 章	跑通 v1 11 维与 v2 143 维两套管线；理解 SRM 与 143d/53d 双版本模型
第 10 周	C++/GPU/数据工程/GUI	第 9 章	看懂加速原理、自校验机制与多源数据集管线 (SRM/BOSSbase)
第 11–12 周	综合项目与汇报	第 10–11 章	完成一个可演示的改进实验
动手做 请把这张表抄成一张打卡表贴在桌前，或直接使用附录 C 的检查表。每周结束时勾掉一行，并回答该章末尾的“想一想”。	动手做 请把这张表抄成一张打卡表贴在桌前，或直接使用附录 C 的检查表。每周结束时勾掉一行，并回答该章末尾的“想一想”。	动手做 请把这张表抄成一张打卡表贴在桌前，或直接使用附录 C 的检查表。每周结束时勾掉一行，并回答该章末尾的“想一想”。	动手做 请把这张表抄成一张打卡表贴在桌前，或直接使用附录 C 的检查表。每周结束时勾掉一行，并回答该章末尾的“想一想”。

0.4 学完以后，你应该能做到

用自己的话解释：隐写、隐写分析、嵌入效率、收缩、湿纸编码、伴随式；

说出 LSB 隐写为什么会被卡方检验与 RS 分析发现；

手算一个 $p=3$ 的汉明矩阵嵌入例子（块内最多改 1 位）；

解释 nsF5 与 F5 的本质区别，以及为什么“无收缩”更重要；

解释图像哈希键控如何实现解码自同步，并说明它的局限；

解释机器学习二分类的完整流程，包括为什么要按照片分组交叉验证；

读懂 v1 的 11 维与 v2 的 143 维特征（含 SRM 统计），能解释 143d 稳健版与 53d 可解释版的双模型策略；

讲清 C++ DLL 与 GPU 加速分别加速了什么，为什么需要一致性校验；

独立完成一个小的改进实验并写出诚实的结果分析。

0.5 项目地图：先认识你要学的代码

整个项目是一个“研究工具”：既能做算法实验，也打包成带 GUI 的软件。先记住这些关键文件，后面每一章都会回来找它们：

文件	角色	学习顺序
src/image_io.py	图像读写封装：打开/保存 8bit 灰度与彩色图	第 2 章
src/ns5_core.py	算法核心：汉明码、湿纸、哈希键控、嵌入/解码	第 3–6 章
src/steganalysis.py	盲隐写分析：卡方、RS、综合概率与判读	第 3 章
src/efficiency.py	码族与嵌入效率理论/实测曲线	第 4 章
src/matrix_demo.py	伴随式查找的教学演示逻辑	第 4 章
src/fsfeatures.py + cpp/fsfeatures.dll	v1 11 维统计特征的 ctypes 绑定 (v2 143 维见 featurize_v2.py)	第 8 章
src/make_dataset.py / train_model.py	数据集生成（v1/v2 特征、SRM、多源）与训练/评估	第 8 章
src/ml_predict.py	单图 ML 判定封装（143d/53d 双版本 + 灵敏度）	第 8 章
src/cppembed.py + cpp/nsf5embed.dll	C++ 嵌入/置乱热路径与自校验	第 9 章
gpu/*.py	PyTorch 批量特征（v1 11 维 / v2 143 维）与 GPU 训练	第 9 章
src/gui.py	tkinter 图形界面，含教学面板	第 9 章

文件	角色	学习顺序
src/test_core.py 等测试	验证嵌入往返、分析与误报的回归测试	全程
src/featurize_v2.py + src/srm_filter.py + gpu/featurize_v2_gpu.py	v1.4 新增：30 核 SRM 预处理与 143 维 v2 特征（CPU/GPU 双实现）	第 8–9 章
回到项目看代码 建议从现在起保持项目窗口打开。手册里提到某个文件时，就切过去找到它；第 2 章之后，多数知识点都要求你“先运行、再阅读”。	回到项目看代码 建议从现在起保持项目窗口打开。手册里提到某个文件时，就切过去找到它；第 2 章之后，多数知识点都要求你“先运行、再阅读”。	回到项目看代码 建议从现在起保持项目窗口打开。手册里提到某个文件时，就切过去找到它；第 2 章之后，多数知识点都要求你“先运行、再阅读”。

0.6 学习小贴士

在 CSAPP 的前言里，作者们反复强调一件事：这门课的价值不在结论，而在你亲历的“啊，原来如此”时刻的次数。本手册据此给出一个三遍读法，配合每章的练习食用：

1. 第一遍（跑通）：只读“动手做”方块和代码，把可运行的例子全部跑起来，容忍看不懂的细节——先让现象出现；
2. 第二遍（精读）：回到正文与算例，把每个手算例子在草稿纸上复算一遍，算错的地方就是理解的缺口；
3. 第三遍（批判）：合上书回答每章开头的“开场问题”，再做章末练习。能给别人讲明白的，才是你的。

先跑通，再读懂。很多概念在纸上绕，跑一次端到端脚本就通了；

公式分两步消化。第一遍只看结论和直觉，第二遍再回到推导；

用自己的话复述。每章小结能讲给同学听，才算掌握；

记录实验日志。改了哪些参数、结果如何，是最后综合项目最宝贵的材料；

善用测试。项目自带的 test_core.py / test_steg.py 是最快的“对不对”裁判。

想一想 | 在你开始之前，先花 5 分钟回答：加密和隐写有什么不同？如果你说不清，太好了——这正是第 3 章要解决的第一个问题。

0.7 v1.4.0 更新说明 (2026-09-06)

本手册 9 月 3 日初稿对应项目 v1.3；9 月 6 日项目更新到 v1.4.0，本版手册已同步。隐写算法部分（第 1–6 章）不受影响；机器学习与工程部分补充了 8.8 与 9.7 两节，并把旧指标标注为“v1 基线”。

ML 检测从 11 维特征 + LR/XGB (AUC 约 0.75–0.79) 升级为 143 维 LGB 默认版 (8-split 平均 0.8980) 与 53d 可解释版 (0.8461)：143d 更准、53d 每一维都能解释，详见 8.8 与 docs/RESULTS.md；

新增 SRM 高通滤波预处理、143 维 v2 特征与 12 档变体，并补入真实 JPEG 干净样本与 BOSSbase 全量多源数据，详见 8.8 与 9.7；

项目迁移到新 GitHub 仓库 (Yukinoshita-lin/nsf5-steganography)，许可证改为 Apache-2.0 并新增 NOTICE；

术语表、命令速查、12 周打卡表与概念→代码定位表已同步补充 v1.4 内容。

第 1 章 · 数字图像与二进制（第 1 周）

本章导航 · 先看问题，再读正文

核心问题 一张 256×256 的图片，在计算机的硬盘上到底是什么？

读完你能回答

- 口述灰度图与彩色图在内存里的数据布局（行优先存储、通道顺序）；
- 做十进制 ↔ 二进制换算，并说清四种位运算（ $v \& 1$ / $v | 1$ / $v \& 0xFE$ / $v \wedge 1$ ）各自在隐写里的用途；
- 区分视觉冗余与统计冗余，解释“凭什么能藏东西而不被发现”；
- 预测 LSB 位平面的外观，并准确使用 cover / stego 两个项目术语。

本章没有一行隐写代码——这是刻意的：先把“图像是数字”变成可以动手操作的确定知识。

开场问题：一张 256×256 的图片，在计算机的硬盘上到底是什么？是一张“图”吗？如果你回答“是一堆数字”，恭喜你已经站在这一章的终点；如果你不确定“一堆数字”和“你看到的画面”之间隔着几层约定，这一章就是为你写的。CSAPP 讲计算机系统时的核心手法是“剥洋葱”：任何让你觉得神奇的东西，往下剥一层，通常是一个朴素的约定。数字图像就是这样一个约定：所谓“图片”，不过是一张写满整数的表格，外加一句“请按亮度解释这些数”的说明书。

本章没有一行隐写代码——这是刻意的。隐写的一切技巧都建立在“图像是数字”这个事实上，这个事实在大多数人脑中是模糊的直觉，而我们要把它变成可以动手操作的确定知识。读完本章，你应当能不看任何资料，回答出下面这张地图里的问题：

小节	回答的问题	你的产出
1.1 图像即矩阵	一张图在内存里长什么样？	能口述灰度图/彩色图的数据布局
1.2 二进制与位	LSB 到底是哪个位？	会做十进制↔二进制换算与位运算
1.3 冗余	凭什么能藏东西不被发现？	能区分视觉冗余与统计冗余
1.4 位平面	一张图拆成 8 层各是什么样？	能预测 LSB 平面的外观
1.5 项目术语	cover / stego 是什么？	能跑通第一个端到端脚本

本章配了一件“零代码的实物”。 CSAPP 的第一条纪律是“先摸实物、再讲原理”，本章的实物就是“图像 = 数字矩阵”这件事本身。但原文给出的摸法都要写 Python——那是第 2 章才教的本事。为了让零基础的你在写下第一行代码之前就能亲手验证，本章配套了一个独立小工具 **ImageMatrix**（同作者的姊妹项目，与手册代码库 `F:\Steganography` 没有任何依赖关系，本身也不含隐写功能）：它能把任意图片直接导出成 R/G/B/灰度 的数字矩阵文本，也能把改过的文本导回成图片。本章每一节末尾的“动手做”都会用到它，完整说明见**附录 L**。

动手做 | 本章目标：把“图像 = 数字矩阵”的直觉焊死；会手算二进制与字节；能说出为什么人眼和统计都能容忍一点“像素级扰动”。

1.1 一张数字图像，其实就是一堆数字

把“表格”这个说法再精确一点：一张 $W \times H$ 的 8bit 灰度图，在内存里就是 $W \times H$ 个**连续排列的字节**，按行优先存储——先存完第 0 行的所有像素，再存第 1 行。所以“第 r 行第 c 列的像素”在一维编号里是 $r \times W + c$ 。这个一维视角在隐写里极其重要：**第 5 章的湿纸编码和第 6 章的置乱，都会把二维图像当作一维长条来编号**。现在先记住：二维的“图”和一维的“比特串”是同一份数据的两种看法，切换只在一念之间。

避坑提醒 |

彩色图的通道顺序是初学者第一大坑。Pillow 读出的数组是 $(H, W, 3)$ ，通道顺序是 R、G、B；但如果你日后用 OpenCV (cv2)，它读出的顺序是 **B、G、R**——同样一张橙色图，一个库里是 $(255, 128, 0)$ ，另一个库里是 $(0, 128, 255)$ 。用错库的顺序做隐写分析，直方图特征会整体错位，结论全错且很难察觉。本项目统一用 Pillow，暂时不必担心；写下这一条是因为你迟早会遇到 cv2。

还有一个值得养成的习惯：看到“图像”两个字，脑中同时浮现两张表——**亮度的表**（你看到的画面）和**数字的表**（计算机存的东西）。本章剩下的全部工作，就是在两张表之间来回切换，并确认它们说的确实是同一件事。

把一张 8bit 灰度图放大到像素级，你会看到它是一张由 0 ~ 255 整数组成的二维表格。0 代表全黑，255 代表全白，中间是不同深浅的灰。计算机只保存这张表格的“尺寸”和“数字”，不保存你肉眼看到的画面。

彩色图则是三张这样的表格叠在一起：R（红）、G（绿）、B（蓝）三个通道。常见的 RGB 图每个像素有 3 个 0~255 的值。比如一个橙色像素可以写成 R=255、G=128、B=0。

在代码里，灰度图就是一个二维数组，彩色图就是一个三维数组。项目的 `src/image_io.py` 把这件事封装成了两个函数：`load_as_gray()` 负责把任意支持的图片读成灰度数组，`save_image()` 负责把数组保存成 PNG。

新手提示 | 数学里我们用“行、列”描述矩阵，图像处理里则常说“高、宽”：一张 512×512 图就是 512 行、512 列。数组形状 (512, 512) 的写法正是“行数在前”。

动手做 |

第一次“摸到”图像矩阵（零代码，约 5 分钟）

- ① 安装并打开 ImageMatrix（安装包见附录 L；也可直接运行 `build\imagematrix.exe`）；
- ② 点“生成测试图案”，得到一张自带彩条 / 灰阶 / 渐变的测试图，不必自己找素材；
- ③ 把“矩阵格式”选成 GRAY，点“查看矩阵文本”——你现在看到的，就是本节说的那张“整数表格”；
- ④ 点“导出为 TXT...”，把这张图变成一份纯文本，用记事本打开它。

做完再回头看本节第一段：所谓“图片”，真的只是这些数字，外加一句“请按亮度解释”。

```

# ImageMatrix 1.0 - 图像数值矩阵 (image -> matrix)
# width 320
# height 240
# format CHANNELS
# max 255
# gray = round(0.299*R + 0.587*G + 0.114*B)
# 分通道矩阵: R 矩阵 -> G 矩阵 -> B 矩阵 -> GRAY 矩阵, 每个矩阵 h 行 w 列

# channel R (红色通道)
0 0 1 2 3 3 4 5 6 7 7 8 9 10 11 11 12 13 14 15 15 16 17 18 19 19 20 21 22 23 23 24 25 26 27
232 233 234 235 235 236 237 238 239 239 240 241 242 243 243 244 245 246 247 247 248 249 250
0 0 1 2 3 3 4 5 6 7 7 8 9 10 11 11 12 13 14 15 15 16 17 18 19 19 20 21 22 23 23 24 25 26 27
232 233 234 235 235 236 237 238 239 239 240 241 242 243 243 244 245 246 247 247 248 249 250
0 0 1 2 3 3 4 5 6 7 7 8 9 10 11 11 12 13 14 15 15 16 17 18 19 19 20 21 22 23 23 24 25 26 27
232 233 234 235 235 236 237 238 239 239 240 241 242 243 243 244 245 246 247 247 248 249 250
0 0 1 2 3 3 4 5 6 7 7 8 9 10 11 11 12 13 14 15 15 16 17 18 19 19 20 21 22 23 23 24 25 26 27
232 233 234 235 235 236 237 238 239 239 240 241 242 243 243 244 245 246 247 247 248 249 250
0 0 1 2 3 3 4 5 6 7 7 8 9 10 11 11 12 13 14 15 15 16 17 18 19 19 20 21 22 23 23 24 25 26 27
232 233 234 235 235 236 237 238 239 239 240 241 242 243 243 244 245 246 247 247 248 249 250
0 0 1 2 3 3 4 5 6 7 7 8 9 10 11 11 12 13 14 15 15 16 17 18 19 19 20 21 22 23 23 24 25 26 27
232 233 234 235 235 236 237 238 239 239 240 241 242 243 243 244 245 246 247 247 248 249 250
0 0 1 2 3 3 4 5 6 7 7 8 9 10 11 11 12 13 14 15 15 16 17 18 19 19 20 21 22 23 23 24 25 26 27
232 233 234 235 235 236 237 238 239 239 240 241 242 243 243 244 245 246 247 247 248 249 250
0 0 1 2 3 3 4 5 6 7 7 8 9 10 11 11 12 13 14 15 15 16 17 18 19 19 20 21 22 23 23 24 25 26 27

```

图 1-1 ImageMatrix 的“矩阵文本预览”。文件头用 # 注释声明 width / height / format / max 与灰度公式，下面就是逐行的数值矩阵——这正是本节所说的“图像即数字表格”。（截图来自 ImageMatrix 仓库）

矩阵格式	导出内容	对应本手册的哪个概念
GRAY	一张 W×H 的灰度矩阵	1.1 灰度图 = 二维整数表
CHANNELS	R、G、B、灰度 四张矩阵	1.1 彩色图 = 三张表叠起来
RGB / RGBA	每个像素一行 R,G,B,(A)	1.1 的“橙色像素 = 255,128,0”
HEX	每个像素一行 RRGGBB	1.2 的十六进制记法

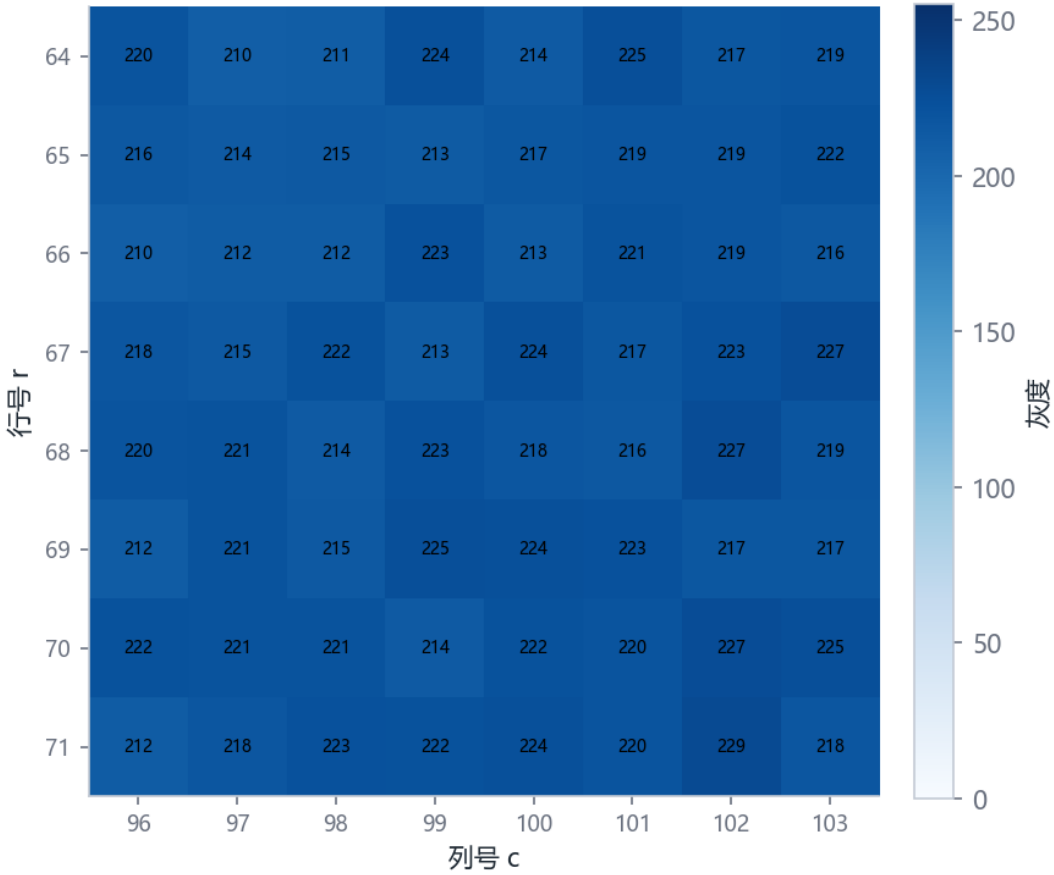
这张表请留着：第 2 章你用 NumPy 读出的数组、第 3 章你逐位改动的像素，形状与这里看到的完全一致。ImageMatrix 只是把“代码里的数组”提前变成了“屏幕上看得见的数字”。

同一份数据的两种看法：画面 与 数字表



图 1-2 画面与数字表的对应。这张图回答的是：一张图在计算机里到底是画面，还是一张数字表？左侧画面中被框出的 8×8 区域，右侧即其数值表——这就是“图像即数字”最直白的证据

图像就是一张整数表：每格是一个 0-255 的灰度值



取自载体图 cover.png 的第 64-71 行、第 96-103 列

图 1-3 图像即整数表（8×8 像素块）。这张图回答的是：一个像素块在内存里究竟长什么样？8×8 像素块的热力图，格中数字即该像素的灰度值

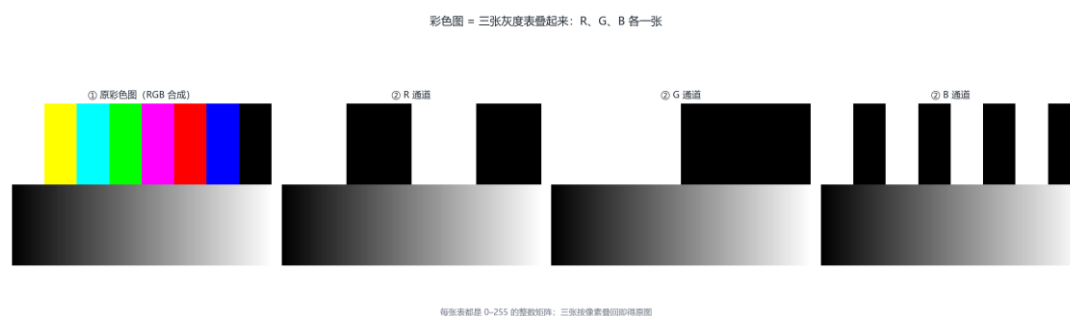


图 1-4 RGB 三通道分解。这张图回答的是：彩色图为什么等于三张灰度表叠起来？一张彩色图拆成 R / G / B 三张灰度矩阵，彩色图 = 三张表叠起来

1.2 二进制、字节与位

十进制与二进制只是同一数量的两种记法，但位运算给“按位操作”提供了四种瑞士军刀，隐写代码里天天见面，值得在这里一次配齐。设 v 是一个像素值：

运算	写法	作用	隐写里的用途
取最低位	$v \& 1$	得到 LSB (0 或 1)	提取：读出藏在像素里的比特
置最低位	$v 1$	把 LSB 强行变成 1	嵌入比特 1
清最低位	$v \& 0xFE$	把 LSB 强行变成 0	嵌入比特 0 的第一步
翻转最低位	$v \wedge 1$	LSB 在 0/1 之间翻转	F5/nsF5 的“改一位”——原来是什么不重要，翻一下就行

注意最后一行的妙处：翻转 (XOR 1) 不关心原值是 0 还是 1，翻一下必然变成另一个值，而且**数值只变化 ± 1** 。这就是为什么第 4 章之后所有“改一个系数”的操作，底层都是异或——它把“改成想要的值”这个问题，简化成了“要不要翻”。

想一想 |

200 的二进制是 11001000。不写代码，口算回答：它的最高有效位 (MSB) 是几？把它从 200 改成 201，二进制里变的是哪一位？把 200 改成 72 (=01001000)，变的是哪一位？两种改法对画面亮度的影响，差别有多大？——想清楚这三问，你就明白为什么隐写偏爱与 LSB 而不是 MSB 打交道。

计算机底层只认识 0 和 1。一个 0 或 1 叫 1 bit（位），8 个 bit 组成 1 byte（字节）。8 个 bit 一共能表示 $2^8=256$ 种组合，正好对应灰度值 0~255。

例如十进制 $200 = 128+64+8$ ，写成 8 位二进制是 11001000；它的最低有效位（LSB）就是最右边的那个 0。把 200 的 LSB 改成 1，得到 201——在人眼里，灰度 200 和 201 几乎无法区分。

“最低有效位”是整本手册最重要的概念之一：LSB 隐写，就是偷偷修改像素的 LSB，把秘密比特藏进去。

动手做 | 打开一个 Python 交互环境（或写个小脚本），验证下面这段：

体验：十进制、二进制与 LSB

```
v = 200
print(bin(v))           # 0b11001000
print(v & 1)             # 最低位 = 0
print(v ^ 1)            # 翻转最低位 = 201
print(v & 0b11111110)   # 把最低位清零 = 200
print(v & 0b11111110 | 1) # 先清零再置 1 = 201
```

把 1.2 节的“ $200 = 11001000$ ”写成数学上的**按权展开式**——每一位的权重是 2 的幂，从右往左依次是 $2^0, 2^1, \dots, 2^7$ ：

$$\begin{aligned} 156 &= 1 \cdot 2^7 + 0 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 \\ &\quad + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 \\ &= 128 + 16 + 8 + 4 = 156 \end{aligned}$$

最右一项的权重是 $2^0 = 1$ ——所谓**最低有效位（LSB）**，字面意思就是“2 的零次方那一位”。翻转它对数值的贡献恰好 ± 1 ，这为第 3 章的一切埋下伏笔。

动手做 |

零代码翻转一个 LSB——隐写的最小动作

- ① 用 ImageMatrix 打开一张图，格式选 GRAY，导出为 TXT；
- ② 用记事本在矩阵里找任意一个偶数值（比如 200），把它改成 201——这就是把它的 LSB 由 0 翻成 1；
- ③ 回到 ImageMatrix，点“导入 TXT...”，选刚才改过的文件；

④ 点“另存为图片...”存成 PNG，再与原图并排比较。

你几乎不可能看出差别——可这一次改动，已经往图里写进了 1 bit。把同样的动作做 240 次（第 3 章教你用代码批量完成），就是一次完整的 LSB 隐写。

想看得更“二进制”一点，把格式换成 HEX 再导出一遍：同一个像素会写成 `C8` 这样的十六进制，而 `C8` 就是本节 200 的二进制 `11001000` 的另一种写法。ImageMatrix 底部的状态栏还会在鼠标划过像素时实时显示它的 R/G/B/灰度 与 `#RRGGBB`，你可以逐点核对“数值 ↔ 二进制 ↔ 颜色”是否对得上。

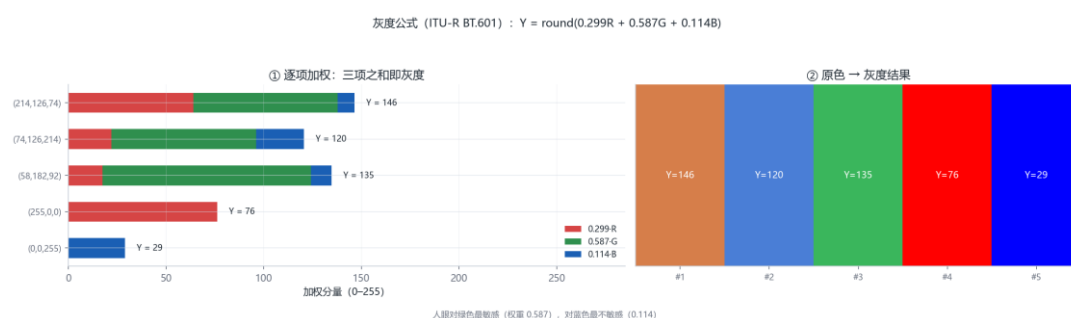


图 1-5 灰度公式逐项对照。这张图回答的是：灰度值 Y 到底是怎么从 R / G / B 算出来的？每个颜色的 R / G / B 加权分量与所得灰度，验证 $Y = \text{round}(0.299R + 0.587G + 0.114B)$

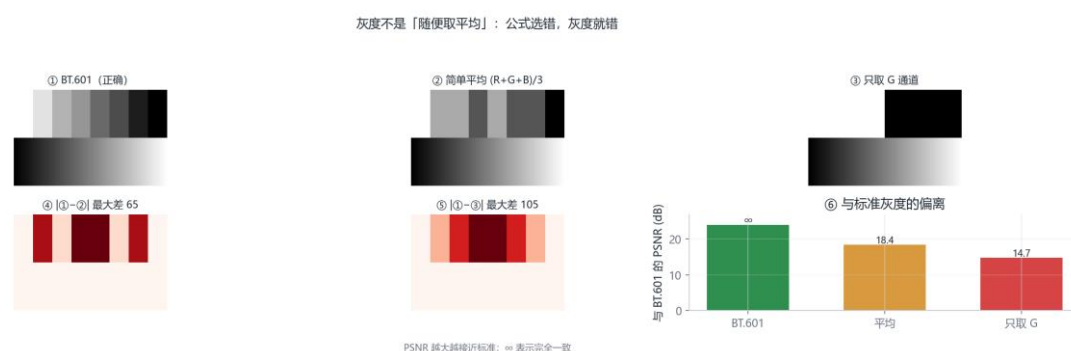


图 1-6 三种灰度公式对照。这张图回答的是：换一种灰度公式，结果会差在哪里？标准公式与两种常见错误做法的灰度结果及偏离程度——错在哪里一眼可见

1.3 为什么图像里“有空地”可以藏东西

“视觉冗余”可以再量化一点：人眼对亮度差的分辨能力（术语叫 JND，Just Noticeable Difference）在中等亮度区域大约是 1% ~ 2% 的相对变化。8bit 灰度里相邻两级（比如 128 与 129）的相对差只有 0.78%，远低于 JND——这就是“改 LSB 肉眼看不见”的生理学根据。注意它的边界：

在接近纯黑或纯白的区域，人眼的分辨能力反而更强，同样的 ± 1 更容易被察觉——所以专业隐写会把“改哪里”与“像素亮度”挂钩，第 8 章的嵌入率（payload）分档就是这件事的工程化。

“统计冗余”则有一组可以亲手验证的数字：拿任何一张自然照片计算相邻像素的相关系数，水平方向通常在 0.90 ~ 0.97 之间。这意味着相邻像素的值高度“可预测”——一张图第 r 行第 c 列的值，用它左边的邻居就能猜个八九不离十。**可预测，就是统计检测的弹药**：第 3 章的 RS 分析，本质就是度量“把像素按规则重排后，可预测性塌了多少”。

新手提示 |

“自然图像”这个词会反复出现。它的意思是：用真实相机拍出来的、未经人工合成的照片。天空渐变、皮肤纹理、草地噪声，这些真实世界的复杂性让像素的统计分布呈现出稳定规律；而计算机生成的纯色图、截图、二值图不在这个规律里。本项目数据集一栏反复强调“真实照片”，原因就在这里——第 8 章你会看到，在“非自然”图像上训练的检测器，换到真实照片上会水土不服。

图像里藏着大量冗余，这要从两个层面理解：

视觉冗余：人眼对微小亮度差、高频细节不敏感。把某个像素从 128 改成 129，几乎没人看得出来；

统计冗余：自然图像相邻像素高度相关，大部分信息集中在低频。JPEG 等压缩格式就是靠去掉这些冗余才把文件变小；

LSB 平面近似随机：自然照片的最低几位看起来“像噪声”，这恰好提供了天然的“掩护环境”。

于是出现了两类使用冗余的方式：压缩想把冗余去掉让文件变小，隐写想在冗余里塞秘密而尽量不破坏视觉与统计特征。二者争夺的是同一片“可修改空间”，这也是为什么很多隐写算法要针对特定压缩格式设计。

避坑提醒 | 不要以为“看不见 = 安全”。肉眼看不见只是最弱的要求；真正专业的检测者用统计工具，LSB 直接替换会留下非常明显的统计痕迹，这正是第 3 章卡方检验与 RS 分析能抓住它的原因。

只动最低位：肉眼几乎看不出，掩码把它标了出来



图 1-7 最低位翻转前后对比。这张图回答的是：改一个像素的最低位，肉眼看得出来吗？翻转最低位前后的图像与变化掩码：改动真实发生，肉眼却完全无差别

像素级放大：翻转最低位 = 把奇数改偶数、偶数改奇数

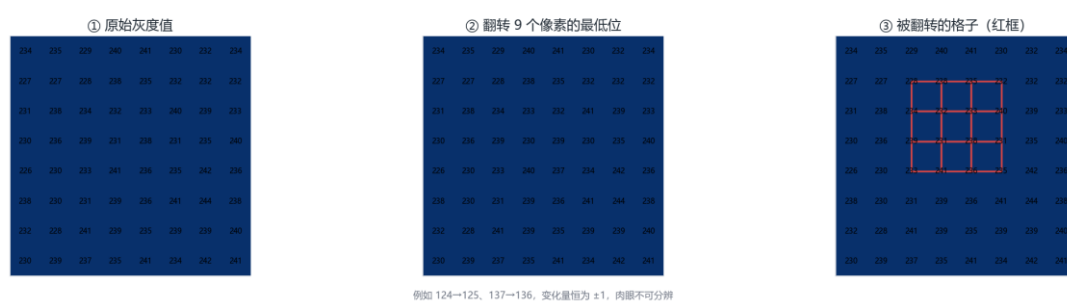


图 1-8 最低位翻转的像素级放大。这张图回答的是：藏进去的 1 bit，在数值上究竟改了什么？8 $\times$ 8 灰度值在翻转最低位前后的逐格对照——一格之差，就是那 1 bit

1.4 位平面：把一张图拆成 8 层

8 个位平面从 MSB 到 LSB，携带的“画面信息”逐层递减，这点可以用一个数字感受：把一张图的 MSB 平面单独拿出来看，你能认出照片的内容；看第 5 位平面，像蒙了一层细沙；看 LSB 平面，与电视雪花无异。信息量递减的另一面是**随机度递增**：自然图像的 LSB 平面相邻像素几乎不相关，与抛硬币无异。

这给了 LSB 隐写一层“天然的掩护”，也埋下它的“致命的破绽”。掩护是：往一片本来就随机的区域里再混入秘密比特，视觉上无从分辨；破绽是：**嵌入使得“值对”的分布变得均匀**——自然图像里 128 和 129 这对相邻值的出现次数原本并不相等（奇偶不平衡），均匀嵌入后它们被迫趋近相等。第 3 章的卡方检验，抓的正是这个趋近。

想一想 |

如果把一张图的 LSB 平面**整体**替换成伪随机比特（而不是只塞一点消息），直方图会怎么变？提示：每个像素的值都在“相邻两级”之间被随机重排。想不清就先记下这个问题，第 3 章的卡方一节会给出答案。

把灰度值写成 8 位二进制后，可以按“第几位”把图像拆成 8 张二值图，称为位平面。最高位（MSB）平面决定画面大轮廓；最低位（LSB）平面几乎全是细碎的噪声点。

这带来一个直觉：LSB 平面原本就接近随机，往里面再塞一点伪随机秘密比特，肉眼和简单直方图都很难察觉。但也正是这种“随机化”，改变了图像内在的结构，可以被更聪明的统计方法测出来。

数一数像素与 LSB 的取值（项目里到处是这样的 numpy 操作）

```
import numpy as np
img = np.asarray([[200, 201], [128, 129]], dtype=np.uint8)
print(img.shape, img.dtype)  # (2, 2) uint8
print(img & 1)               # LSB 位平面 [[0,1],[0,1]]
print(np.unpackbits(np.array([200], dtype=np.uint8)))
# [1 1 0 0 1 0 0 0] ← 最高位在前
```

位平面在 ImageMatrix 里没有现成的按钮，但原料唾手可得：把一张真实照片按 GRAY 导出 TXT，随便挑一列像素，把它们各自写成 8 位二进制（1.2 节的方法），再只看每一位那一列——最高位那一列几乎全是相同的 1 或 0（大轮廓），最低位那一列却像随机抛硬币。这正是本节 numpy 片段在做的事，只不过这次你先用眼睛看了一遍。



图 1-9 在 ImageMatrix 里把图片放大到 8 倍以上，会显示像素网格——你看到的每一个小格，就是矩阵里的一个数。所谓“位平面”，就是把这些数各自摊成 8 个二进制位后，按位重新叠出来的 8 张图。（截图来自 ImageMatrix 仓库）

一张 8bit 灰度图 = 8 个位平面叠起来

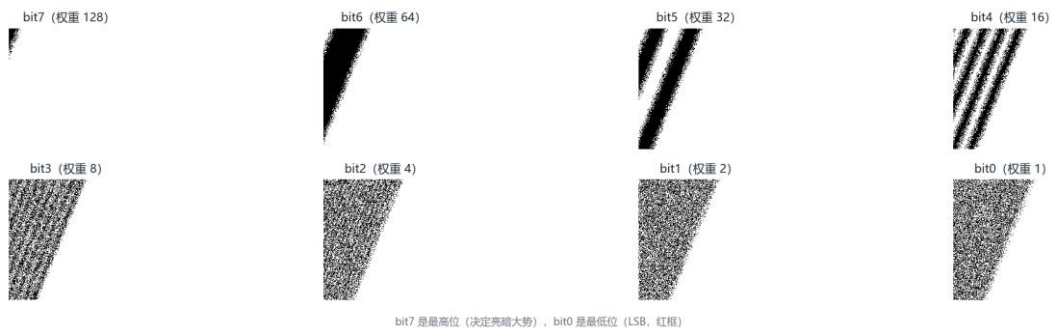


图 1-10 8 个位平面分解。这张图回答的是：把一张图拆成 8 层位平面，最低那一层为什么像噪声？载体图的 bit7...bit0 八个位平面；最低位平面已接近随机——这正是可藏之处

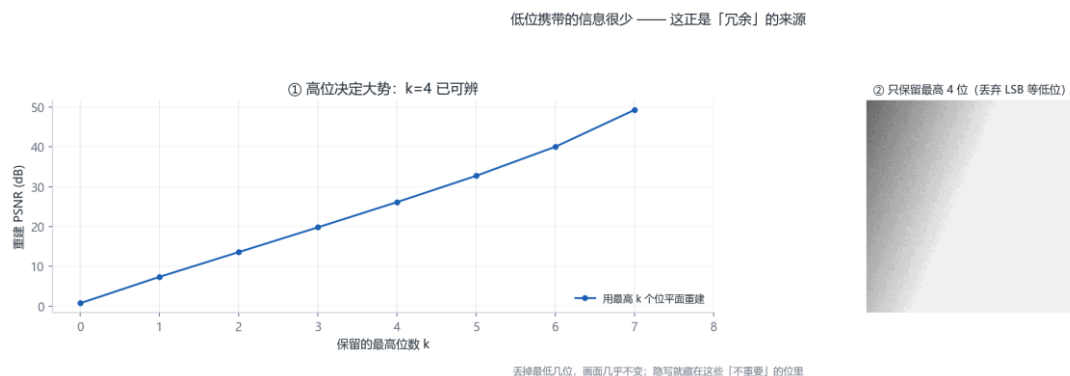


图 1-11 位平面权重与重建。这张图回答的是：丢掉低位平面，图像会损失什么？保留最高 k 位重建原图的质量

曲线与示例：高位管轮廓、低位管细节

1.5 回到项目：先认识 cover 与 stego

术语	英文	一句话解释	在项目里
载体	cover	没有秘密的原始图像	img/cover.png (256×256 演示图)
含密图	stego	藏入秘密后的图像	output/cover_stego.png / .jpg
嵌入	embed	把比特写进载体的过程	nsf5stego embed / GUI 「嵌入并保存」
提取	extract	从含密图还原比特的过程	nsf5stego extract / GUI 「解码提取」
隐写分析	steganalysis	判断一张图里有没有秘密	nsf5stego analyze / GUI 「分析」

这张术语卡片值得背下来——它同时是本手册的“变量命名表”。读代码时遇到 cover_img、stego、embed、extract，直接对号入座，不再当生词查。

隐写领域里有两个固定称呼：没有秘密的原始载体叫 cover（载体），藏入秘密后的图像叫 stego（含密图）。项目里这两个词随处可见：img/cover.png 是演示用载体，output/stego_*.png 是嵌入后的含密图。

回到项目看代码 | 打开 src/run_e2e.py 第 1 步：它用 numpy 生成一张 256×256 的平滑“封面图”并保存到 img/cover.png。仔细看生成方式——np.linspace 制造渐变背景，再加一点小噪声。理解这段代码，你就理解了一张自然感图像是如何从数字“长”出来的。

动手做 |

用记事本造出你的第一对 cover / stego

- ① 把一张图按 GRAY 格式导出 TXT——这份图就是 cover（载体）；
- ② 在 TXT 里，按事先约定的位置（例如“从第 1 个数起，每隔 3 个取一个”），把选中的数改成你想要的奇偶（要藏 1 就改成奇数、要藏 0 就改成偶数）；
- ③ 导入改过的 TXT，另存为 PNG——这份图就是 stego（含密图）；
- ④ 把 cover 与 stego 并排：肉眼分不出谁是谁。

你刚刚用记事本走完了第 3 章才正式讲的 LSB 隐写的全部动作；项目所做的，只是用代码、口令与矩阵编码把它做得更快、更隐蔽、且能被正确提取。

关于这个工具的边界（重要）。 ImageMatrix 是独立的姊妹项目，与本手册学习的 F:\Steganography 代码库没有任何依赖或调用关系；它只负责“把图看成数字、再把数字看回图”，本身不含任何隐写功能。把 cover 改成 stego 的那一步，是**你**在记事本里完成的——这正是本节想让你体会的：隐写的门槛不在工具，而在“知道该动哪一位”。

二维的「图」与一维的「比特串」是同一份数据

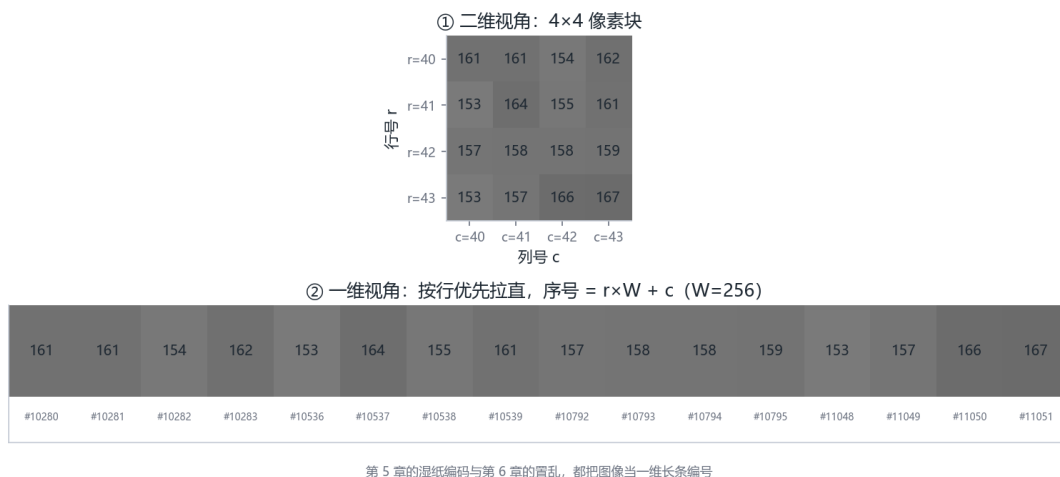


图 1-12 行优先编号：二维 → 一维。这张图回答的是：项目里说的“第 N 个载体”，是按什么顺序数的？4×4 像素块及其按行优先拉直后的一维编号——项目里“第 N 个载体”就是按这个顺序数的

1.6 小结与自我检查

灰度图 = 0 ~ 255 整数矩阵；彩色图 = 三个通道矩阵；

LSB 是像素值二进制的最低位，翻转它对视觉影响极小；

压缩消除冗余，隐写利用冗余，二者针锋相对；

cover = 原始载体，stego = 含密图。

想一想 | 一张纯黑色（全 0）的图适不适合做 LSB 隐写的载体？一张纯随机噪声图呢？把想法写下来，学完第 3 章再回来对照。

本章小结。三句话概括：图像是数字的表格（二维的看法），也是连续的字节串（一维的看法）；LSB 是数值变化最小、随机度最高的一位，是隐写的主战场；冗余是隐写的生存空间，视觉冗余（JND）保护你逃过人眼，统计冗余（相关性）是检测者的武器库。

动手做 |

章末练习（提示与答案见附录 J）

P1.1（手算）把 156 写成 8 位二进制；它的 LSB 是几？分别用“清零后置位”和“异或 1”两种方法把它的 LSB 变成相反值，写出新的十进制数。

P1.2（手算）像素行 [132, 133, 130, 129, 134, 131, 128, 135] 依次嵌入比特 1,0,1,1,0,0,1,1。写出嵌入后的 8 个像素值与每个像素的变化量。完成后与 3.2 节开头的表对照。

P1.3（思考）一张纯黑图（全 0）做 LSB 隐写，为什么“视觉上”依然没问题，但“统计上”立刻暴露？请用“值对分布”的语言回答。

P1.4（上机）运行 1.4 节的 numpy 片段，然后改动它：把 img 换成随机 1000×1000 数组，分别统计每个位平面中 1 的占比。哪个平面最接近 50%？

第 1½ 章 · JPEG 是什么，DCT 系数是什么？（压缩域隐写的战场）

本章导航 · 先看问题，再读正文

核心问题 为什么全世界的隐写教科书，讲 nsF5 时都在讲 JPEG 系数，而不是像素？

读完你能回答

- 画出 JPEG 压缩的七步流水线，并标出哪一步可逆、哪一步是损毁核心；
- 手算一个常数块的 DC 值，解释高频系数为什么会被量化归零；
- 口述系数上“载体 / 湿点 / 减幅”的含义，说清 $+3 \rightarrow +2$ 与 $+3 \rightarrow 0$ 的区别；
- 跑通 `--jpeg` 并读懂 `--json` 的关键字段（carriers_changed、capacity_bits 等）。

本章是第 1 章与第 5 章之间的桥：知道 LSB 的“死敌”是谁，也知道 nsF5 的“家”在哪里。

开场问题：为什么全世界的隐写教科书，讲 nsF5 时都在讲 JPEG 系数，而不是像素？ 第 1 章你已经知道像素 LSB 能藏东西。但真实世界里，图片在社交网络上传播的第一站就是重新压缩成 JPEG——像素域藏进去的秘密会在这次压缩里灰飞烟灭。要让秘密**活过压缩**，就必须住进压缩产物本身：量化后的 DCT 系数。这就是“压缩域隐写”名字的由来，也是 1990 年代末 F5 系列论文全部的故事。

这一章是 v1.9.0 新增的“编外章”，编号带个 ½，因为它在课程里是第 1 章和第 5 章之间的桥：先建立 JPEG/DCT 的直觉，第 3 章讲 LSB 时你会知道它的“死敌”是谁，第 5 章讲 nsF5 时你会知道它的“家”在哪里。读完本章，你不需要会推导 DCT 公式，但必须能不假思索地回答这张地图里的问题：

小节	回答的问题	你的产出
1.5.1 JPEG 流水线	秘密为什么藏不过重压缩？	能画出压缩七步并标出哪步可逆
1.5.2 DCT 系数	DC/AC、量化、非零系数是什么？	能手算常数块的 DC 并解释归零
1.5.3 系数上的 nsF5	载体/湿点/减幅在系数上长什么样？	能口述 $+3 \rightarrow +2$ 与 $+3 \rightarrow 0$ 的区别

小节	回答的问题	你的产出
1.5.4 真图实战	CLI/GUI 怎么跑 JPEG 域？	跑通 --jpeg 并读懂 --json 关键字段

新手提示 | 这一章写于 v1.9.0 (2026-10)，是应"学习路径应该先讲清楚 JPEG 和 DCT，再进入 LSB/隐写"的反馈补进来的。它编号是"1½"而不是把后面各章整体重排：手册已有大量"见第 N 章"的交叉引用、10 个按章 Notebook 和事实校验脚本，重编号的收益远小于风险。你只需要在读完第 1 章之后、进入第 3 章之前，把这一章读完。

第 1 章把图像拆成了像素与位平面，接下来按理就该讲 LSB 隐写了。但本项目的主线算法 nsF5 有两个"家"：主仓库 ns5_core.py 里的像素域实现（改灰度像素），以及 v1.9.0 起接入主线的 JPEG 压缩域实现（改量化 DCT 系数，来自姊妹项目 yccstego）。教科书里的 F5/nsF5 讲的是后者。不先弄懂 JPEG 和 DCT 系数，第 4、5 章里的"+3→+2"就永远只是口号。

1.5.1 JPEG 是什么：一台"有损压缩机"

把三步流水线展开成完整的七步，每一步标注“可逆吗”——这张表在后面无数地方会救你：

步骤	做了什么	可逆？	与隐写的关系
1 色彩空间	RGB → YCbCr（亮度/色度分离）	可逆（有舍入）	隐写只动 Y 通道（见下）
2 下采样	Cb/Cr 尺寸减半（4:2:0）	不可逆（丢分辨率）	色度载体也因此减半
3 分块	切成 8×8 的块	可逆	隐写以“块”为单位收集系数
4 DCT	像素 → 64 个频率权重	可逆（数学变换）	权重即“系数”
5 量化	权重 ÷ 量化表，四舍五入	**不可逆（有损核心）**	秘密就藏在这一步的产物里
6 之字扫描	二维系数 → 一维序列	可逆	决定“提取顺序”
7 熵编码	哈夫曼等无损压缩	可逆	nsF5 之后重写这一步，输出 .jpg

只动 Y 通道有两层理由。**容量理由**：自然图像的能量集中在亮度（Y）通道，量化后 Y 的非零 AC 系数远多于 Cb/Cr（色度量化的步长大，大量系数直接归零）——载体多，才藏得下消息。

教科书理由：Provos 与 Honeyman 在 2001 年提出的 F5 就是在 Y 通道上定义的，yccstego 沿用这一口径，论文对得上、复现得准。至于“改亮度会不会更容易被发现”——会，这正是第 8 章检测特征盯着 Y 通道直方图看的原因；隐写与检测永远在同一张地图上对弈。

JPEG 不是一种文件格式那么简单，它是一条有损压缩流水线，对每个 8×8 像素块依次做三件事：

1. DCT 变换：把 8×8 的像素值改写成 64 个“花纹权重”（下一节细讲）；
2. 量化：每个权重除以量化表里对应位置的一个步长，再四舍五入取整——有损就发生在这一步：右下角（高频）的步长大，除完取整后大量直接变成 0；
3. 熵编码：把剩下的非零系数按之字顺序串起来做压缩编码，写成 .jpg 文件。

第 3 步是可逆的（能一字不差地解回来），第 2 步不可逆（取整丢掉的信息回不来）。JPEG 压缩域隐写的全部艺术，就是让秘密住进第 2 步的产物——量化之后的系数——里。

避坑提醒 | 这也解释了第 3 章 FAQ 的那句话“JPEG 会破坏像素 LSB”：一张藏了像素 LSB 的 PNG 被另存为 JPEG 时，整条流水线重新跑一遍，量化取整会把 LSB 结构搅乱。反过来，JPEG 域隐写写的是量化系数本身，编码器会带着这些系数原样做完熵编码——所以它的含密图必须是 JPEG 自己，而且不能再被任何工具重新另存（重存等于重新量化，载荷即毁）。GUI 对 JPEG 域含密图按原始字节保存，就是这个原因。



图 1½-1 JPEG 压缩流水线。这张图回答的是：JPEG 把图像压小的七步里，秘密要住进哪一步？RGB → YCbCr → DCT → 量化 → 熵编码 的压缩链路中，秘密要住进量化后的系数

1.5.2 DCT 系数是什么： 8×8 块的“配方表”

把 8×8 块看成一幅微型画像。DCT（离散余弦变换）准备了 64 种固定的“花纹”：左上角的花纹是整块的均匀明暗（最低频），往右下走花纹越来越细密（高频）。DCT 变换做的事，就是回答一个问题：这一块像素，按每种花纹各用多少比例调配出来？得到的 64 个配比就是 DCT 系数。

- 左上角第一个系数叫 DC（直流），表示整块的平均亮度，不参与隐写；
- 其余 63 个叫 AC（交流）系数；
- 自然图像的能量集中在左上（低频），所以量化后右下角大批系数归 0——归零的系数不是载体，活下来的非零 AC 系数才是。

动手做 | 下面的片段用项目压缩域实现里同一套 DCT 矩阵与量化表（yccstego/dct.py），对同一个 8×8 块分别按质量 60、85、95 量化，数一数非零 AC 载体和 $|c|=1$ 的“湿点”各有多少。跑之前请确认已安装压缩域依赖：pip install yccstego。

```
import numpy as np
from yccstego import dct as jdct

# 造一个 8x8 块：平滑渐变(低频为主) + 轻噪声(高频细节)
rng = np.random.default_rng(3)
block = np.linspace(90, 200, 8)[None, :] + np.linspace(0, 60, 8)[:, None]
block = np.clip(block + rng.integers(-8, 9, (8, 8)), 0, 255).astype(float)

counts = {}
for q in (60, 85, 95):
    T = jdct.scale_qtable(jdct.LUM_QT, q)          # libjpeg 风格的质量缩放
    F = jdct.dct_blocks(block - 128)              # 中心化后做 DCT (JPEG 的做法)
    Q = np.round(F / T).astype(int)                # 量化：有损就发生在这一步
    ac = Q.reshape(-1)[1:]                          # 去掉 DC，剩 63 个 AC
    nz = int(np.sum(ac != 0))
    wet = int(np.sum(np.abs(ac) == 1))
    counts[q] = nz
    print(f"质量 {q}: 非零 AC 载体 {nz} 个， $|c|=1$  湿点 {wet} 个")

assert counts[95] > counts[60], "质量越高，活下来的载体应越多"
print(f"质量 95 的非零 AC 载体多于质量 60: {counts[95]} > {counts[60]}")
```

新手提示 | 观察 | 你会看到两个规律：质量从 60 提到 95，非零 AC 载体明显变多（量化步长变小，更多系数“活了下来”）；同时 $|c|=1$ 的系数不少——它们马上就要给你惹麻烦了。

DCT 不必会推导，但有一个数字应当手算一遍——它是所有 JPEG 隐写分析的锚点。实现里的标准做法是先把像素减 128（电平偏移）再变换：对 DCT-II（正交归一化），**一块完全相同的像素，变换后只有 DC 一个非零系数， $DC = 8 \times (\text{平均亮度} - 128)$** 。拿全 100 的块验证： $DC = 8 \times (100 - 128) = -224$ ，其余 63 个 AC 严格为 0。若把电平偏移补回去（加回 $8 \times 128 = 1024$ ），数值上等于 $8 \times \text{平均亮度} = 800$ ——两种口径在文献里都常见，本书统一按“实现口径”（先减 128）讲述。用项目的压缩域实现复算：

```
from yccstego import dct as jdct
import numpy as np
blk = np.full((1, 8, 8), 100.0)
F = jdct.dct_blocks(blk - 128.0)[0] # 电平偏移 -128 后变换
print(F[0, 0]) # -224.0 = 8 x (100-128)
print(np.abs(F[1:, :]).max()) # 0.0 —— AC 全零
print(jdct.scale_qtable(jdct.LUM_QT, 85)[0, 0]) # 5 (q85 亮度量化步长)
```

第三行打印的 5 是 q85 亮度量化表的左上角元素（标准表首项 16 经质量因子缩放得到）。于是这块的量化 $DC = \text{round}(-224 \div 5) = -45$ ——一个整数，抗住了取整。对比一个高频系数：量化步长动辄 60~100，除完直接归零。**载体 = 活下来的非零 AC**，这块一个都没有——纯色块藏不进东西，这就是最直观的解释。

想一想 |

把块从“全 100”换成“左半 100、右半 140”的水平渐变，DC 还是 800 吗？会有哪些 AC 活下来？

（提示：水平方向的“亮暗差异”对应某一系列低频 AC；块越平滑，活下来的越少。）拿 1.5.2 节的代码把块改成渐变跑一遍，对照你的预测。

把“ $DC = 8 \times (\text{平均亮度} - 128)$ ”从 DCT 的定义推出来。正交归一化 DCT-II 的第 (0,0) 个系数定义为（两个一维基的零频分量相乘，权重各为 $1/\sqrt{8}$ ）：

$$F(0,0) = \frac{1}{8} \sum \sum f(x,y) - 128 = 8 \times (\text{mean} - 128)$$

推导只有两步：常数块满足 $f(x,y) \equiv \text{mean} - 128$ ，双重求和共 64 项相加，得 $64 \times (\text{mean} - 128)$ ；再乘系数 $1/8$ ，即 $8 \times (\text{mean} - 128)$ 。代入手算例子：

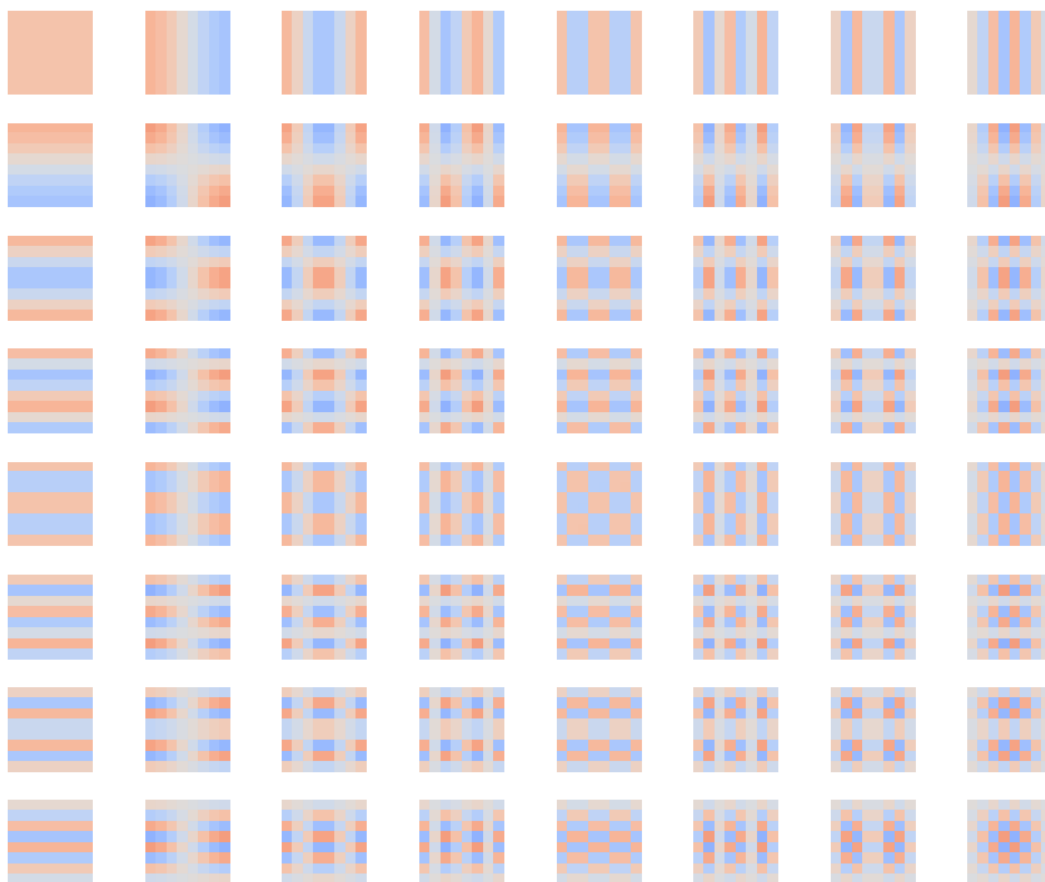
$$F(0,0) = 8 \times (100 - 128) = -224, \quad Q(0,0) = \text{round}(-224 / 5) = -45$$

量化表本身也不是天上掉下来的——libjpeg 用质量因子 q (1..100) 对标准表 T 做线性缩放：

$$S = 200 - 2q = 30 \quad (q = 85), \quad QQ_{ij} = \text{clip}(\text{floor}((TT_{ij} \cdot S + 50) / 100), 1, 255)$$

其中下标 ij 遍历 8×8 的 64 个位置。代 T 的首项 16 进去： $\text{floor}((16 \times 30 + 50) / 100) = \text{floor}(5.3) = 5$ ，正是代码打印的那个 5。顺带读懂这个公式的两个边界： $q < 50$ 时 $S > 100$ （步长放大、质量更低）， $q = 100$ 时 $S = 0$ 、缩放结果被 clip 钳在 1（全 1 表 = 不损失）。

8×8 DCT 的 64 张「配方表」：左上直流、越往右下频率越高



每个系数 = 空间块与对应基函数的内积；高频基函数被量化表重罚 → 多数归零

图 1½-2 DCT 基函数 (8×8)。这张图回答的是：为什么任何 8×8 块都能写成 64 个基函数的加权和？64 个 DCT-II

基函数：低频在左上、高频在右下，任何 8×8 块都是它们的加权和

量化表：低频罚得轻、高频罚得重——这就是「有损」的来源

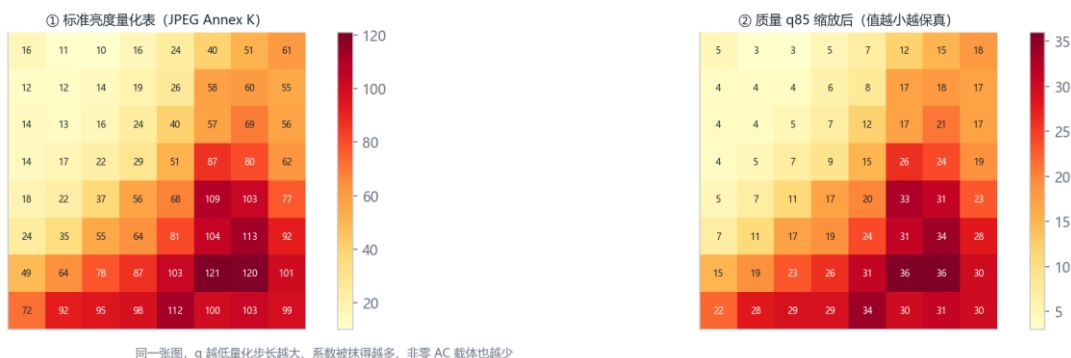


图 1 1/2-3 量化表与质量缩放。这张图回答的是：量化步长决定了哪些系数会被抹掉？标准亮度量化表与 q85 缩放后的步长——量化步长决定哪些系数会被取整归零

1.5.3 在系数上做 nsF5：载体、湿点与减幅

JPEG 域 nsF5 的载体系数是量化 Y 块（亮度）的非零 AC 系数，每个系数贡献一个比特： $x = c \& 1$ （奇偶）。往一个比特里写 1，不是把系数翻转，而是 F5 的“减幅”：幅值减 1、保符号。

原系数	修改后	说明
+3	+2	幅值减 1，奇偶从 1 变 0
-5	-4	保符号，同样翻 LSB
+1	湿点，不碰	再减就归 0（收缩），系数会从载体集合里消失
-1	湿点，不碰	同上

为什么“收缩”是灾难：提取端要按同样的顺序收集载体系数，一个系数从 +1 缩成 0 就从集合里消失，后面所有比特错位。这正是 F5 的老毛病，也是 nsF5 用湿纸编码（把 $|c|=1$ 的系数划为“湿点”、只在干点上解方程）修掉的问题——第 5 章展开。

动手做 | 三行代码看懂“减幅”与“湿点”的边界：

```
for c in (3, -5, 1, -1):
    action = c - (1 if c > 0 else -1) if abs(c) > 1 else "湿点，不碰"
    print(c, "->", action)
```

想一想 | 1. +3 和 +2 的二进制 LSB 都是 1，减幅之后 LSB 怎么会变？——提示：想想 +1、-1 与奇偶的关系，再想 $|c|=1$ 时为什么不能减。2. 与“翻转 LSB”（+3 与 +4 之间跳）相比，减幅让系数的幅值直方图在 $|c|=1$ 处堆积。这种堆积是伪装的破绽还是伪装的代价？第 8 章的“magnitude-1 指纹”会反过来用它抓隐写。

避坑提醒 | JPEG 域的口令键控与像素域同理：隐藏路由“色度系数哈希 + 口令”派生，解码端 p / 口令 / 质量（若重嵌）任一不一致都提取不出来。重跑契约两域一致：yccstego 0.2.0 起湿纸求解种子由输入派生，相同输入逐字节复现；档案记录建档时的 yccstego 版本，重跑同版本走字节级校验，跨版本自动退回“提取一致”（旧档兼容），这写在了 src/experiment.py 的文档里。

像素域与压缩域的容量对比，用项目演示图的真实数字最有感觉（256×256 演示图，p=3，质量 85，实测 2026-10-04）：

维度	像素域 (PNG)	JPEG 域 (质量 85)
候选载体	65 536 个像素（每个 1 位）	5 150 个非零 AC 系数（实测）
本次嵌入	240 bit → 改 108 像素 (0.2%)	240 bit → 改 117 个系数
理论容量	≈ 8 KB 文本	2 205 bit ≈ 275 字节文本
抗重压缩	差（重存即毁）	强（系数随位流存活）

注意容量从 8 KB 缩到 275 字节——**压缩域隐写用容量换鲁棒性**。这张表还解释了 GUI 里的一个现象：同样一句话，像素域轻松嵌入，JPEG 域却显示“改动系数 117 / 载体 5150”——不是 JPEG 域低效，而是它的载体总数本来就少两个数量级，2.3% 的相对改动率与像素域的 0.2% 属于同一档克制。

避坑提醒 |

JPEG 域的“湿点”是 $|c| == 1$ 的系数。它们一旦被减幅就变成 0，提取端收集载体时整条序列错位。0.2.0 版的 yccstego 曾有一个连作者都踩过的坑：编解码器的块划分与 DCT 定义有错，导致“消息能提取、但画面是品红条纹”。如果你在自己的实验里遇到“提取成功但图像坏掉”，先怀疑

编解码器的对称性（编码端怎么分块、解码端就要怎么收集），再怀疑算法。该案例的完整排查过程记录在 yccstego 仓库 issue #1，是极好的调试教学材料。

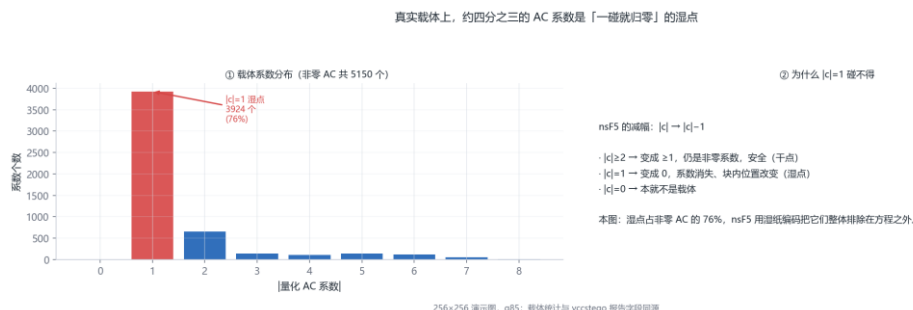


图 1½-4 DCT 系数分布与湿点。这张图回答的是：为什么 $|c|=1$ 的系数“碰不得”？量化 AC 系数的幅值分布与 $|c|=1$ 湿点占比：湿点碰不得，是容量与保真的共同约束

1.5.4 在真 JPEG 上跑一遍

前面两段是在“单块”上看原理。真正的嵌入由 yccstego 完成：整图分块、逐块量化、在全部载体上做伴随式编码、再熵编码回标准 .jpg。下面用桥接层把它跑通（GUI 的“JPEG 域”和 CLI 的 -jpeg 走的就是这同一份代码）：

```
import sys
sys.path.insert(0, "src")          # 源码运行时需要；pip 安装后可删掉这行
import numpy as np
import jpegstego

rng = np.random.default_rng(7)
y = np.linspace(0, 220, 128)[None, :]
x = np.linspace(0, 90, 128)[:, None]
gray = np.clip(110 + y + x + rng.integers(-6, 7, (128, 128)), 0, 255).astype(np.uint8)
rgb = np.stack([gray, gray, gray], axis=-1)      # JPEG 域吃 RGB

jpg, rep = jpegstego.embed_jpeg(rgb, "你好, DCT 域!", p=3, quality=85)
print("改动系数:", rep["carriers_changed"], "容量(bit):", rep["capacity_bits"])

msg, _, tampered, head_match = jpegstego.extract_jpeg(jpg, p=3)
assert msg == "你好, DCT 域!" and not tampered and head_match
print("往返一致:", msg)
```

命令行等价形式（安装后可用）：

```
nsf5stego embed cover.png --jpeg -m "秘密" --quality 85
nsf5stego extract cover_stego.jpg --jpeg
```

跑完 1.5.4 的代码，终端会吐出一段 JSON。它就是第 6 章要讲的“实验档案”，现在只需要认识五个字段：

JSON 字段	含义	演示图实测值
carriers_changed	被修改的系数个数	117
capacity_bits	当前参数下的理论容量	2 205 bit
cover_hash	封面色度哈希（篡改感知锚点）	de2f2560...
truncated	消息是否因容量不足被截断	false
yccstego_version	执行嵌入的引擎版本	0.2.1

其中 carriers_changed 与 capacity_bits 的比值 ($117/5150 \approx 2.3\%$) 就是嵌入率的直观体现；它远低于像素域上限——这不是缺陷，而是伴随式编码在“少改动”上换来的奖励，第 4 章专讲这笔账怎么算。

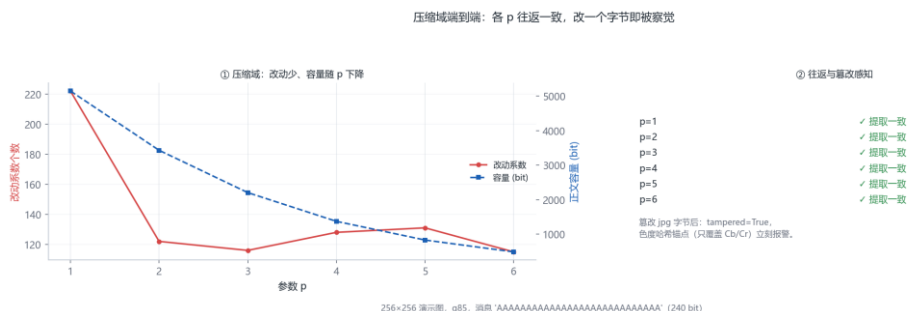


图 1½-5 压缩域往返与篡改感知。这张图回答的是：在真 JPEG 上嵌入再提取，一个 bit 都不会错吗？压缩域各 p 的改动系数、容量与提取一致性——往返全绿，篡改必失败

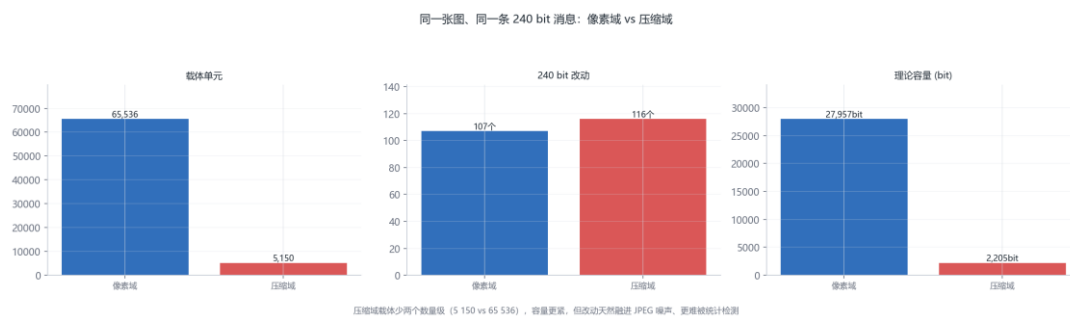


图 1½-6 像素域 vs 压缩域。这张图回答的是：同一个秘密，在像素域和压缩域里各要付什么代价？两种域下载体单元、改动数与容量的对比：同一张图，两个战场

1.5.5 回到项目看代码

概念	文件	优先阅读内容
桥接层（统一入口/降级提示）	src/jpegstego.py	embed_jpeg / extract_jpeg
块级 nsF5（减幅 + 湿纸）	yccstego/yccstego/nsf5.py	_embed_block / solve_wet_paper
DCT 与量化表	yccstego/yccstego/dct.py	_dct_matrix / scale_qtable / LUM_QT
JPEG 域分析（ c =1 指纹）	yccstego/yccstego/steganalysis.py	analyze_y
重跑契约（两域差异）	src/experiment.py	verify() 与模块文档
GUI 域切换	src/gui.py	_on_domain_change / _do_embed_jpeg

想一想 | 把同一张 128×128 的图分别用像素域（第 5 章的 nsF5Pixel）和本章的 JPEG 域嵌入，统计“可写位置”的数量：像素域约有 16000 个像素位，JPEG 域只有几千个系数——是什么决定了两者的差距？质量 60 时再算一次呢？这个容量直觉会在第 10 章的综合实验里派上用场。

本章小结。 JPEG 是一条以“量化”为损毁核心的流水线，秘密要活过重压缩就必须藏进量化后的系数；载体 = 非零 AC 系数（DC 不碰），湿点 = |c|=1（碰了就错位），修改方式 = 减幅（|c| 减 1、保符号）；容量比像素域小两个数量级，换来的是抗重压缩的鲁棒性。

动手做 |

章末练习（提示与答案见附录 J）

P1½.1 (手算) 全 100 的 8×8 块, 减 128 后 $DC = -224$ 。若 DC 量化步长是 5, 量化后的 DC 是多少? 反量化并还原电平后, 块的平均亮度是多少? (答案应 ≈ 100 , 量化误差 0.125。)

P1½.2 (思考) 系数 +3 改成 +2, LSB 从 1 变 0, 视觉无感; 为什么不能改成 +4 (4 的 LSB 是 0)? 从“幅值变化量”与“频率域影响”两个角度回答。

P1½.3 (上机) 跑 1.5.2 节代码, 把质量从 60 改到 95 (步长 5), 记录非零 AC 系数个数随质量的变化; 再回答: 质量越高, JPEG 域容量越大还是越小? 为什么?

P1½.4 (上机·选做) 把同一消息分别用像素域与 JPEG 域嵌入同一张演示图, 比较两个 JSON 里的 `carriers_changed` 与容量字段。

第 2 章 · 准备工具：Python、NumPy 与图像读写（第 2 周）

本章导航 · 先看问题，再读正文

核心问题 为什么这本手册不让你先学完 Python 再碰项目？

读完你能回答

- 让项目代码在你的机器上跑起来，`test\_core.py` 全绿；
- 解释 NumPy 数组的四个要素，并识别三个高频坑（uint8 回绕 / 视图不是拷贝 / 隐式升类型）；
- 读图、改像素、存回，并解释 PNG 往返无损与 JPEG 往返有损的分水岭；
- 读懂 `run\_e2e.py` 输出里的嵌入比特数与改动数。

你的产出不是“学会了 NumPy”，而是“项目代码跑起来了，且每一行数组操作都看得懂”。

开场问题：为什么这本手册不让你先学完 Python 再碰项目？ 因为隐写代码用到的 Python 子集小得惊人：数组切片、位运算、函数调用，仅此而已。CSAPP 的做法是“需要什么学什么，用到的那一周学透那一块”。本章把隐写代码 90% 会用到的 NumPy 概念压进一节，剩下的边用边查。读完本章，你的产出不是“学会了 NumPy”，而是**项目代码在你机器上跑起来，且你能读懂它的每一行数组操作。**

小节	回答的问题	你的产出
2.1 安装环境	怎么让项目跑起来？	test_core.py 全绿
2.2 NumPy 思维	数组四要素是什么？	能解释 uint8 回绕与视图陷阱
2.3 图像读写	image_io.py 封装了什么？	能读图改像素存回
2.4 端到端	run_e2e.py 的输出怎么读？	能找到嵌入比特数与改动数
2.5 报错速查	常见错怎么自救？	收藏这张表，报错先查它

动手做 | 本章目标：装好环境、跑通端到端脚本，会用 NumPy 读写图像数组。从此以后，你做的每个实验都能立刻在项目里验证。

2.1 安装环境

项目依赖很轻：核心只需要 NumPy 与 Pillow，机器学习部分再用 scikit-learn、joblib 等。建议用较新的 Python 3.9+（README 示例使用 3.14，低版本同样可用）。

在项目根目录安装并做第一次自检

```
cd F:\Steganography
pip install numpy pillow
python src\test_core.py      # 核心算法自测
python src\test_steg.py      # 隐写分析自测
python src\run_e2e.py        # 端到端：嵌入→解码→分析→绘图
```

避坑提醒 | 如果系统默认 python 没有 tkinter，GUI 无法启动。README 给出的办法是使用带 tkinter 的解释器（例如 C:\Python314\python.exe）；只跑命令行脚本则不受影响。遇到 ModuleNotFoundError 时先确认你安装到了“运行它的那个解释器”里：python -m pip install ... 更稳妥。

跑配套脚本前，先确认这四件工具都在

环境自检 · python tools/env_check.py

✓	Python	3.10.11
✓	NumPy	2.2.6
✓	Pillow	12.3.0
✓	Matplotlib	3.10.8
✓	中文字体	Microsoft YaHei

全部就绪 → 可以开始第 2 章的读图实验

图 2-1 环境自检面板。这张图回答的是：动手之前，怎么确认自己的环境是可复现的？Python、NumPy、Pillow、Matplotlib 版本与中文字体探测结果——第一件事就是让环境可复现

2.2 用 NumPy 思考图像

NumPy 数组的核心概念只有四个：形状（shape）、类型（dtype）、切片（slice）与向量化运算。隐写算法本质上都在做：取出数组 → 按某种顺序改写部分元素的 LSB → 存回数组。

四个核心概念的 10 行速览

```
import numpy as np
a = np.arange(12).reshape(3, 4)      # 3 行 4 列
print(a.shape, a.dtype)              # (3,4) int64
b = a[:, 1]                          # 取第 2 列
c = a[0:2, 0:3]                      # 左上角 2×3 块
x = np.zeros((64, 64), dtype=np.uint8)
x[10:20, 5:15] = 255                 # 画一个白色方块
print((x & 1).sum())                 # 统计 LSB=1 的数量
```

注意 dtype=np.uint8：图像像素是 0~255 的 8 位无符号整数。做加减时要小心溢出回绕：np.uint8(200) + 100 会得到 44 而不是 300。项目在翻转 LSB 时用“异或 1”而不是“±1”，正是为了避免把 0 减成负数或把 255 加溢出。

原文提到了 uint8 回绕。这里把隐写代码里**最容易踩的三个 NumPy 坑**一次配齐，每个坑配一行可复现代码：

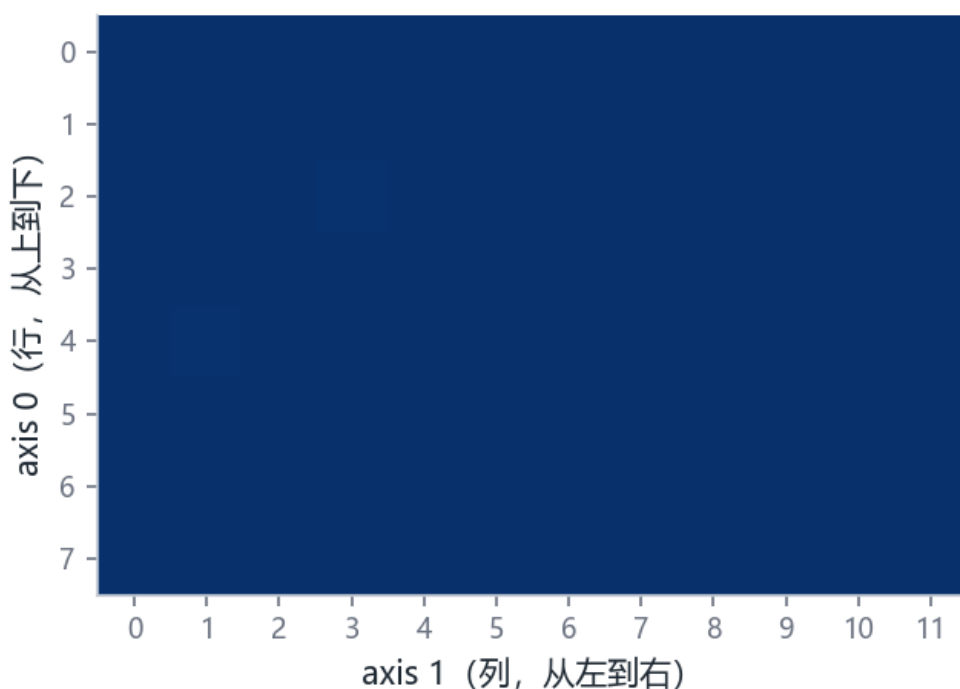
坑	复现代码	现象	解释
加法回绕	np.uint8(200) + np.uint8(100)	得到 44，不是 300	uint8 只存 0~255，300 回绕到 44；位运算不受影响，算术运算要当心
视图不是拷贝	b = a[:]; b[0] = 1	a[0] 也变成 1	切片返回视图（共享内存）；要独立副本必须 a.copy()
隐式升类型	a = np.array([200], np.uint8); a + 1	结果 dtype 变成 int64	与 Python 整数运算会提升类型，存回图像前要 astype(np.uint8)

为什么隐写代码特别容易踩这三坑？因为它全程在做“取出数组 → 改 LSB → 存回”，三步分别踩第三、第二、第一坑。项目源码的防御姿势值得抄：位运算用 ^1（不触发算术回绕），跨函数传递一律显式 dtype，写文件前统一 clip 与 astype（见 image_io.py）。

想一想 |

`a = np.zeros((2,2), np.uint8); b = a[0]; b[0] = 200` 之后，`a` 是多少？如果想要 `b` 是独立副本，正确的写法是什么？——这个坑在第 5 章的湿纸求解里真实存在过：解出的修正位写回载体数组时若用了视图，会连湿点一起改掉。视图问题从来不是“Python 语法问题”，是数据归属问题。

NumPy 里一张灰度图就是一个二维数组：`img.shape = (H, W)`



`img[r, c]` 取第 `r` 行第 `c` 列的像素；行号沿 `axis 0` 增长，列号沿 `axis 1` 增长

图 2-2 数组形状与索引。这张图回答的是：`axis 0` 和 `axis 1` 分别指向哪一行哪一列？二维数组的 `axis 0`（行）与 `axis 1`（列）方向示意——搞错轴是新手最贵的错

选对 dtype: 像素用 uint8, 运算用 int16/float



读图得到 uint8; 任何加减乘除先升到 int16 或 float64, 最后再 clip 回 uint8

图 2-3 dtype 取值域对照。这张图回答的是: uint8 越界之后到底会发生什么? uint8 / int16 / float64 的取值域与典型用途: 越界即回绕, 隐写里是事故高发区

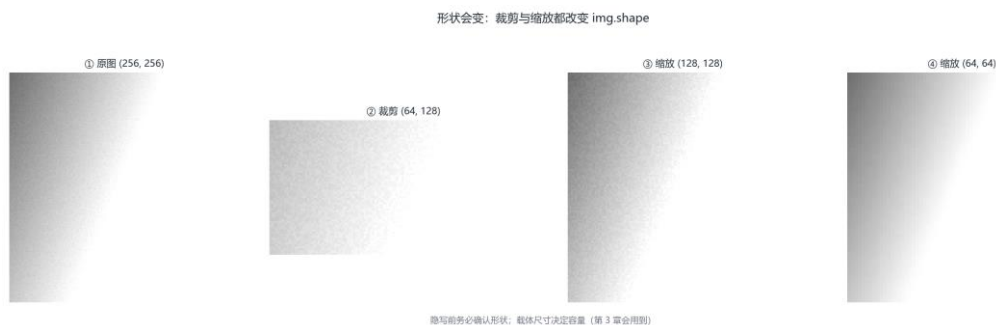


图 2-4 缩放与裁剪的形状变化。这张图回答的是: 改尺寸, 究竟改了矩阵的什么? 原图、裁剪与两种缩放的数组形状对比——改尺寸就是改矩阵的行列

2.3 图像读写: 认识 image_io.py

project/src/image_io.py 的核心接口

```
from PIL import Image
import numpy as np

def load_as_gray(path):
    im = Image.open(path).convert("L")    # 强制转灰度
    return np.asarray(im, dtype=np.uint8)

def save_image(image, path):
    im = Image.fromarray(np.ascontiguousarray(image))
    im.save(path)
```

回到项目看代码 | 自己打开 `src/image_io.py` 全文，注意 `load_image()` 与 `load_as_gray()` 的差异，以及 `SUPPORTED` 支持的扩展名。然后运行下面这段，读入项目自带的封面图：

动手：读图、看形状、存灰度版

```
import sys; sys.path.insert(0, "src")
import image_io as IO
img = IO.load_as_gray("img/cover.png")
print(img.shape, img.dtype)          # (256,256) uint8
print(img.min(), img.max(), img.mean())
IO.save_image(img, "output/my_copy.png")
```



图 2-5 PNG 往返无损验证。这张图回答的是：存成 PNG 再读回来，像素会变吗？原图、PNG 往返结果与逐像素差异：全零差异，证明 PNG 往返无损

用 SHA-256 给「无损」下定义：哈希一致才算真的没变

原始读入	一致 ✓
9add44bee9280a5e9804fbfc78450346f93c537f3b937c796502a72f4727c17a	
PNG 往返后	一致 ✓
9add44bee9280a5e9804fbfc78450346f93c537f3b937c796502a72f4727c17a	
JPEG q85 往返后	不一致 ✗
66015fdd54b0923ce695f94c96606a7a5e5707c11309df97351e6df5350826b5	

PNG 往返哈希不变；JPEG 往返哈希必然改变（有损）

图 2-6 往返一致性（SHA-256）。这张图回答的是：怎么用一个哈希值判断“无损还是有损”？原图、PNG 往返、JPEG 往返三者的 SHA-256 是否一致——无损与有损的分水岭

2.4 第一次端到端运行：run_e2e.py

以 2026-10-04 在本机的一次真实运行为例，把输出里的关键数字钉在墙上，你跑出来的数应当与它完全一致（演示图与参数是确定性的——这正是第 6 章“可复现”的第一次亮相）：

输出行	数字	怎么读
嵌入 504 比特	504	消息 63 字节 $\times 8 = 504$ ；比特数只跟消息长度有关
改动 188 / 183 像素	0.29%	空口令与有口令两次嵌入的改动数不同——口令改变了嵌入位置
分析 封面 prob=0.08	Gn=0.56	干净图被判“不太可能含密”（第 3 章解释 prob 的来历）
效率表 p=3 行	$\alpha=3.38$	平均每改 1 个像素塞进 3.38 比特；理论值 $24/7 \approx 3.43$ ，见第 4 章

这张表里的“效率表”值得多看一眼：p 从 1 扫到 6，改动数从 32502 一路降到 1035，而嵌入比特数不变。**矩阵编码用冗余度换改动数**——这笔交易的定价公式就是第 4 章的主角。把 6 行数字抄进实验日志，第 4 章回来对账。

运行 `python src/run_e2e.py`，脚本会依次完成五件事：生成封面图 → 以 nsF5 p=3 嵌入一段英文消息（分别用空口令与口令）→ 解码比对 → 对干净图与含密图做盲隐写分析 → 绘制码族/效率图。

动手做 | 仔细阅读终端输出，找到三组数字并抄下来：嵌入比特数、改动像素数、干净图与含密图的分析概率。下一章你就能解释它们为什么不同。

你现在还看不懂嵌入内部不要紧——第 2 周的目标只是“让代码在你机器上跑起来”，以及建立“数组改 LSB”的直觉。

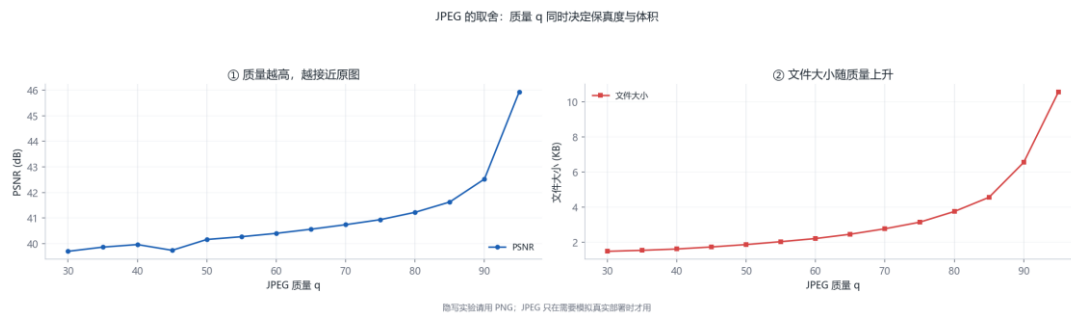


图 2-7 JPEG 质量-保真-体积曲线。这张图回答的是：JPEG 质量、体积、保真三者怎么互相拉扯？JPEG 质量与 PSNR、文件大小的关系曲线——质量、体积、保真三者互相拉扯

2.5 常见报错速查

报错 / 现象	原因	处理
ModuleNotFoundError	包装到了别的解释器	用运行脚本的解释器执行 <code>python -m pip install</code>
图像太小, 无法容纳头部+正文	像素数少于消息容量	换更大的图, 或调小 p / 缩短消息
UnicodeDecodeError / 乱码	旧版解码端按 ASCII 解码 (v1.8 之前)；或提取端与嵌入端编码不一致	升级到 v1.8+：encode_string() 已按 UTF-8 编解码，中文可直接嵌入；旧档案请用建档时的版本提取
GUI 秒退 / TclError	解释器无 tkinter	换带 tkinter 的 Python 运行 <code>src/gui.py</code>
想一想 为什么 image_io 保存时要把数组转成“连续数组” (np.ascontiguousarray)？提示：切片与转置可能产生非连续内存视图，很多底层库不喜欢它们。	想一想 为什么 image_io 保存时要把数组转成“连续数组” (np.ascontiguousarray)？提示：切片与转置可能产生非连续内存视图，很多底层库不喜欢它们。	想一想 为什么 image_io 保存时要把数组转成“连续数组” (np.ascontiguousarray)？提示：切片与转置可能产生非连续内存视图，很多底层库不喜欢它们。



图 2-8 常见坑一：通道顺序。这张图回答的是：为什么同一张图在不同库里颜色会反？RGB 与 BGR 解读下的颜色错位对比——读图库的默认通道顺序并不统一

坑二：dtype。uint8 的 $200 + 100$ 不是 300，而是 44

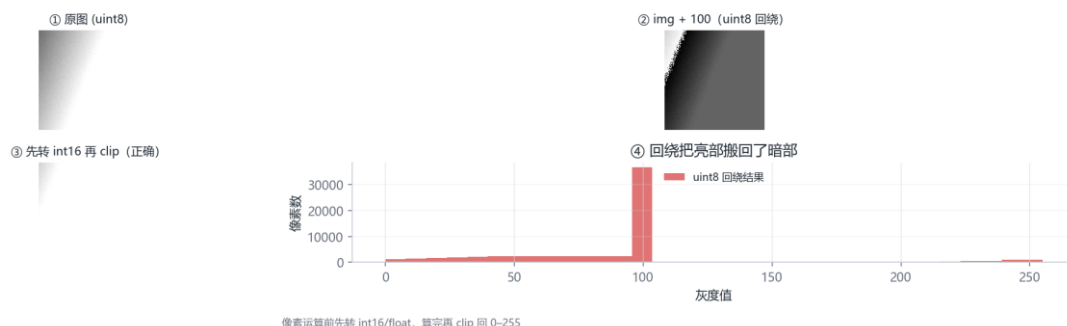


图 2-9 常见坑二：dtype 溢出。这张图回答的是：250 + 10 为什么会变成 4？uint8 加法回绕与正确裁剪的结果对比及直方图

坑三：JPEG 有损。同一张图存成 JPEG，像素就回不去了

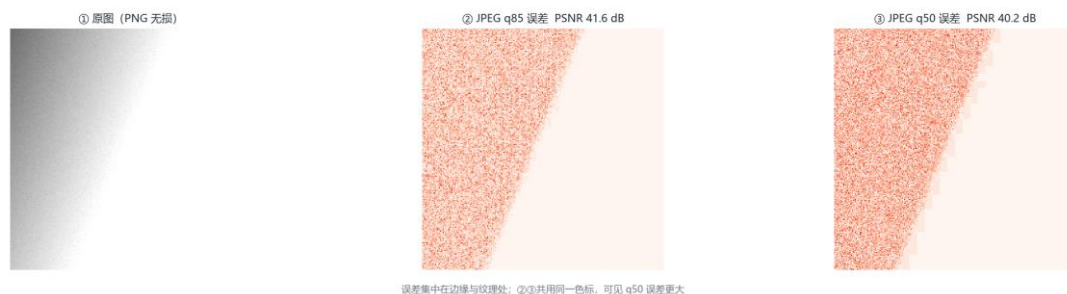


图 2-10 常见坑三：JPEG 有损。这张图回答的是：为什么像素域藏的秘密过不了 JPEG 这一关？JPEG 压缩引入的逐像素误差与 PSNR——像素域的秘密过不了这一关

本章小结。 环境三件套（NumPy/Pillow/项目源码）装好即可，不必精通；NumPy 四要素里对隐写最要命的是 dtype 与视图；run_e2e.py 是全项目的“总开关”，以后每学一个新概念都回到它找对应数字。

动手做 |

章末练习（提示与答案见附录 J）

P2.1（手算）`np.uint8(250) + np.uint8(10)` 等于多少？`np.uint8(3) - np.uint8(5)` 呢？先笔算再上机验证。

P2.2（上机）按 2.2 节“视图陷阱”的复现代码敲一遍，再写出获得独立副本的两种写法，并用 `a.base is None` 验证。

P2.3（上机）把 `run_e2e.py` 的消息改成 126 字节（比如把英文句子复制一份），重跑并记录嵌入比特数与改动像素数。比特数为什么正好翻倍？改动数为什么不是？

P2.4（上机·选做）把 2.3 节读入的演示图做 `img[:, ::2]` 水平抽样后保存，用肉眼与原图的差异——这就是“视图操作”最直观的教具。

第 3 章 · LSB 隐写与它的统计“指纹”（第 3—4 周）

本章导航 · 先看问题，再读正文

核心问题 把秘密藏进 LSB，凭什么说自己“看不见”？又凭什么会被人抓出来？

读完你能回答

- 写出最小 LSB 嵌入与提取的完整流程（逐位替换、逐位读出）；
- 解释卡方检验抓的是什么——**偶奇配对被人为拉平**；
- 解释 RS 分析的“结构塌缩”，并读懂 Gn / Gr 缺口随嵌入率的变化；
- 用 `analyze()` 的综合概率给一张图下判定。

藏与查是同一枚硬币的两面：本章每讲一个藏法，就配一个抓它的统计量。

开场问题：下面两行各 16 个字节，其中一行藏了消息——你能看出来是哪一行吗？第一行来自一张干净照片的字节流，第二行是同一张照片做了 LSB 嵌入后的字节流。盯着看一分钟。看不出来是正常的：嵌入把 114 改成 115、把 119 改成 118，每一个字节都只挪了 1，视觉上与照片的天然噪声无法区分。但统计学家会告诉你：第二行的“指纹”已经变了。本章教你像统计学家一样看图——这也是整个隐写分析领域的起点。

```
cover: 105 114 113 111 112 119 110 119 112 112 118 125 123 124 125 126
stego: 105 115 113 110 112 118 110 118 112 113 119 125 123 125 125 127
```

（两行都取自主项目 img/cover.png 最前面 16 个像素；第二行由 lsb_embed(rate=1.0, seed=0) 生成，可用配套项目 StegoLab 的 ch03 一键复现。16 个字节里有 9 个被改动，肉眼依然分不出哪一行是含密图。）

小节	回答的问题	你的产出
3.1 加密 vs 隐写	它们解决的是同一个问题吗？	能说清威胁模型的区别
3.2 最小 LSB	最朴素的嵌入长什么样？	能手算 8 像素行的嵌入
3.3 卡方检验	怎么用直方图抓 LSB？	能手算一个小卡的 χ^2 与 p 值
3.4 RS 分析	不比直方图还能比什么？	能手算一组的平滑度变化
3.5 综合判读	多个信号怎么合成一个结论？	理解灵敏度=操作点选择

动手做 | 本章目标：亲手完成一次“藏进去→检测出来”的最小闭环；理解卡方检验与 RS 分析各自抓住了什么；看懂 steganalysis.py 的判读逻辑。

3.1 加密 vs 隐写：先回答第一个问题

维度	加密 (Encryption)	隐写 (Steganography)
保护对象	消息内容：没有密钥就读不懂	通信行为：别人看不出有秘密
可见性	密文明显“不像正常数据”	载体看起来完全正常
典型场景	传输密码、证书、加密文件	隐蔽信道、版权水印、取证研究
相互关系	可先加密再隐写（强烈推荐）	隐写隐藏的是“秘密存在”这一事实
新手提示 一句口诀：加密保护“内容”，隐写保护“存在”。专业隐写系统通常两者都用：先把消息加密，再把密文藏进载体。加密后密文近似随机，反而更不容易在统计上露馅。	新手提示 一句口诀：加密保护“内容”，隐写保护“存在”。专业隐写系统通常两者都用：先把消息加密，再把密文藏进载体。加密后密文近似随机，反而更不容易在统计上露馅。	新手提示 一句口诀：加密保护“内容”，隐写保护“存在”。专业隐写系统通常两者都用：先把消息加密，再把密文藏进载体。加密后密文近似随机，反而更不容易在统计上露馅。

3.2 最小可运行的 LSB 隐写

拿第 1 章练习 P1.2 的那行像素把账算完。嵌入前 [132, 133, 130, 129, 134, 131, 128, 135]，依次写入比特 1,0,1,1,0,0,1,1，嵌入后 [133, 132, 131, 129, 134, 130, 129, 135]——8 个像素里改了 6 个，每个都只动 ±1：

像素序号	1	2	3	4	5	6	7	8
嵌入前	132	133	130	129	134	131	128	135
要嵌的位	1	0	1	1	0	0	1	1

像素序号	1	2	3	4	5	6	7	8
嵌入后	133	132	131	129	134	130	129	135
变化量	+1	-1	+1	0	0	-1	+1	0

两个观察，一个是好消息一个是坏消息。好消息：**提取与嵌入完全对称**——对嵌入后的像素逐个取 & 1，就得到 1,0,1,1,0,0,1,1，一个不多一个不少。坏消息：**平均嵌入 1 位要改 0.75 个像素**（8 位改了 6 个）。当消息很长时，改动密度逼近 0.5/像素（一半像素 LSB 本来就对，一半要改）——第 4 章的矩阵编码就是为了把这个数字打下去。

新手提示 |

本例中第 4、5、8 个像素“零改动”纯属运气：它们的 LSB 恰好等于要嵌的位。LSB 替换有个隐藏的不对称——嵌入比特 1 时，偶数像素 +1、奇数像素不动；嵌入比特 0 时相反。消息位如果是均匀随机的，偶数像素被向上推、奇数像素被向下拉的概率各半，长此以往，“偶灰度→奇灰度”与“奇灰度→偶灰度”两个方向的流动把每一对 $(2i, 2i+1)$ 的频数推向相等。记住这句话，3.3 节见。

最朴素的隐写叫 LSB 替换（LSB replacement）：把秘密比特串逐位写进像素的 LSB。灰度图有 $M \times N$ 个像素，就能藏 $M \times N$ 个比特。

教科书版最小 LSB（仅用于理解；项目实际用的是第 4–5 章的矩阵编码）

```
import numpy as np

def text_to_bits(s):
    raw = s.encode("utf-8")
    # 16 bit 长度头(字节数) + 每个字节 8 bit
    head = [(len(raw) >> i) & 1 for i in range(16)]
    body = np.unpackbits(np.frombuffer(raw, np.uint8))
    return np.concatenate([head, body]).astype(np.uint8)

def lsb_embed(cover, bits):
    flat = cover.ravel().copy()
    k = min(len(bits), flat.size)
    flat[:k] = (flat[:k] & 0xFE) | bits[:k] # 清最低位再写入
    return flat.reshape(cover.shape)

def lsb_extract(stego, nbytes):
    flat = stego.ravel() & 1
```

```
return bytes(np.packbits(flat[16:16 + nbytes*8]))
```

避坑提醒 | 有损格式是 LSB 的天敌。PNG/BMP 无损保存，位面不会变；JPEG 有损压缩会把 LSB 打得面目全非。所以项目默认保存 PNG，而 JPEG 域隐写需要另做一套（姊妹项目 yccstego 走的正是量化 DCT 系数路线：<https://github.com/Yukinoshita-lin/yccstego>，独立仓库与 PyPI 包，不在本仓库内）。

为什么消息开头要先放 16 bit 长度头？因为解码端必须知道“读多少字节”才能还原消息。项目 src/ns5_core.py 的 encode_string() 正是先写 16 位小端长度、再写 UTF-8 字节——你在 3.2 的例子中已经把这个结构复制了一遍。



图 3-1 最小 LSB 嵌入流程。这张图回答的是：把一个 bit 藏进 LSB，最小的一步动作是什么？8 个像素逐位嵌入的算例：嵌入前 / 目标位 / 嵌入后，一步一步看清最小动作

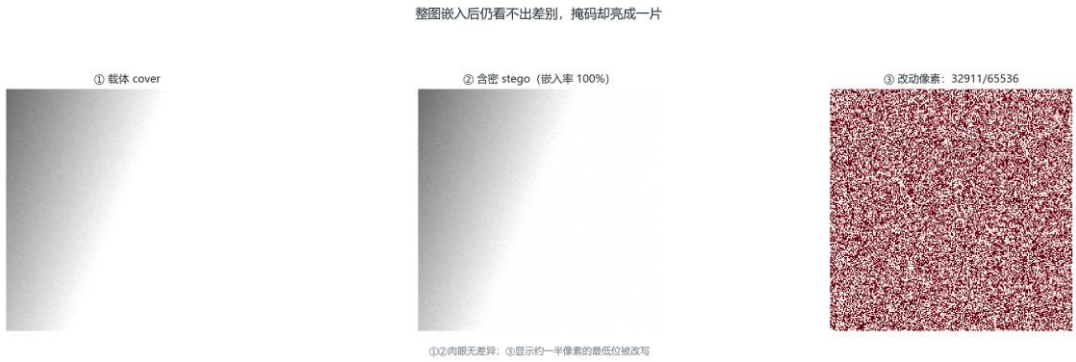


图 3-2 嵌入前后图像对比。这张图回答的是：藏完之后，改动集中在哪些位置？载体、含密图与改动像素掩码——改动集中在少数位置，视觉无痕

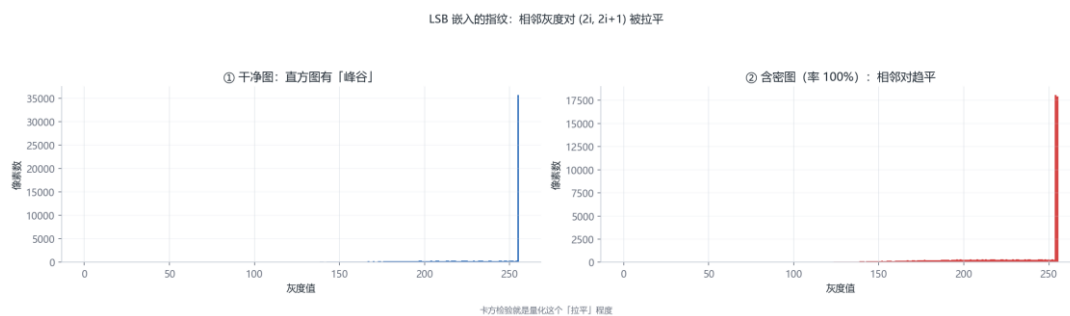


图 3-3 灰度直方图对（干净 vs 含密）。这张图回答的是：嵌入在统计上留下了什么指纹？嵌入前后的灰度直方图：偶奇配对被拉平，这就是统计指纹的源头

3.3 为什么会露馅：卡方检验

自然图像里，相邻灰度 $2i$ 与 $2i+1$ （例如 100 与 101）出现的次数通常不相等。LSB 替换会强制把偶数与奇数“配对拉平”：当某像素从 $2i$ 改成 $2i+1$ 或反过来，两个灰度的频数会趋向均衡。

Westfeld 的卡方检验做的就是这件事：

统计整张图的 256 级灰度直方图，按相邻对 $(0,1)$ 、 $(2,3)$ 、...、 $(254,255)$ 配对；

假设“已嵌入”，则每对的两个频数应近似相等，期望值取对之和的一半；

计算统计量 $\sum(\text{观测}-\text{期望})^2/\text{期望}$ （只统计频数和大于 0 的灰度对）；

把统计量换算成 p 值： p 高表示“观测与均匀假设吻合”→ 疑似 LSB 随机化；

项目还会把图切成 20 个前缀段分别计算 p 值，取中位数，得到更稳的判据。

注意 p 值的含义容易反：这里 p 越大越可疑，因为它在说“如果 LSB 已被完全随机化，看到这种分布的几率很大”——而干净的平滑图通常不会这么均匀。

避坑提醒 | 天然噪声图会骗过卡方检验。数码照片本身 LSB 就接近随机，卡方 p 天然很高。所以项目引入了“内容本底随机度”修正：先用灰度差分熵估计图像本身有多‘随机’，再决定是否把高 p 当作嵌入证据。这是把启发式做成可用的关键工程细节。

原理原文已经讲了（配对、期望、统计量、 p 值）。现在按 CSAPP 的方式把它落到一个能笔算的例子。想象一张极小的图，只统计前 4 个灰度对——也就是灰度 $(0,1)$ 、 $(2,3)$ 、 $(4,5)$ 、 $(6,7)$ 。数出来的频数是：

灰度对	(0,1)	(2,3)	(4,5)	(6,7)
偶灰度频数 n_{2i}	12	6	9	3

灰度对	(0,1)	(2,3)	(4,5)	(6,7)
奇灰度频数 n_{2i+1}	4	6	9	11
对和 n_i	16	12	18	14
期望 $n_i/2$	8	6	9	7
偏差 $n_{2i} - n_i/2$	+4	0	0	-4

统计量 $\chi^2 = \sum (\text{偏差})^2 / \text{期望} = 4^2/8 + 0 + 0 + (-4)^2/7 = 2 + 2.286 = 4.286$ ，自由度 $= 4 - 1 = 3$ 。用项目 steganalysis.py 自带的 chi2_sf 查尾概率： $\text{chi2_sf}(4.286, 3) = 0.232$ 。怎么判读？若这张图从未被 LSB 嵌入，看到“偏差这么大”的偶奇分布并不罕见（ $p=0.23$ ，约四分之一的概率）；但若把这张图重新数一遍——注意第 1 对与第 4 对的偏差方向**恰好相反**（一个偶多奇少、一个偶少奇多）——这更像自然照片的天然不平衡，而非嵌入的系统性拉平。把整张图的 128 个对全部统计进来再算一次，如果 p 仍然居高不下，才有资格说“疑似嵌入”。

想一想 |

为什么自由度是“对数 - 1”而不是“对数”？提示：每一对里，知道偶灰度的频数，奇灰度的频数就被“对和”锁死了——独立信息少一份。统计学里的自由度，数的就是独立信息的份数。

新手提示 |

原文最后一句强调“ p 值高 $\neq$ 安全”。补一个工程细节：项目把直方图切成 20 个**前缀段**（前 1/20 的灰度、前 2/20……）分别算 p ，再取中位数。为什么？卡方对“整体均匀”敏感，对“局部嵌入”迟钝——攻击者只在图像下半部分嵌入，整图统计会被上半部分的天然不平衡稀释。前缀分段让局部随机化无处藏身，中位数又避免单段噪声翻转结论。这是“一个统计量不够就造一族”的经典工程手法。

最后把这次手算写成一般的式子。记第 i 个灰度对为 $(2i, 2i+1)$ ，偶灰度频数 n_{2i} 、期望频数 $m_i = n_i/2$ （对和的一半），则统计量与自由度为：

$$\chi^2 = \sum \frac{(n_{2i} - m_i)^2}{m_i}, \quad df = k - 1$$

其中 k 是频数和大于 0 的灰度对个数，自由度取 $k - 1$ 的原因在 3.3 节的“想一想”里。p 值按“假设成立时，看到不小于该统计量的概率”计算——本项目用不完全贝塔函数实现 (steganalysis.py 的 chi2_sf)：

$$p = P(X \geq \chi^2), X \sim \chi^2(df)$$

代入手算例子： $\chi^2 = 4.286$ 、 $df = 3$ ， $\text{chi2_sf}(4.286, 3) = 0.232$ 。再说一遍方向：p 越大，观测越像“已经均匀化”，越可疑——这与常见假设检验“p 小才拒绝”正好相反，因为这里的原假设被换成了“已嵌入”。

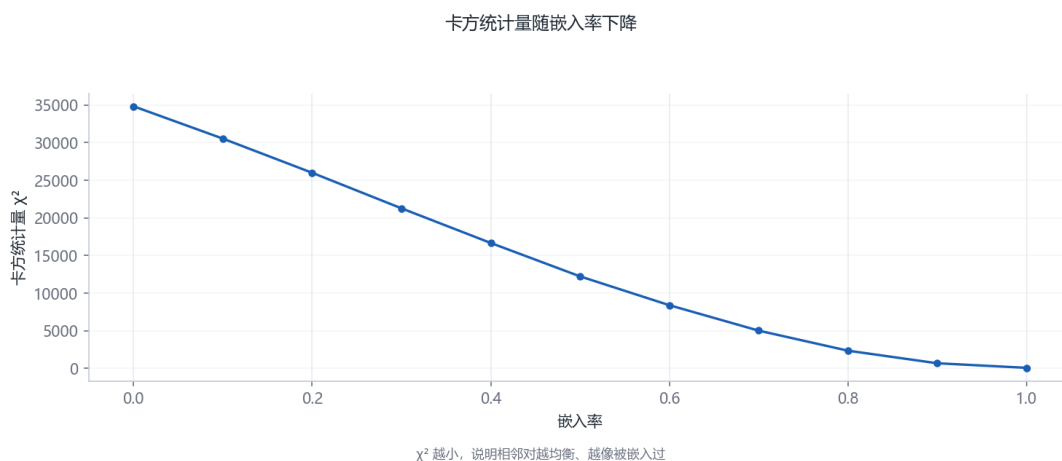


图 3-4 卡方统计量 vs 嵌入率。这张图回答的是：藏得越多，卡方统计量会怎么变？ χ^2 随嵌入率的变化——藏得越多，配对越平，统计量越低

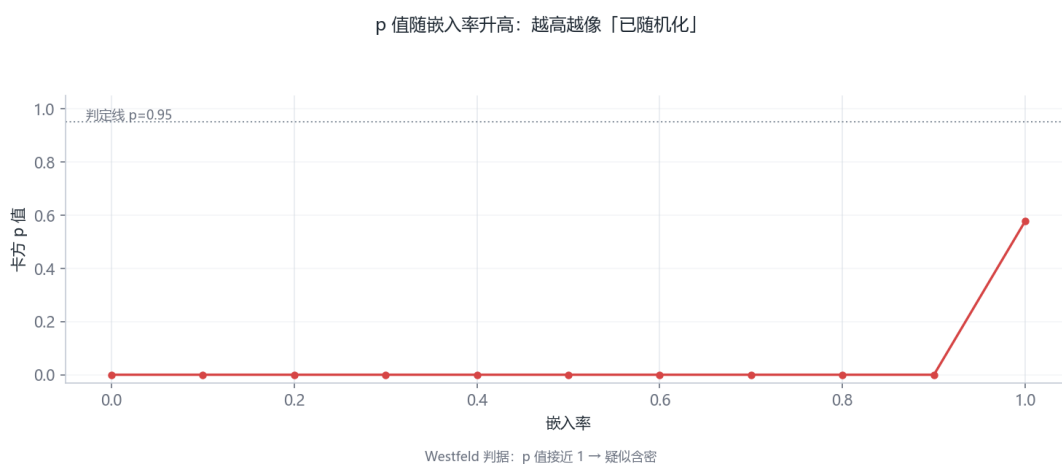


图 3-5 卡方 p 值 vs 嵌入率。这张图回答的是：卡方 p 值到多少就该报警了？卡方 p 值随嵌入率的变化：p 值趋近 1 就是“疑似随机化”的报警

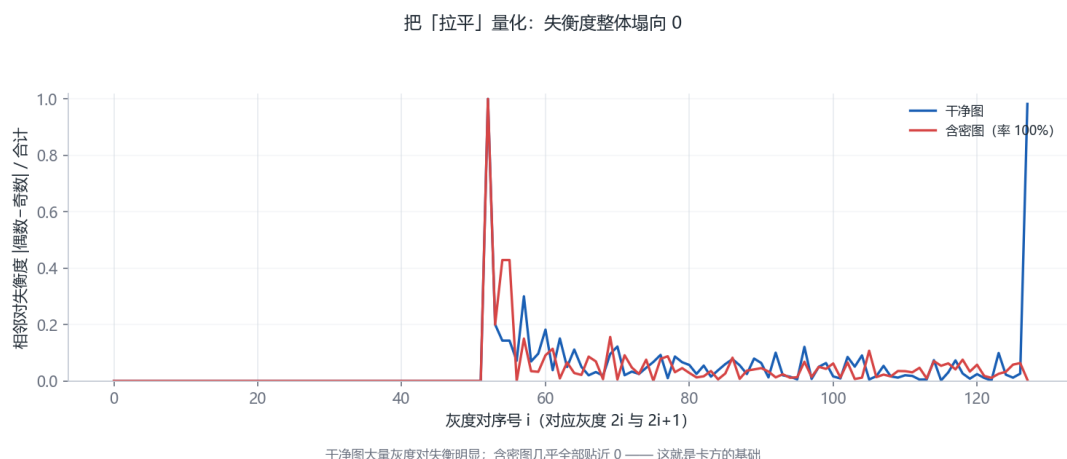


图 3-6 相邻灰度对失衡曲线。这张图回答的是：什么样的图，嵌入后的信号最强？干净图与含密图的相邻灰度对失衡度对比——本底越不平衡，信号越强

3.4 RS 分析：LSB 的“结构塌缩”

Fridrich 等人的 RS 分析换了一个角度：不比较灰度对频数，而是观察 LSB 位面的空间结构。做法是把像素流切成 4 像素一组，定义一个平滑度判别式 f （组内相邻差绝对值之和），再分别用“正掩码”与“负掩码”翻转组内部分像素的 LSB，统计翻转后 f 变大（规则 R）还是变小（奇异 S）的组数。

干净的自然图像在负掩码下有明显的“常规倾向”，常用量 $G_n = (R_n - S_n) / N$ 显著为正（项目里干净平滑图常在 0.3 ~ 0.7）。LSB 替换让位面随机化后，这个结构“塌缩”， G_n 明显下降。项目同时输出 G_r 、估计嵌入率等，共同组成“RS 证据”。

项目 steganalysis.py 中 RS 的核心逻辑（伪代码，仅示意分组与统计思想）

```
groups = flat[:m].reshape(-1, 4)          # 每 4 像素一组
fg = np.abs(np.diff(groups, axis=1)).sum(axis=1)  # 原平滑度 f
pos = (groups ^ np.array([0,1,1,0])) & 0xFF      # +M 翻转
neg = (groups ^ np.array([1,0,0,1])) & 0xFF      # -M 翻转
Rm = (f(pos) > fg).sum(); Sm = (f(pos) < fg).sum()
Rn = (f(neg) > fg).sum(); Sn = (f(neg) < fg).sum()
Gn = (Rn - Sn) / n                          # 干净图明显为正，随机化后≈0
```

RS 分析同样可以笔算。原文定义了平滑度判别式 f （组内相邻像素差绝对值之和）。拿一组 4 像素 [130, 132, 129, 131] 走一遍。翻转规则有两种：F（负翻转， $1 \leftrightarrow 2$ 、 $3 \leftrightarrow 4$ 、即奇数 +1、偶数 -1）与 M（正翻转，即 LSB 翻转 $0 \leftrightarrow 1$ ）。套在这组像素上：

状态	像素组	平滑度 $f = \sum \text{相邻差} $
原始	[130, 132, 129, 131]	$2 + 3 + 2 = 7$
F 翻转后	[129, 131, 130, 132]	$2 + 1 + 2 = 5$ (更平滑)
M 翻转后	[131, 133, 128, 130]	$2 + 5 + 2 = 9$ (更粗糙)

关键现象：**F 翻转让这组更平滑 (f 从 7 降到 5)**，**M 翻转让它更粗糙 (升到 9)**。对全图统计： R_m = M 翻转后变平滑的组占比， S_m = F 翻转后变平滑的组占比……原文给出的经验是干净自然图的 $G_n = (R_m - S_m)/N$ 显著为正 (0.3 ~ 0.7)。为什么？直觉版本：M 翻转是“往随机方向推”，自然图像的平滑结构经不起推；F 翻转是“往成对中心收”，反而贴合结构。LSB 嵌入把位面随机化之后，这股“F 更平滑”的天然倾向被抹平， G_n 塌向 0——检测器要的就是这个塌缩量。

避坑提醒 |

别把 G_n 当成万能指标。它对“平滑图”灵敏（天空、墙面），对“本就杂乱”的图（树叶、人群）几乎失明——杂乱图的 M/F 翻转都不怎么改变平滑度。这正是 3.5 节综合判读存在的理由：任何单一统计量都有“照不到的角落”，多路信号互补，才能把误报与漏报同时压住。

平滑度判别式写成式子（组内 4 像素 $v_1 \dots v_4$ 的相邻差绝对值之和）：

$$f(v_1, v_2, v_3, v_4) = |v_2 - v_1| + |v_3 - v_2| + |v_4 - v_3|$$

R_m 与 S_m 的定义也随之清晰：对全图每个 4 像素组分别施加 M 翻转与 F 翻转，统计“f 下降”的组占比，合成组间不平衡度：

$$G_n = \frac{R_m - S_m}{R_m + S_m}, \quad G_n \in [-1, 1]$$

干净自然图 G_n 显著为正 (0.3 ~ 0.7)，LSB 随机化把它拉向 0——3.4 节的玩具组 [130, 132, 129, 131] 就是这个式子的一个取样：原始 $f = 7$ ，F 翻转后 $f = 5$ ，M 翻转后 $f = 9$ 。

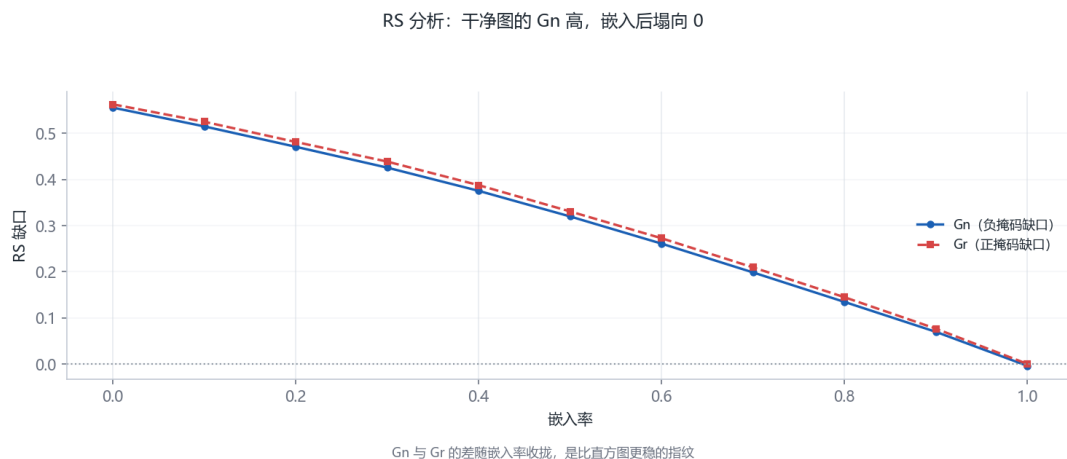


图 3-7 RS 曲线 (Gn / Gr vs 嵌入率)。这张图回答的是：RS 分析抓的是哪一种“结构塌缩”？RS 缺口 Gn / Gr 随嵌入率的变化：LSB 位面从“有结构”塌缩到“随机”

3.5 项目的综合判读：analyze()

用一个真实数字串起 3.3 与 3.4：对演示封面图（干净）跑 `analyze()`，输出 `prob=0.08`、`Gn=0.56`（实测 2026-10-04）。`prob` 是四路信号加权后的“疑似含密概率”，0.08 表示“不太可能”；`Gn=0.56` 正是 3.4 节说的干净图天然正值。对这张图做 0.2% 的 nsF5 嵌入后，`prob` 只爬到 0.08 附近——**0.2% 的改动量对一阶统计量几乎不可见**，这正是第 8 章要引入机器学习的原因：人工设计的十来个统计量够不着的地方，143 维特征加上梯度提升树能再往下探一截。

`src/steganalysis.py` 的 `analyze(image, sensitivity=...)` 并不只看一个统计量，而是组合四路信号：

卡方中位 p 值（判断灰度对是否被拉平）；

RS 塌缩幅度（Gn 相对内容自适应基线的下降比例）；

灰度差分熵与 LSB 差分熵（判断“随机”是不是图像本底自带）；

前缀 p 值序列（观察随机化是局部还是整体）。

三档灵敏度（严格/均衡/宽松）调整“可能/高度可能”的判定阈值与概率牵引系数，本质是在误报与漏报之间选择操作点。

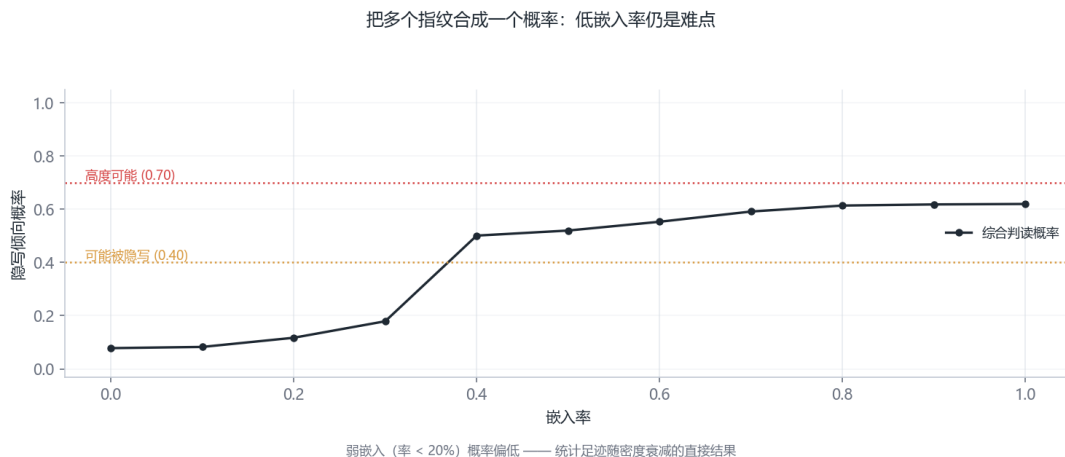


图 3-8 综合判读概率 vs 嵌入率。这张图回答的是：多个指纹合成一个结论时，判定线怎么划？综合隐写倾向概率随嵌入率的变化与两条判定线——多个指纹合成一个结论

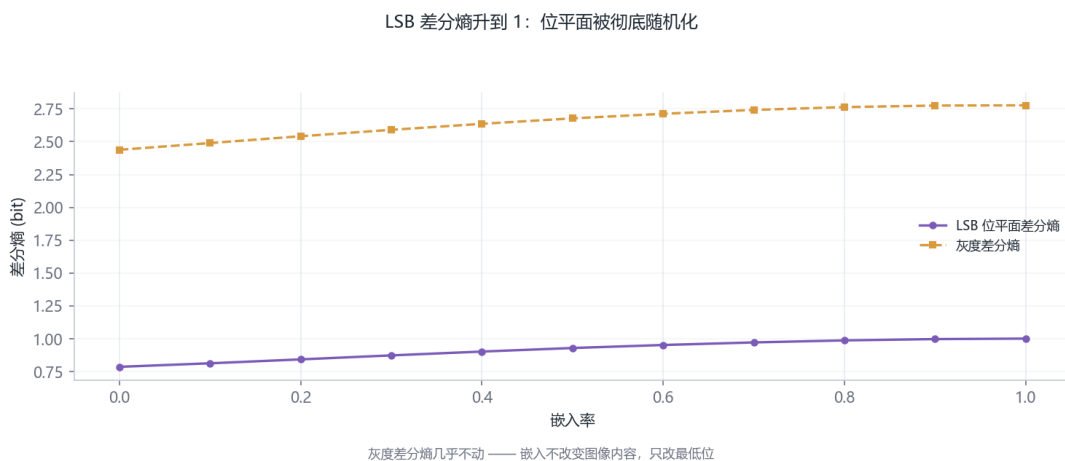


图 3-9 差分熵 vs 嵌入率。这张图回答的是：熵升高到底说明了什么？LSB 位平面差分熵与灰度差分熵随嵌入率的变化——熵升 = 随机性升

3.6 动手实验：藏一句英文，再抓它出来

利用项目真实 API：嵌入 → 保存 → 分析（可在 Jupyter 中逐段执行）

```
import sys; sys.path.insert(0, "src")
import numpy as np
import image_io as IO
from ns5_core import embed_string, extract_string
import steganalysis as SA

clean = IO.load_as_gray("img/cover.png")
stego, report, nbits = embed_string(
    clean, "Hello nsF5!", method="nsF5", p=3, password=""
)
IO.save_image(stego, "output/lab3_stego.png")
print("改动像素:", report["cover_changed"])
```

```
print("解码:", extract_string(stego, method="nsF5", p=3))

for name, im in (("干净", clean), ("含密", stego)):
    r = SA.analyze(im)
    print(name, "Gn=%.3f" % r["RS_Gn"],
          "chi2p=%.3f" % r["chi2_pvalue"],
          "prob=%.2f" % r["stego_probability"], r["verdict"])
```

动手做 | 把消息改成 5000 个字符再跑一次（参考 src/test_steg.py），观察改动像素占比、Gn 下降幅度与隐写概率的变化。再换一张噪声较多的照片，看看卡方与 RS 是否还能区分。

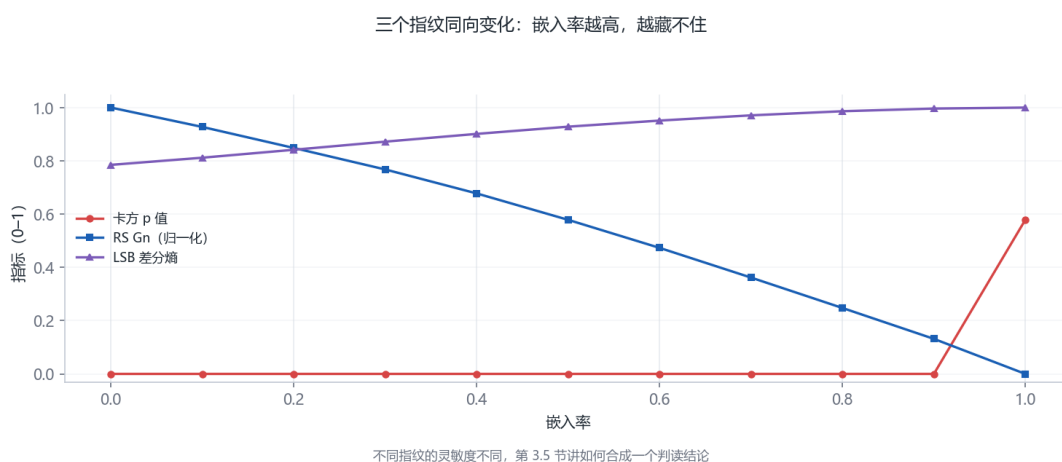


图 3-10 嵌入率综合扫描。这张图回答的是：藏与查是不是同一枚硬币的两面？卡方 p 值、RS Gn、LSB 差分熵随

嵌入率的同向变化——藏与查的完整扫描，数据见 K.6

3.7 小结与自我检查

LSB 替换会拉平相邻灰度对频数（卡方可测）并破坏位面结构（RS 可测）；

p 值高 ≠ 安全；要先判断图像本底是否天然随机；

误报/漏报不可兼得，灵敏度设置就是选择操作点；

项目 analyze() 是“统计信号 + 内容基线”的工程化组合。

想一想 | 纯 LSB 替换“藏 1 bit/像素”，改动率平均 50%。如果只藏少量消息、改动不到 1% 像素，卡方还查得到吗？RS 呢？带着这个问题进入第 4 章。

本章小结。LSB 替换留下两类指纹：直方图偶奇配对被拉平（卡方可测）、位面空间结构塌缩（RS 可测）；两者都怕“图像本底天然随机”的干扰，所以项目用四路信号 + 内容基线 + 前缀分段做综合判读；灵敏度三档 = 在误报与漏报之间选操作点，没有全都要。

动手做 |

章末练习（提示与答案见附录 J）

P3.1（手算）把 3.3 节的观测频数换成 [10, 6, 6, 6, 9, 9, 7, 7]，重算 χ^2 、自由度与 p 值方向判断。哪几对在“拉平”？

P3.2（手算）对像素组 [128, 128, 131, 131] 与 [200, 201, 202, 203] 分别算原始/F/M 三种平滑度，判断哪些组会贡献 R_m 、哪些贡献 S_m 。

P3.3（思考）一张纯黑图（全 0）做 LSB 嵌入后，卡方 p 值会发生什么变化？结合“期望=0 的对不参与统计”想一想检测的盲区在哪。

P3.4（上机）跑 3.6 节实验，把嵌入消息改成 8 KB 随机比特（接近满容量），比较 analyze() 的 prob 与 G_n 变化；再回答：为什么满容量嵌入反而“更显眼”？

P3.5（思考·选做）如果把嵌入顺序改成“只嵌入亮度在 100 ~ 150 之间的像素”，卡方与 RS 各会受到什么影响？（这是“自适应隐写”的雏形，第 8 章末尾会回看这句话。）

第 4 章 · 矩阵编码与 F5：少改一点，藏得更多（第 5 周）

本章导航 · 先看问题，再读正文

核心问题 每像素只能藏 1 位的 LSB，能不能做到“少改一点、藏得更多”？

读完你能回答

- 写出汉明校验矩阵 H 与伴随式定位的完整过程；
- 手算一个 $p=3$ 的例子：从 H 、 x 、 s 、 m 到“翻转哪一列”；
- 解释嵌入效率 α 与参数 p 的关系，说明矩阵编码为什么更划算；
- 说清 $F5 = \text{矩阵编码} + \text{JPEG 系数减幅}$ 。

从“每像素 1 位”到“平均每次改动藏 p 位”——这一章讲的是那笔省下来的账。

开场问题：把 3000 比特藏进一张图，最少要改多少个像素？ 朴素 LSB 的答案是约 1500 次（一半像素 LSB 不对）。但 1990 年代末 Crandall 提出的矩阵编码给出了一种交易：**用更多的位置，换更少的改动**——把像素分成块，每块 7 个位置只承载 3 个比特，并且保证每块至多改 1 个位置。改动数立刻从 1500 降到约 875。这不是魔法，是汉明码。本章把这笔交易的定价公式推导出来，并用项目实测数据对账。

小节	回答的问题	你的产出
4.1 效率问题	“少改”值多少钱？	能定义嵌入效率 α
4.2 汉明码速成	H 矩阵和伴随式怎么用？	能写出 $p=3$ 的 H 并解伴随式
4.3 手算例子	一整块怎么嵌、怎么纠错？	能独立复算 4.3 的例子
4.4 F5	换到 JPEG 系数上会出什么新问题？	能解释收缩的连锁反应
4.5 回到项目	实测效率与理论差多少？	能对照效率曲线读出结论

动手做 | 本章目标：理解“嵌入效率”为什么重要；能手算 $p=3$ 的汉明伴随式嵌入例子；知道 F5 的“收缩”问题出在哪。

4.1 从“每像素 1 位”到“平均每次改动藏 p 位”

朴素 LSB 每藏 1 bit 就平均改动 0.5 个像素，嵌入效率很低。如果有一种编码：每读 n 个像素组成的块，就能通过最多改动 1 个像素写入 p 位消息，改动次数就会大幅下降。这个想法叫 矩阵嵌入 (Matrix Embedding)，它用到的数学工具是二元汉明码。

定义嵌入效率 α = 平均每次修改所携带的比特数。理论极限是每次修改最多携带 p 位， $\alpha(p) = p \cdot 2^p / (2^p - 1)$ ： $p=2$ 时约 2.67， $p=3$ 时约 3.43， $p=4$ 时约 4.27，随 p 增大而上升，但需要的块长 $n=2^p-1$ 也在变大。

新手提示 | 为什么 p 越大“越划算”？ 因为汉明码用 $n=2^p-1$ 个位置承载 p 位消息； p 越大， n 相对 p 的“浪费”越少，平均改 1 次能寄的信息越多。代价是块变大、可嵌入消息对载体结构更挑剔。

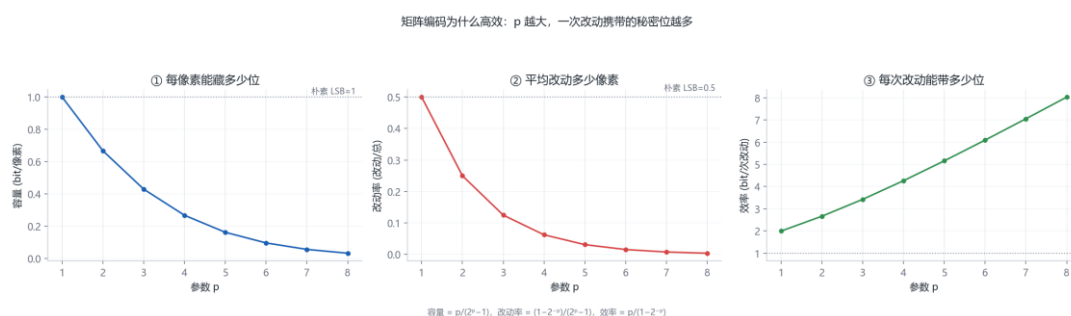


图 4-1 容量 / 改动率 / 效率随 p 的理论曲线。这张图回答的是：矩阵编码为什么比朴素 LSB 更划算？三条理论曲线，虚线为朴素 LSB 参照——矩阵编码为什么更划算，一图看懂

容量-改动率权衡：同等改动下，矩阵编码能藏更多

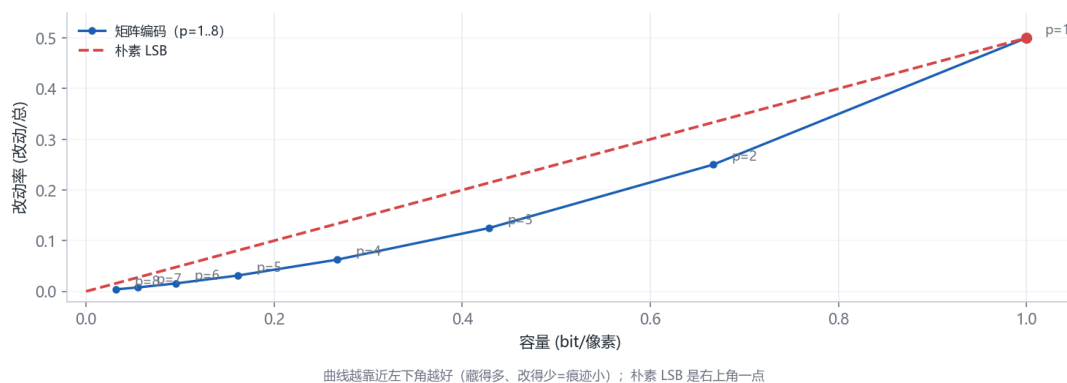


图 4-2 容量-改动率权衡曲线。这张图回答的是：少改一点，究竟能省多少？矩阵编码的容量-改动率曲线与朴素 LSB 对照点：少改一点，藏得更多

4.2 汉明码速成：校验矩阵与伴随式

设块长 $n=2^p-1$ 。构造一个 $p \times n$ 的校验矩阵 H ，它的 n 列恰好是 $GF(2)^p$ 里的全部非零向量：
 $p=3$ 时，七列就是二进制 1..7。把块内 n 个像素的 LSB 写成列向量 x ，定义伴随式 $s = H \cdot x \pmod{2}$ ，它是一个 p 位向量。

嵌入 p 位消息 m 时，我们想通过翻转个别位，让新伴随式等于 m 。因为 H 的每一列都不同，翻转第 j 位只会把伴随式变成 $s \oplus H_j$ ——也就是说，任意需要的伴随式变化都对应某一行。

计算当前伴随式 $s = H \cdot x$ ；

若 $s == m$ ，块内无需改动；

否则求差值 $d = s \oplus m$ (p 位非零向量)；

在 H 中找到等于 d 的那一行，记为第 j 行；

翻转第 j 个像素的 LSB，完成 ($H \cdot x' = m$)。

把 4.2 节的流程变成一张可以贴在墙上的“对账表”： $p=3$ 时， H 的 7 列恰好是二进制 1..7 的全部非零向量，于是“伴随式差值 d ”与“要翻转的位置”——对应——查表即可，不用解方程：

差值 d (伴随式变化)	命中的 H 列	要翻转的像素 (块内第几位)
[1,0,0]	H_1	第 1 位
[0,1,0]	H_2	第 2 位
[1,1,0]	H_3	第 3 位
[0,0,1]	H_4	第 4 位
[1,0,1]	H_5	第 5 位
[0,1,1]	H_6	第 6 位
[1,1,1]	H_7	第 7 位

这张表还藏着矩阵编码最漂亮的一个性质： **$d = [0,0,0]$ (当前伴随式恰好等于目标) 时，一个位都不用翻**。消息随机时，这种情况以 $1/2^p$ 的概率发生 ($p=3$ 即 $1/8$ 的块“免改”)。这不是运气，是设计： n 个位置承载 p 位消息，本来就有 2^p 种伴随式状态，目标状态撞上当前状态的概率就是 $1/2^p$ 。第 5 章你会看到，nsF5 还要在这个“免改概率”上叠加一层湿纸约束。

想一想 |

为什么 H 的列必须**互不相同**？假设有两列相同（比如 $H_3 = H_5$ ）， $d = H_3$ 时翻第 3 位或第 5 位都行——看似更灵活，但 $d = H_3 \oplus H_5 = 0$ 时会产生什么歧义？反过来想： n 列互不相同且取遍全部非零向量，这为什么恰好要求 $n = 2^p - 1$ ？（这是“块长为什么是 2^{p-1} ”的代数根源。）

把伴随式的定义写成式子。所有运算都在 $GF(2)$ （模 2）上进行：加法即异或，乘法即与。 $p \times n$ 校验矩阵 H 的第 j 列记作 H_j ，则块内 LSB 向量 x 的伴随式是：

$$s = H \cdot x \pmod{2} = \bigoplus_{j: x_j = 1} H_j, \quad d = s \oplus m$$

也就是说：**伴随式 = 所有“值为 1 的位置”所对应 H 列的异或**。嵌入时希望伴随式从 s 变到 m ，需要补上的差值 $d = s \oplus m$ ，而 d 恰好等于某一列时，翻转那一位即可。 $p=3$ 的 H 矩阵如下（列从左到右为 $H_1 \dots H_7$ ，与 4.3 节的手算一致）：

1	0	1	0	1	0	1
0	1	1	0	0	1	1
0	0	0	1	1	1	1

七列按二进制读出恰好是 1..7 的全部非零向量（列序 4,2,6,1,5,3,7 是项目 `build_hamming` 的固定实现）。这张表配上 4.2 节的查表流程，就是矩阵编码的全部数学。

校验矩阵 H 的每一列，就是一个 3 位二进制的非零数

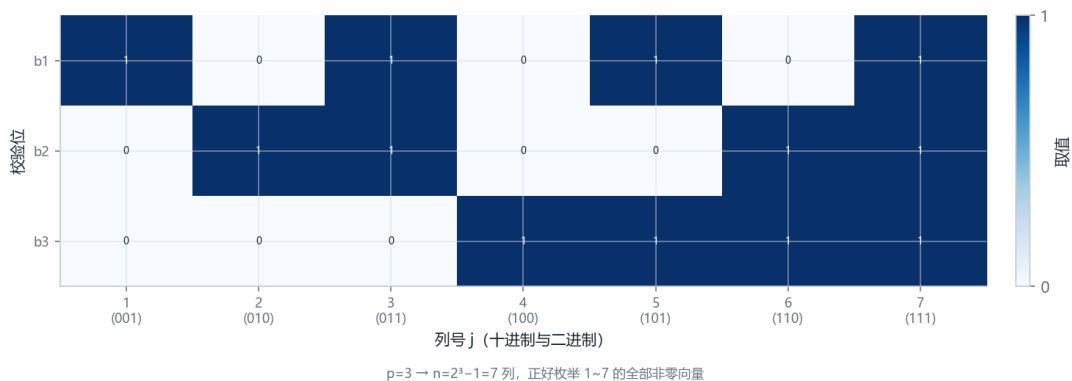


图 4-3 汉明校验矩阵 H ($p=3$)。这张图回答的是：伴随式凭什么能定位到“该翻哪一位”？ H 的列按 1~7 的二进制排列，第 j 列即 j 的三位二进制——伴随式定位的全部秘密

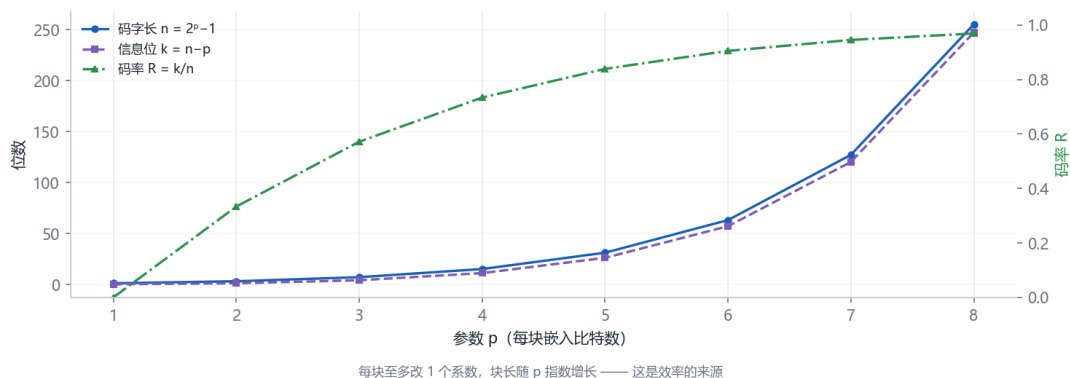
二元汉明码族 $[n, k, 3]$: p 越大, 一块越长、码率越高

图 4-4 汉明码族参数。这张图回答的是：参数 p 变大，是省了改动还是费了位置？码字长 n 、信息位 k 与码率 R 随参数 p 的变化—— p 越大越省改动、越费位置

4.3 手算一个 $p=3$ 的例子

设块内 7 个像素的 LSB 为 $x = [0, 1, 0, 1, 1, 0, 1]$ 。列向量按二进制顺序（低位在上）： $H_1 = [1, 0, 0]$, $H_2 = [0, 1, 0]$, $H_3 = [1, 1, 0]$, $H_4 = [0, 0, 1]$ 依此类推。先算伴随式： x 中取 1 的位置是第 2、4、5、7 列，做异或： $H_2 \oplus H_4 \oplus H_5 \oplus H_7 = [0, 1, 0] \oplus [0, 0, 1] \oplus [1, 0, 1] \oplus [1, 1, 1] = [0, 0, 1]$ ，所以 $s = [0, 0, 1]$ 。

想嵌入 $m = [1, 1, 0]$ ： $d = s \oplus m = [0, 0, 1] \oplus [1, 1, 0] = [1, 1, 1]$ 。 $H_7 = [1, 1, 1]$ ，命中第 7 列，于是只翻转第 7 个像素的 LSB： $x' = [0, 1, 0, 1, 1, 0, 0]$ 。再算一次 $s' = H \cdot x' = [1, 1, 0] = m$ ，成功。

7 个像素承载 3 位消息，但平均只需要改动 $(2^3 - 1)/2^3 \approx 87.5\%$ 的块里改 1 位——约 0.875 次/块，折算每 3 位约 0.875 次改动，即每次改动约带 3.43 位。

动手做 | 项目里 `src/matrix_demo.py` 把这个过程变成了可点击的可视化面板（GUI 的“矩阵编码演示”标签页）。先用 `demo_step(p, x, m_bin)` 看返回的字典里 s 、 d 、命中的列号，再手工验证 $H \cdot x' == m$ 。

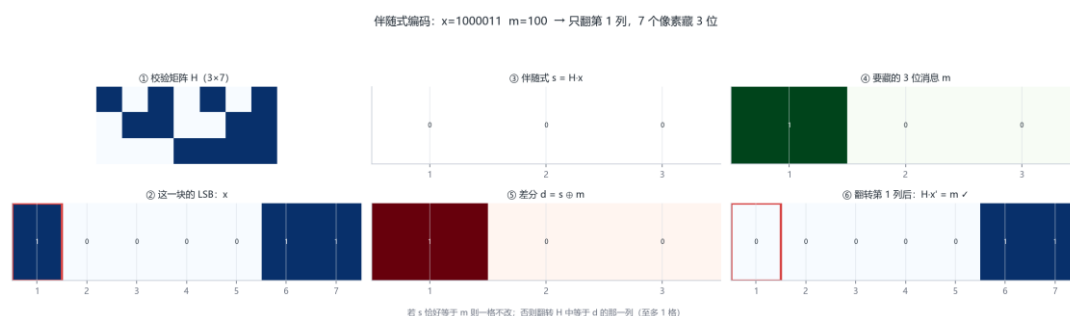


图 4-5 伴随式编码六步流程 ($p=3$ 手算例)。这张图回答的是: 从 H 、 x 、 s 、 m 到“翻转哪一列”, 中间经过哪几步? 从 H 、 x 、 s 、 m 、 d 到翻转一列的完整算例——块内至多修改 1 个系数

理论效率 $\alpha(p) = p \cdot 2^p / (2^p - 1)$ 的推导, 值得用三行算草纸写清楚, 因为它解释了“为什么值得” (原文 4.1 已给出公式): 每块 $n = 2^p - 1$ 个位置承载 p 位; 由 4.2 节, 块需要改动的概率是 $(2^p - 1) / 2^p$, 因此每块平均改动次数是 $1 - 1/2^p$; 于是

$$\begin{aligned} \alpha(p) &= \text{携带比特} / \text{平均改动} = p / (1 - 1/2^p) \\ &= p \cdot 2^p / (2^p - 1) \\ p=1: 2/1 &= 2.00 \quad p=2: 8/3 \approx 2.67 \\ p=3: 24/7 &\approx 3.43 \quad p=4: 64/15 \approx 4.27 \\ p \rightarrow \infty: \alpha &\rightarrow p \text{ (每块最多改 1 位, 极限就是“每改 1 位带 } p \text{ 位”)} \end{aligned}$$

理论要过实测这一关。用 run_e2e.py 的效率扫描 (同一张图、**满容量填充**, 实测 2026-10-04; 数字口径为 compute_comparison(6), 即把每个 p 的可用位置全部填满):

p	块长 n	码率 k/n	理论 α	实测 α	实测改动像素
1	1	1.00	2.00	2.01	32 502
2	3	0.33	2.67	2.62	16 655
3	7	0.43	3.43	3.38	8 263
4	15	0.27	4.27	4.18	4 149
5	31	0.16	5.16	5.08	2 054
6	63	0.10	6.10	5.90	1 035

三行结论, 行行有味道。其一, 实测 α 略低于理论 α ——像素域的湿点保护 (第 5 章) 会 occasionally 改变可翻转集合, 这是真实系统对理想公式的“摩擦力”。其二, 改动数随 p 指数级下降, 但**码率** (每位置承载的比特数) 也在暴跌: $p=6$ 时 63 个位置只带 6 位, 大部分容量被浪费了。其三, 改动越少, 统计指纹越弱——**效率、容量、安全性三者构成一个铁三角**, 任何隐写

算法都是在这三者之间选一个操作点。这本书剩下的所有算法对比，都是在那这个三角形的顶点。

口径提醒（勿混比）。上表的“实测改动像素”是**满容量填充**的结果，不是某条固定消息的结果。若换成 run_e2e.py 里那条 63 字节（504 比特）的演示消息，同一张图 $p=3$ 只需改动 **188** 像素（第 2 章 run_e2e 输出里的“改动 188 像素”就是它）。8 263 与 188 分母不同：前者是“填满所有可用位置”，后者是“只藏 504 比特”。附录 K.2 已把两个口径分列成两行，便于逐行对账。

想一想 |

run_e2e.py 实测里 $p=1$ 的改动数是 32 502，而朴素 LSB（无矩阵编码）改同一消息约需多少次？

（提示：504 比特 $\times 0.5 = 252$ 。）咦，比 32 502 少多了？——注意 $p=1$ 时块长 $n=1$ 、码率 1.0，它就是朴素 LSB 本身；32 502 是“整个随机测试图全部位置”的改动数，而 252 是 504 比特消息的改动数。搞清这两个数字的分母差异，你就真正读懂了效率表。

把效率推导写成两行式子。一块 $n = 2^p - 1$ 个位置承载 p 位，由查表可知：伴随式差值 d 均匀分布于 2^p 种取值，其中 $d = 0$ （免改）占 $1/2^p$ ，其余每种各占 $1/2^p$ 、各对应恰好 1 次翻转。于是每块平均改动次数为：

$$E[\text{改动/块}] = 0 \cdot \frac{1}{2^p} + 1 \cdot \left(1 - \frac{1}{2^p}\right) = 1 - 2^{-p}$$

每块携带 p 位，所以效率（每次改动平均携带的比特数）为：

$$\alpha(p) = \frac{p}{1 - 2^{-p}} = \frac{p \cdot 2^p}{2^p - 1}, \quad \alpha(3) = \frac{24}{7} \approx 3.43$$

而实测 $\alpha(3) = 3.38$ （504 比特、8 263 次改动，2026-10-04）——与理论的差值来自像素域湿点保护，第 5 章会把这个“摩擦力”变成主角。

4.4 F5：矩阵编码 + JPEG 系数减幅

把“收缩”的连锁反应用一个具体序列走一遍，你会看到它为什么比“损失容量”更讨厌。设某块抽到的载体系数（按提取顺序）是 $[+3, +1, -2, +4, +1, -3, +5]$ ，伴随式计算指出需要翻转第 2 位。第 2 位是 $+1$ ——减幅后变成 0。此时灾难分三步发生：

1. 提取端收集载体时按“非零 AC”过滤，这个突然归零的系数从序列里消失，**后面所有系数的序号整体前移一位**——不仅是这一块，包括后续所有块；
2. 伴随式从此全错，除非把这一块整个作废、消息从下一块重新开始（F5 的做法）；

3. 作废意味着容量损失，重试意味着更多减幅、更多 +1 系数被卷进来——嵌得越满，收缩雪崩越猛。

更隐蔽的是第三层：被减到 0 的系数在直方图上留下凹痕（0 过多、 ± 1 偏少），这本身就是可检测特征——第 5 章的湿纸编码会把这批“碰不得”的位置圈出来绕着走，一劳永逸地掐灭这场雪崩。

F5 是 Westfeld 提出的 JPEG 隐写：在量化 DCT 交流系数上做矩阵嵌入。修改“位”不是翻转 LSB，而是让系数的绝对值减小 1（例如 $+3 \rightarrow +2$ 、 $-5 \rightarrow -4$ ），这样既改变 LSB 奇偶，又不容易引入显著统计异常。

问题出在“收缩”：当绝对值已经是 1 的系数被减幅，会直接变成 0。而 0 系数在 JPEG 解码里意味着“没有这个频带分量”，F5 认为它不再是合法载体，于是丢掉 这块、重试后续消息。收缩导致实际载荷变小、修改效率低于理论值，还留下可被统计的特征。

4.5 回到项目：MatrixEmbedding 与效率曲线

回到项目看代码 | 打开 src/ns5_core.py 第 216 行附近的 MatrixEmbedding 类：_embed() 正是 4.3 步骤的直接代码——计算 s、比较 m、找列、翻转。src/efficiency.py 则画出了理论 $\alpha(p)$ 与实测效率的对比图。

MatrixEmbedding._embed() 核心（真实代码节选）

```
x1 = (c[pos] & 1).astype(np.uint8)    # 取块内 LSB
s = syndrome(H, x1)                   # s = H·x
if np.array_equal(s, m): continue      # 恰好命中，不改
d = (s ^ m).astype(np.uint8)           # 差值
col = int(np.where(np.all(H == d[:, None], axis=0))[0][0])
c[pos[col]] ^= 1                       # 翻转命中像素
```

生成并查看码族/效率图

```
import sys; sys.path.insert(0, "src")
from efficiency import plot_code_family_and_efficiency
path = plot_code_family_and_efficiency(p_max=6)
print(path) # output/efficiency.png
```

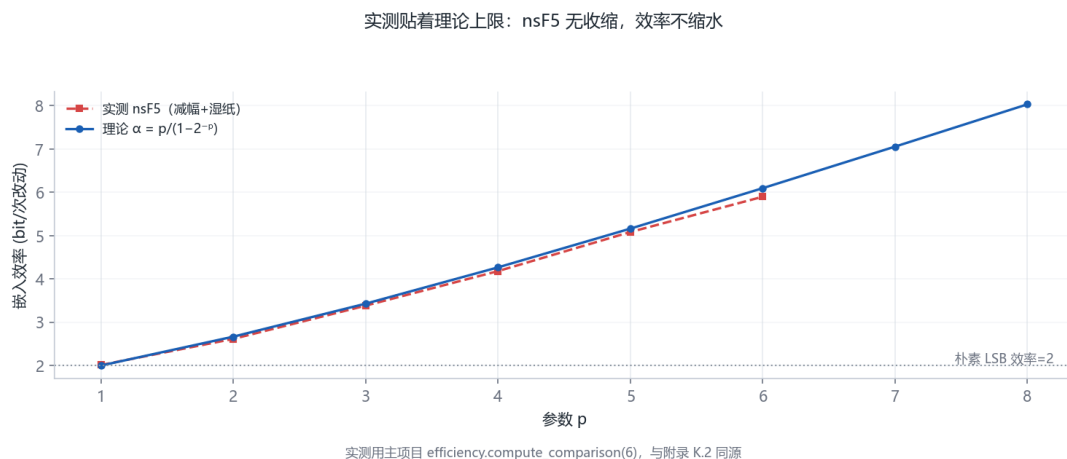


图 4-6 理论 vs 实测嵌入效率。这张图回答的是：汉明码族的理论收益，在真实载体上兑现了多少？理论嵌入效率曲线与 nsF5 实测值——汉明码族的理论收益在真实载体上兑现了多少

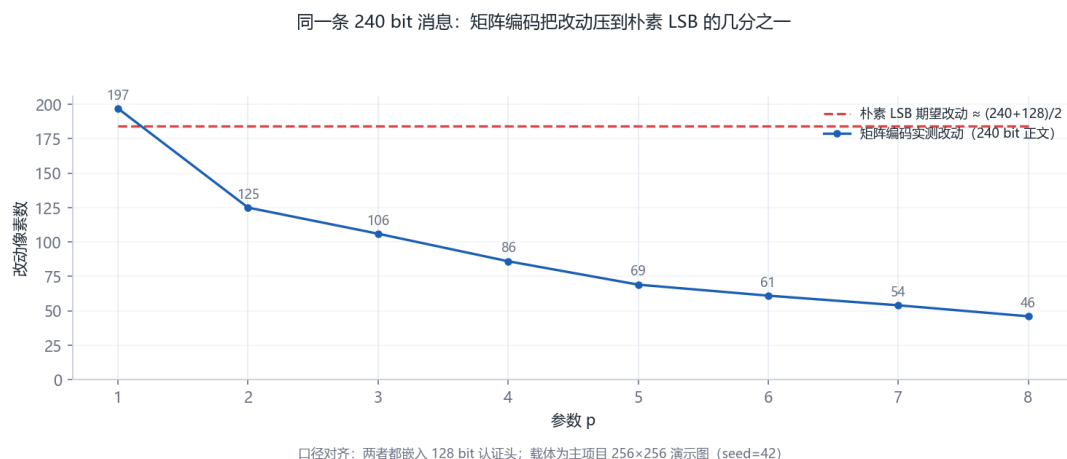


图 4-7 固定消息的改动数对比。这张图回答的是：实测改动数真的贴合理论吗？同一 240 bit 正文下，各 p 的实测改动像素与朴素 LSB 期望——实测贴合理论

4.6 小结与自我检查

矩阵嵌入以“块”为单位：n 个位置承载 p 位、至多改 1 位；

H 列互不相同 $\Rightarrow$ 任意伴随式差值都能定位到唯一需要翻转的位置；

嵌入效率 $\alpha = p \cdot 2^p / (2^p - 1)$ ，随 p 增加但块长也增加；

F5 在 JPEG 系数上减幅嵌入，遇到 |系数|=1 减到 0 就“收缩”。

想一想 | 如果 F5 遇到收缩就“跳过重试”，消息还可能正确解码吗？解码端如何知道哪块被跳过？想不通没关系——第 5 章的 nsF5 用湿纸编码彻底绕开了这个难题。

本章小结。 矩阵编码 = 用块内冗余换改动次数： $n=2^p-1$ 个位置带 p 位、至多改 1 位、平均改动 $1-1/2^p$ 次；效率 $\alpha = p \cdot 2^p / (2^p - 1)$ ($p=3$ 时 $24/7 \approx 3.43$ ，实测 3.38)；F5 把这套编码搬到 JPEG 系数上，用减幅代替翻转，代价是收缩。

动手做 |

章末练习（提示与答案见附录 J）

P4.1（手算） $x = [1, 1, 0, 1, 0, 0, 1]$ ，想嵌入 $m = [0, 1, 1]$ 。算出伴随式、差值 d 、要翻转的位置与最终码字；再对码字做一次“错第 5 位→纠错”的演练。

P4.2（推导） $p=2$ 时写出 H 的全部 3 列（二进制 1、2、3），并推导 $\alpha(2) = 8/3$ 。若一条 600 比特的消息分别用 $p=2$ 与 $p=3$ 嵌入，各需要多少块？各平均改多少位？

P4.3（思考）矩阵编码的“免改概率”是 $1/2^p$ 。如果消息不是随机的而是全 0（比如一段静音数据），免改概率还成立吗？这对隐写实践有什么提醒？

P4.4（上机）跑 4.5 节的 efficiency 扫描，把消息长度改成 1008 比特，观察各 p 档的改动数是否恰好翻倍；解释你的观察。

P4.5（思考·选做）“改动少”与“容量大”在 p 的选择上是对抗的。如果你要往一张 512×512 的图里藏一封 2 KB 的信，你会选哪个 p ？列出你的算术。

第 5 章 · nsF5 与湿纸编码：项目的核心算法（第 6 周）

本章导航 · 先看问题，再读正文

核心问题 F5 的“收缩”为什么会毁掉消息？nsF5 用什么绕开它？

读完你能回答

- 解释减幅映射 $|c| \rightarrow |c|-1$ 与 F5 收缩之间的因果关系；
- 划出湿点与干点，说明“哪些位置碰不得、哪些位置可以安全减幅”；
- 写出湿纸方程 $H \cdot e = d$ 并解一个 $p=3$ 的算例；
- 说明 nsF5 为什么能把改动控制在“至多一处”。

先划禁区，再解方程——这是本项目核心算法的全部直觉。

开场问题：如果“能修改的位置”不由你指定，而是由图像内容当场圈定，方程还解得出来吗？第 4 章的矩阵编码默认所有位置都听你的；但 JPEG 的世界里， $|c|=1$ 的系数是“减幅即塌方”的雷区，必须绕行。绕行之后，伴随式方程的可用列少了一截，原来“查表即得”的解法失效。湿纸编码（Fridrich et al.）给出的答案漂亮而彻底：**方程照样有解，而且通信双方甚至不用商量哪些位置被划掉了**。这就是 nsF5 比 F5 多出来的那块拼图，也是本项目像素域实现的核心。

小节	回答的问题	你的产出
5.1 收缩问题	像素域的“雷区”长什么样？	能指出 127/128/129 为什么危险
5.2 湿纸编码	干点上怎么解方程？	能手算一个 6 载体的玩具例
5.3 读代码	<code>_embed()</code> 的决策树是什么？	能逐行对应到 5.2 的数学
5.4 三个求解器	单列/双列/消元怎么分工？	能说清“少改优先”的次序
5.5 验证实验	nsF5 比矩阵编码好在哪？	能设计一个公平的对比实验

动手做 | 本章目标：说清“收缩”为什么伤效率；理解湿纸编码“湿点不动、干点解方程”的思想；能对照 `ns5_core.py` 讲出 nsF5 嵌入的完整流程。

5.1 先把问题拧清楚：F5 的收缩

F5 在 JPEG 量化系数上做减幅修改：系数绝对值每被减 1，LSB 奇偶就翻转一次。但绝对值 = 1 的系数（例如 +1、-1）再减就变成 0。对 JPEG 来说，0 系数是“缺席的频带”，不适合继续当载体，F5 只能放弃这一块重新寻找位置——这就是收缩（shrinkage）。

收缩让实际容量低于理论值：块被浪费，消息需要更多位置；

收缩让嵌入效率低于 $\alpha(p)$ ：为补偿损失需要更多次修改；

收缩还会留下可检测的统计特征：被改到 0 的系数变多，直方图出现异常。

项目在空间域（像素）上做了一个巧妙类比：把像素值减去 128 变成“有符号值” $xv = \text{像素} - 128$ ，于是 $|xv| \leq 1$ 的像素（127、128、129）就是“减幅会撞到零附近”的危险位置。矩阵嵌入在 JPEG 上的收缩难题，在像素域里变成了“127/128/129 不能作为普通修改对象”的同一问题。

像素域的类比值值得再钉一遍，因为它把 JPEG 的抽象问题变成了看得见的数字：把像素减 128 得到有符号值 xv ，则 $xv=+3$ 的像素是 131。减幅 = $|xv| - 1$ 且保符号：+3 → +2（像素 131→130），LSB 从 1 变 0，正是“改一位”。而 $xv = -1$ （像素 127）减幅会变成 +1——**符号翻转了，但 LSB 恰恰没变**（-1 与 +1 的奇偶相同）； $xv = 0$ （像素 128）减幅更是无定义。所以 127、128、129 三个像素是“动之即乱”的雷区，统一划为湿点。

这个细节暴露了一个容易被忽略的事实：“**改一位**”的目标并不是任何数值变化都能达成。LSB 是奇偶，数值变化要翻转奇偶，只有 ± 1 、 ± 3 、 ± 5 ……这类奇数步长才行；而减幅固定是 -1（奇数步长 ✓），但它同时要求幅值不能到 0、符号不能翻。把这两个约束合并，就得到“湿点 = $|xv| \leq 1$ ”这条朴素而准确的规则。

减幅操作本身也值得一个正式定义——它是整个 nsF5 的“原子修改”：

$$c' = c - \text{sgn}(c) \quad (|c| \geq 2) \Rightarrow \text{LSB}(c') \neq \text{LSB}(c)$$

两个条件各管一件事： $|c| \geq 2$ 保证减完不塌方（ $|c'| \geq 1$ ）、不翻符号；而“减 1”是奇数步长，奇偶必然翻转——所以它是一次合法的“改一位”。对照第 4 章矩阵编码的“查表翻一位”：矩阵编码负责**选哪个位置翻**，减幅负责**翻的方式**，两者在 nsF5 里合体。

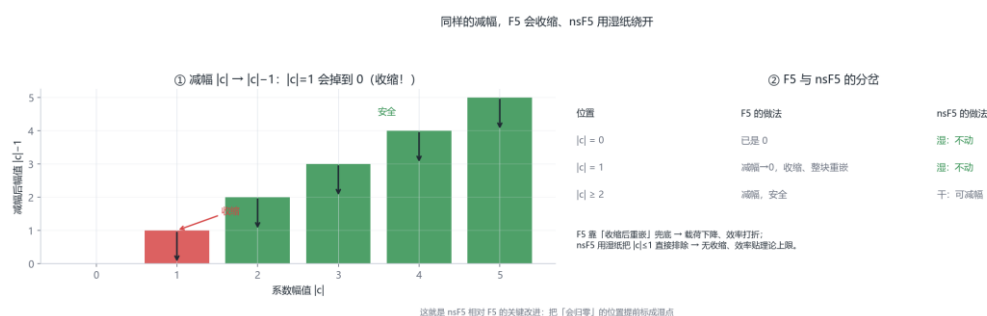


图 5-1 减幅映射与 F5 收缩。这张图回答的是: F5 的“收缩”是怎么把消息毁掉的? 减幅映射 $|c| \rightarrow |c|-1$ 的收缩点与 F5/nsF5 分岔——收缩是 F5 的老问题, nsF5 绕开了它

5.2 湿纸编码: 先划湿点, 再解方程

湿纸编码 (Wet Paper Coding) 是 Fridrich 等人提出的框架, 用来回答一个问题: 如果载体里有一部分位置“碰不得”(像被水浸湿的纸, 写了也看不见), 能不能只用其余“干”位置完成消息嵌入?

答案是可以, 而且不需要通信双方预先商量哪些位置是湿的——只要算法能从图像内容本身重建出同样的湿/干划分即可。项目正是这样做的:

对每个块, 先找出“减幅会变成零/危险”的位置, 标记为湿点;

真正的问题是: 在干位置集合上找一个修改向量 e , 使翻转这些位置后整块的伴随式恰好等于目标 m ;

即解方程 $H[:, \text{干}] \cdot y = d$, 其中 $d = s \oplus m$ 是期望的伴随式变化;

项目按“单个干列命中 $\rightarrow$ 两个干列异或命中 $\rightarrow$ GF(2) 高斯消元兜底”的顺序求解, 尽量少改;

解出的 y 是 0/1 向量: $y=1$ 的位置就是实际要减幅的干位置。

新手提示 | 为什么“先标记湿点”就不收缩了? 因为收缩的根源是“改到一半才发现改成了 0”。

nsF5 把所有会撞零的位置提前排除, 只在保证安全的干点上求解, 于是每个块都能成功嵌入, 既不用重试, 也不会出现多余的 0 系数。

按 CSAPP 的规矩, 用一个能全部笔算的玩具例把湿纸编码走通。设 $p=2$ (块内载体 6 个, H 的 6 列按 $[1,0]$ 、 $[0,1]$ 、 $[1,1]$ 、 $[1,0]$ 、 $[0,1]$ 、 $[1,1]$ 排布), 载体系数与其 LSB (x) 如下:

位置	1	2	3	4	5	6
系数数据 c	+3	+1	-2	+4	+1	-7

位置	1	2	3	4	5	6
LSB x (奇偶)	1	1	0	0	1	1
H 列	[1,0]	[0,1]	[1,1]	[1,0]	[0,1]	[1,1]
湿/干 ($ c =1$ 为湿)	干	**湿**	干	干	**湿**	干

目标伴随式 $m = [1,1]$ 。先算当前伴随式 $s = H \cdot x$ ：只把 $x=1$ 的干位列异或起来——位置 1、6 的列是 $[1,0]$ 和 $[1,1]$ （位置 2、5 是湿点不参与）， $s = [1,0] \oplus [1,1] = [0,1]$ 。差值 $d = s \oplus m = [1,0]$ 。在干位里找等于 d 的列：位置 1（ $[1,0]$ ）命中——翻转它。位置 1 的系数 +3 减幅成 +2（ $3 \rightarrow 2$ ，幅值 -1、保符号，恰好是一次合法减幅）。验证：新 $x = [0,1,0,0,1,1]$ ， $s' = [1,1] \checkmark$ 消息到位。

这个例子里藏了湿纸编码的全部要点：**湿点不参与方程**（位置 2、5 的列被划掉，哪怕它们恰好也能命中 d ）；**翻转天然就是一次合法减幅**（干点 $|c| \geq 2$ ，减 1 不会塌方）；**双方无需共享湿点表**——提取端用同样的图像内容、同样的规则重新推导湿/干划分即可。最后一点是湿纸编码真正的贡献：它把“哪些位置可用”从通信双方必须协商的先验信息，变成了载体自身的函数。

想一想 |

如果把 toy 例里的目标改成 $m = [0,1]$ ， $d = s \oplus m = [0,0]$ ——零向量在 H 的列里不存在，这意味着什么？（提示：回到 4.2 节“ $d=[0,0,0]$ 免改”的结论。）那如果 $d = [1,1]$ 而干位里没有任何单列等于它呢？——检查位置 3 与 6： $[1,1] \oplus [1,1] = [0,0]$ ，说明同时翻转这两位也能凑出 d 。“两列异或命中”正是项目三个求解器里的第二级。

湿纸编码的数学内核只有一行方程——在 $GF(2)$ 上解一个限制了自由变量的线性方程组：

$$H \cdot e = d \pmod{2}, \quad e_j = 0 \quad (j \in \text{湿点})$$

其中 $d = s \oplus m$ 是伴随式差值， e 是 0/1 修改向量（ $e_j = 1$ 表示翻转第 j 个载体）。5.2 节玩具例的方程展开来是：

$$[1,0] \cdot e_1 \oplus [0,1] \cdot e_2 \oplus [1,1] \cdot e_3 \oplus [1,0] \cdot e_4 \oplus [0,1] \cdot e_5 \oplus [1,1] \cdot e_6 = [1,1]$$

湿点约束划掉 e_2 与 e_5 （强制为 0），方程退化为 4 个未知数 2 个方程——解空间有 $2^2 = 4$ 个解，项目的求解器按“单列命中 $\rightarrow$ 两列异或 $\rightarrow$ 高斯消元”的次序取**改动最少**的那一个（toy 例取

$e_1 = 1$)。解空间非空的条件由 $GF(2)$ 的秩决定：只要湿点没有把 H 的列秩“打残”，方程就有解——这正是“湿纸”名字下真正的数学保证。

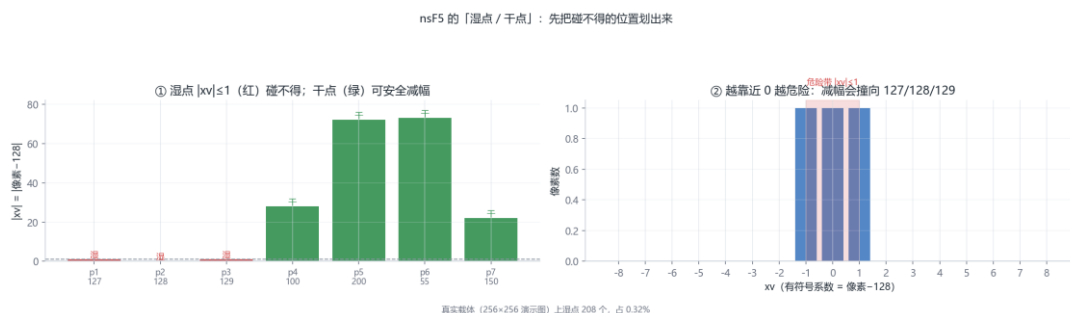


图 5-2 湿点 / 干点划分。这张图回答的是：哪些位置碰不得、哪些位置可以安全减幅？湿点（碰不得）与干点（可减幅）的划分及危险带——先划禁区，再解方程

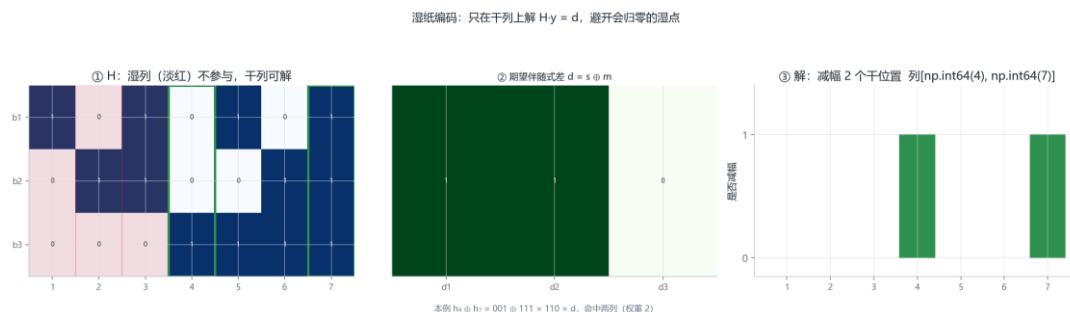


图 5-3 湿纸求解 ($p=3$ 算例)。这张图回答的是：在干列上，方程 $H \cdot e = d$ 是怎么解出来的？在干列上解出减幅位置：单列命中或双列异或命中——湿纸编码怎么把方程解出来

5.3 读懂项目：nsF5Pixel._embed()

src/ns5_core.py 里 nsF5Pixel 类继承自 MatrixEmbedding，只重写了 `_embed()`。逐行读这段代码，你会看到完整的 nsF5 决策树：

nsF5Pixel._embed() 决策树（伪代码流程，逻辑与原代码一致）

```
xv = c[pos].astype(np.int16) - 128    # 像素→有符号系数
s = syndrome(H, (xv & 1).astype(np.uint8))
if s == m: continue                    # 伴随式已匹配
tc = 命中列(H, s ^ m)                  # 最“直接”的候选位置
if abs(xv[tc]) > 1:                     # 减幅不会撞零 → 直接改
    c[pos[tc]] -= sign(xv[tc])          # 向 128 靠拢一步
else:                                   # 候选位置是湿点
    dry = [k for k in range(n) if abs(xv[k]) > 1]
    e = solve_wet_paper(H, dry, d)      # 只在干点上解方程
```

```
if e: 按 e 减幅所有命中干点
else: 兜底翻转目标位 LSB (保证可解码)
```

5.4 再看三个求解函数

三个求解器的分工次序，本质是一条“改动数量递增”的阶梯：先看有没有单列直接命中（改 1 位）；再看有没有两列异或命中（改 2 位）；都没有才上 GF(2) 高斯消元（理论上界 p 位）。为什么要这么讲究次序？因为**改动次数 = 统计指纹**，第 3 章刚讲完。还有一个容易被忽视的性质：整个求解过程不引入任何随机数——种子由输入派生（第 6 章细讲），相同输入必然得到相同的修改方案。0.2.0 起这条“逐字节可复现”承诺同样覆盖 JPEG 域，它是 repro 契约（实验档案一键重跑）的基石。

函数	作用	关键点
<code>solve_wet_paper(H, dry_cols, target)</code>	在干列里找尽量稀疏的解	权重 1 → 权重 2 → 高斯兜底
<code>gauss_solve_GF2(cols, b)</code>	GF(2) 高斯消元解线性方程组	自由变量置 0 压低改动数
<code>permute_index(total, seed)</code>	确定性伪随机置换	splitmix64 + Fisher–Yates，可 C++ 加速
回到项目看代码 打开 src/ns5_core.py 第 145–215 行， 对照 5.4 的表格逐段读。建议先 读函数注释，再用 src/test_core.py 的 test_wet_paper_needed() 验 证：它把像素强制设成 127/128/129 制造湿点，仍能完 成嵌入/解码往返。	回到项目看代码 打开 src/ns5_core.py 第 145–215 行， 对照 5.4 的表格逐段读。建议先 读函数注释，再用 src/test_core.py 的 test_wet_paper_needed() 验 证：它把像素强制设成 127/128/129 制造湿点，仍能完 成嵌入/解码往返。	回到项目看代码 打开 src/ns5_core.py 第 145–215 行， 对照 5.4 的表格逐段读。建议先 读函数注释，再用 src/test_core.py 的 test_wet_paper_needed() 验证： 它把像素强制设成 127/128/129 制造湿点，仍能完成嵌入/解码 往返。

5.5 验证实验：效率与往返

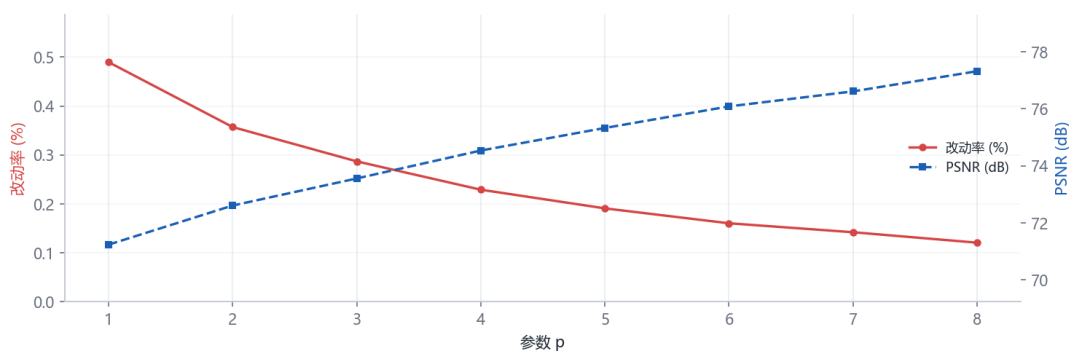
运行核心自测 + 对比两种方法

```
python src\test_core.py
```

```
# 对比 matrix 与 nsF5 改动像素数:
import sys; sys.path.insert(0, "src")
import numpy as np; from ns5_core import embed_string
img = np.random.default_rng(0).integers(0, 256, (64, 64), np.uint8)
for method in ("matrix", "nsF5"):
    stego, rep, nb = embed_string(img, "Hello nsF5!", method=method, p=3)
    print(method, "改动", rep["cover_changed"], "像素 / 容量", nb)
```

对同一张随机图、同一条消息，matrix 与 nsF5 的改动数不一定谁更少——因为 nsF5 的保护范围（127/128/129）在随机图上很少触发。真正的差异出现在“危险像素 密集”的图上：nsF5 依然能无收缩完成嵌入，matrix 则可能反复失效。

改动越少，PSNR 越高：弱嵌入几乎不损画质



同一条 504 bit 消息；PSNR 均 > 71 dB，肉眼无法分辨

图 5-4 改动率与 PSNR。这张图回答的是：改动越少，保真一定越高吗？nsF5 各 p 的改动率与 PSNR——改动越少，保真越高

nsF5 端到端：所有 p 都能原样取回消息

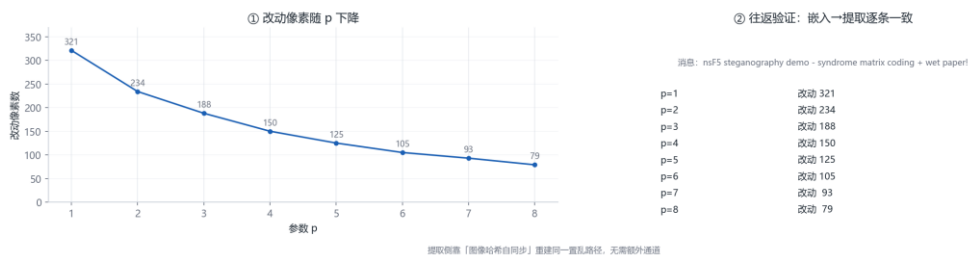


图 5-5 端到端往返验证。这张图回答的是：容量换保真之后，还能一个 bit 都不错吗？各 p 的改动像素数与往返提取一致性——容量换保真，但一个 bit 都不能错

5.6 小结与自我检查

收缩 = 减幅撞零导致块作废，损伤容量、效率与统计安全；

湿纸编码把“不能改的位置”预先划为湿点，只在干点解伴随式方程；

nsF5 = 矩阵嵌入 + 湿纸编码，无收缩、无重试；

项目在像素域用 $xv = \text{像素} - 128$ 模拟 JPEG 系数， $|xv| \leq 1$ 视为湿点。

想一想 | 为什么湿纸编码“不需要预先约定湿点位置”？解码端怎么知道哪些像素不能用来承载消息？提示：解码端只需要读 LSB 算伴随式，它不需要知道嵌入时绕过了谁。

本章小结。收缩的根源是“减幅撞零”，它的代价是容量、效率、统计安全三连击；湿纸编码把雷区划为湿点、只在干点上解伴随式方程，双方凭图像内容各自重建湿/干划分；nsF5 = 矩阵编码 + 湿纸编码，无收缩、无重试；像素域用 $xv = \text{像素} - 128$ 完美复刻 JPEG 的语义。

动手做 |

章末练习（提示与答案见附录 J）

P5.1（手算）把 5.2 节玩具例的目标换成 $m = [0,1]$ ，重算 d 与需要的翻转；再换成 $m = [1,1]$ （用两列异或）算一次。

P5.2（手算）像素行 $[131, 128, 129, 134, 130, 133]$ 的 xv 与湿点各是什么？若伴随式要求翻转位置 4，减幅后像素值变成多少？

P5.3（思考）湿纸编码为什么要求“提取端能从图像内容重建湿/干划分”？如果湿点由口令的伪随机数决定而不由内容决定，会发生什么？（提示：嵌入会改动图像，改动后重算的湿点还一样吗？）

P5.4（上机）跑 5.5 节对比实验，把测试图换成 `img/cover.png`（平滑图），比较 matrix 与 nsF5 的改动数——这次 nsF5 应该明显更少。解释为什么换图后结论反转。

P5.5（思考·选做）如果一张图的全部载体都是湿点（比如全 128 的纯色图），湿纸编码还有解吗？项目会怎么表现？（对照 `test_capacity_error` 的行为。）

第 6 章 · 口令、SHA-256 与“键控”安全设计（第 6–7 周）

本章导航 · 先看问题，再读正文

核心问题 如果嵌入位置是固定的，攻击者会怎样？键控如何让它变得不可预测？

读完你能回答

- 说明“固定位置必被抓”的道理（位置本身就是可统计的信号）；
- 复述两条种子规则：`seed0 = f(口令)`、`seedB = f(cover\_hash, 口令)`；
- 解释 SHA-256 的**雪崩效应**，说明“差一字符、seed 差千里”；
- 说清篡改感知能感知什么、不能感知什么。

键控把“藏在哪里”从公开约定变成只有持钥者知道的秘密。

开场问题：消息藏好了，攻击者也下载到了图——他离秘密还差几步？第 5 章结束时，算法本身已经无懈可击，但有个前提没说出口：攻击者得先知道**藏在哪里**。如果嵌入位置是“从左上角开始顺序”，那这一步等于白送。本章解决“藏在哪”的问题，顺便回答一个更深的问题：**怎么让“藏的位置”既由口令决定、又由图像自己决定，而且通信双方不需要额外传一张“藏宝图”**。答案由三块积木搭成：SHA-256（强哈希）、splitmix64（确定性伪随机）、Fisher–Yates（无偏洗牌）。

小节	回答的问题	你的产出
6.1 固定路径之殇	不加键控会输在哪？	能列出三类位置级攻击
6.2 两条种子规则	头部/正文两池怎么设计？	能画出嵌入端与解码端的对称流程
6.3 确定性置乱	种子怎么变成排列？	能手算 splitmix64 的第一步
6.4 篡改感知	哈希比对能感知什么、不能什么？	能说清键控 ≠ 完整性认证

动手做 | 本章目标：理解为什么嵌入位置要“由密钥决定”；会讲清解码自同步与篡改感知的原理；知道这套设计的边界在哪里。

6.1 如果嵌入位置是固定的，会怎样？

假设每次都是从左上角开始顺序嵌。攻击者知道算法后，可以：① 直接读取前 N 个 像素的 LSB 还原消息；② 把消息原位复制到另一张“看起来相同”的图像上（位置 替换攻击）；③ 对固定区域做针对性扰动，既破坏你的消息又不改动其余图像。

因此现代隐写会把嵌入路径做成密钥控制的伪随机顺序：没有口令（或不知道 内容哈希）的人，不知道消息在哪一块，也无法进行位置级攻击。

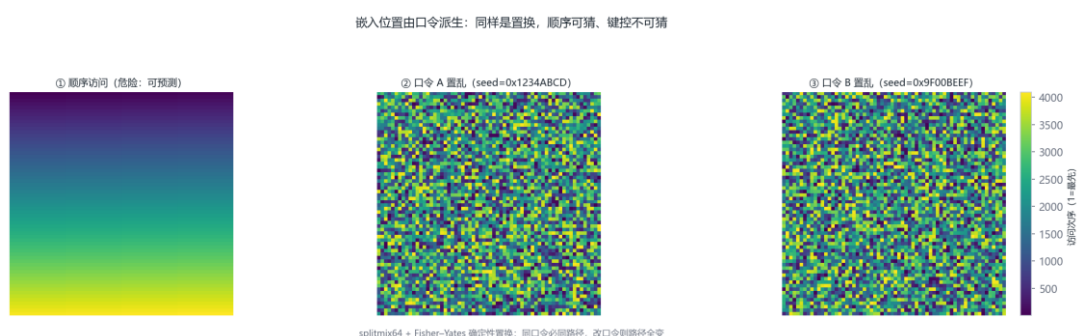


图 6-1 键控置乱 vs 顺序访问。这张图回答的是：位置固定为什么必被抓？键控改变了什么？顺序访问与两条口令派生的键控置乱路径（ 64×64 网格）——固定位置必被抓，键控让位置不可预测

6.2 两条种子规则：先认图，再找正文

“两条种子规则”听起来抽象，其实是一张对称的流程表。嵌入端与解码端做的是**同一件事**的两个方向：

步骤	嵌入端	解码端
①	derive_seed(口令) $\rightarrow$ seed0, 置乱头部池, 写入 cover_hash	derive_seed(口令) $\rightarrow$ seed0, 逆置乱头部池, 读出 cover_hash
②	校验：头部写入前算的哈希 = 当前图像哈希	校验：读出的 cover_hash = 当前图像哈希
③	derive_seed(cover_hash + 口令) $\rightarrow$ seedB, 置乱正文池, 嵌入消息	同左, 找到正文位置, 提取消息

为什么要把两把钥匙拧在一起？单用口令：口令往往是人选的弱密钥，字典攻击一攻就破；单用内容哈希：谁都能算，等于没锁。**口令 ⊕ 内容哈希**的组合让“知道算法”和“知道口令”都变得不够——而且第二把钥匙直到第①步成功才出现，口令错误的人连正文在哪都不知道，只会得到一致的失败（而不是乱码），这本身就杜绝了“用错误输出反推口令对错”的旁路。

新手提示 |

还有一个更深的巧思：cover_hash 是**原图**的哈希，不是含密图的。这意味着解码端必须拿到“未被改动的图像字节”才能通过第②步——于是这套机制顺手获得了篡改感知能力（6.4 节）：哪怕口令正确，图像被转存、被裁剪、被重压缩，哈希都会失配并明确报错。一个字段，两份用途——好的系统设计常常如此。

项目把像素分成两个池：头部池（开头 N_h 个块）与正文池（其余像素）。密钥派生用两个独立的种子：

seed0 = 仅由口令派生：决定头部池的置乱顺序。头部里预埋的是 cover_hash——由原图像字节计算的 SHA-256 截断前 16 字节（128 位）；

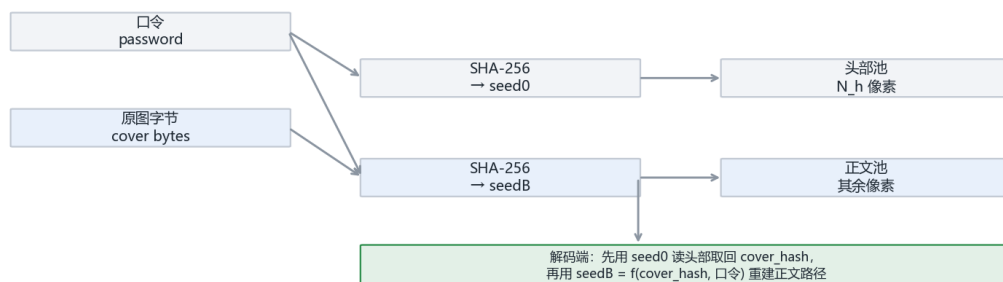
seedB = cover_hash + 口令联合派生：决定正文池的置乱顺序与分块。

解码时，接收方先用自己的口令重建 seed0，读回头部取出 cover_hash，再与口令一起重建 seedB，才能找到正文。口令错误或头部被破坏，后续路径立刻失效——test_pwd_mismatch() 测的就是这一点。

项目中的种子派生（真实代码）

```
def derive_seed(image_bytes, password=""):
    base = hashlib.sha256(image_bytes).hexdigest()
    if password:
        base = hashlib.sha256(
            base.encode() + password.encode()).hexdigest()
    return int(base, 16)
```

两条种子规则：头部只认口令，正文还认「图像内容」



头部放的是 cover_hash (认证锚点)；正文路径绑定 cover_hash，改动原图即失配

图 6-2 两条种子规则。这张图回答的是：为什么头部和正文要用两个不同的种子？头部 $\text{seed0} = f(\text{口令})$ 、正文 $\text{seedB} = f(\text{cover_hash}, \text{口令})$ 的两级派生——先认图，再找正文

SHA-256 雪崩效应：输入动一点，输出面目全非

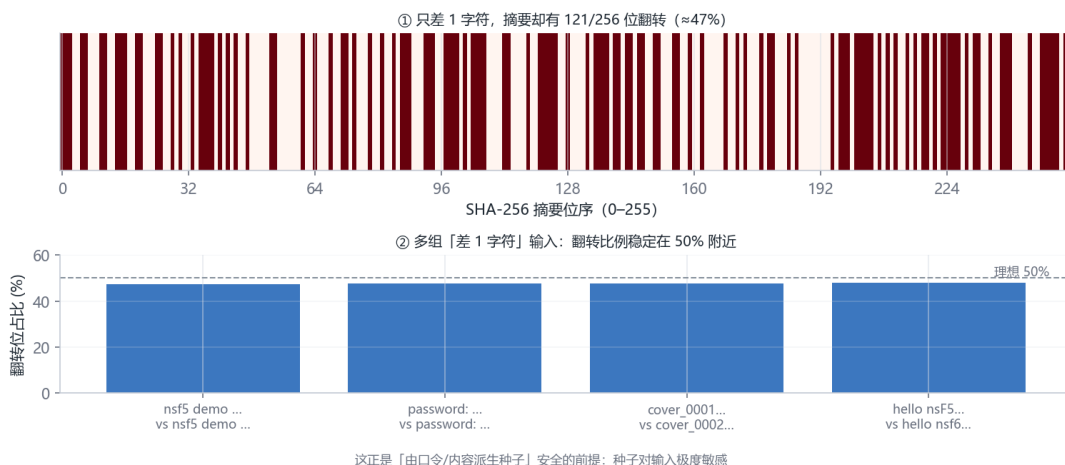


图 6-3 SHA-256 雪崩效应。这张图回答的是：口令差一个字符，种子会差多少？差 1 字符的 SHA-256 摘要差异位图与多组输入翻转比例——差之毫厘，seed 差之千里

6.3 确定性置乱：splitmix64 + Fisher–Yates

有了种子还不够，还要把它变成“1..N 的一个确定排列”。项目用 splitmix64 作为 伪随机源，执行 Fisher–Yates 洗牌：从末尾向前，每一步用伪随机数选一个位置交换。同样的种子必然产生同样的排列——这正是“自同步”的数学基础。

回到项目看代码 | 打开 `src/ns5_core.py` 的 `permute_index()`：DLL 可用时调用 C++ 的

`nsf5_permute`，否则走 Python 同算法 fallback。两套实现必须给出 完全相同的排列，否则用 DLL

嵌入的图在无 DLL 的机器上无法解码。src/cppembed.py 的 selfcheck() 专门做这种跨语言一致性校验。

splitmix64 每一步做两件事：状态推进（加一个“黄金常数” 0x9E3779B97F4A7C15），再用三轮移位-乘法把状态搅碎，输出一个 64 位伪随机数。用种子 0 手算前三步（真实输出，与 ns5_core.py 的 Fisher-Yates 内联逻辑逐位一致）：

步	推进后状态 x	输出 r
1	9e3779b97f4a7c15	16 294 208 416 658 607 535
2	3c6ef372fe94f82a	7 960 286 522 194 355 700
3	daa66d2c7ddf743f	487 617 019 471 545 679

三个输出之间看不出任何规律——这正是“雪崩效应”：输入差 1 位，输出约一半的位会翻转。Fisher-Yates 洗牌从数组末尾向前走，每一步 i 用一个输出 r 选出 $j = r \bmod (i+1)$ ，交换位置 i 与 j 。同样的种子必然得到同一个排列，这就是“自同步”的数学保证；不同的种子，排列近乎必然不同——64 位状态空间意味着你试遍地球上的沙粒也撞不到两个相同排列。

一个值得停下来想的细节： $j = r \bmod (i+1)$ 有没有偏差？64 位远大于 $i+1$ ，模运算的偏差小于 2^{-64} 量级——理论存在、工程可忽略。写进手册是因为“知道自己近似在哪里”与“近似本身”同样重要。

新手提示 |

为什么不用 Python 自带的 random 模块？两个理由。其一，random 的算法与参数随版本可能变化，而 splitmix64 是一篇论文定死的常数——十年后重跑结果不变；其二，random 的内部状态可以被预测者通过观察输出反推，splitmix64 的一次输出不泄露状态的反向信息。“可复现”与“不可预测”在密码学里是一对需要分别设计的性质。

把第一步的全部中间值用十六进制摊开（真实计算，逐行可复算）。每行左边是操作、右边是结果：

```
x ← 0 + 0x9E3779B97F4A7C15      = 0x9E3779B97F4A7C15
z ← x ^ (x >> 30)                = 0x9E3779BB07979AF0
z ← z * 0xBF58476D1CE4E5B9 (mod 264) = 0x6F68261B57E7A770
z ← z ^ (z >> 27)                = 0x6F682616BAE3641A
```

```
z ← z * 0x94D049BB133111EB (mod 264) = 0xE220A838BF5C9DDE
r ← z ^ (z >> 31) = 0xE220A8397B1DCDAF
= 16294208416658607535
```

三招“移位-异或-乘常数”各司其职：第一次异或把低位信息翻到高位，两次大常数乘法（常数本身经过搜索挑选）让任何一位的变化都污染全部 64 位，最后一次异或把高位信息翻回低位。为什么用 $\text{mod } 2^{64}$ ？溢出丢弃是刻意的设计——丢弃进位让输出对输入的低位变化高度敏感，同时保证运算可逆性可控、速度极快。你在自己的机器上按这套常数手推，会得到逐位相同的结果。

6.4 篡改感知：到底能感知什么

用一组真实数字感受雪崩效应（2026-10-04 现算）：对两个只差一个字符的字节串求 SHA-256——

输入	SHA-256 摘要（前 32 个十六进制位）
nsf5 demo image v1	528f940a 88e01819 2e96fdd1 7bc22fce ...
nsf5 demo image v2	b4bc0cc0 d6491e1d a734b65f 0132edb2 ...

两个输入只差最后 1 个字符（256 位输出里实测翻转 121 位 $\approx 47\%$ ，符合“约一半”的期望）。这就是“改 1 像素 → 哈希面目全非”的定量含义，也是 6.5 节实验 B 的原理。同时把边界说透：这套机制能告诉你“图像变过”，但说不出“怎么变的”与“谁变的”——它能感知一次无意转码，也能感知一次恶意替换，在它眼里两者等价。要区分它们，需要把哈希用密钥做 MAC（消息认证码），那是另一层设计，本项目在教学口径下刻意不做。

嵌入端在报告中给出 cover_hash（原图哈希）。任何像素被改动，图像字节的 SHA-256 都会改变。把收到的图像重新计算哈希，与嵌入端报告的（或头部解出的）哈希比较：不一致 = 图像被动过。更妙的是，由于正文路径依赖这个哈希，即使攻击者只改了一个像素，正文置乱种子也随之错位，解码结果会立刻崩溃或乱码。

同时要诚实地说清边界：这套机制是键控（keying）与篡改感知，不是密码学意义上的完整性认证。头部哈希本身没有用密钥做 MAC，理论上有能力重嵌整张图的攻击者可以同步更新头部哈希。README 把这个列为后续工作（例如引入密钥化 MAC），阅读时注意区分“检测普通篡改”与“抵御恶意重嵌”。

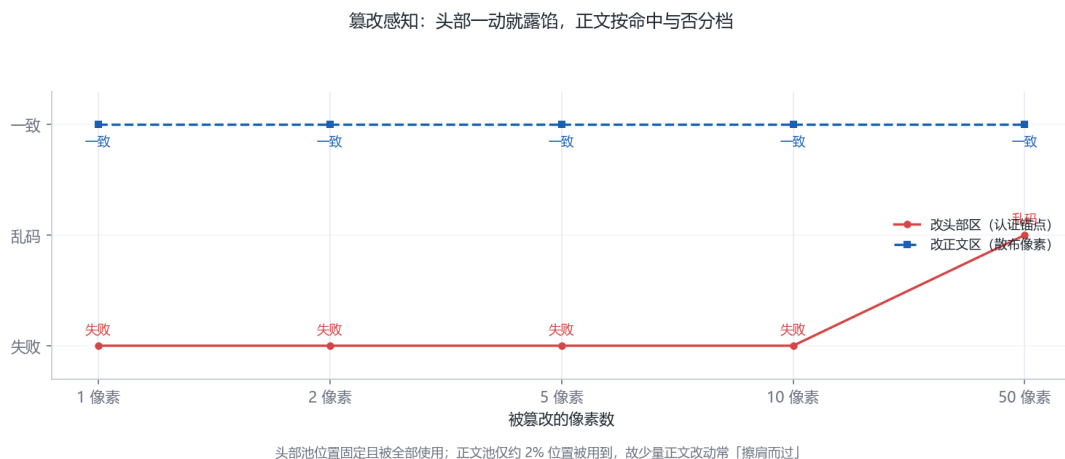


图 6-4 篡改感知。这张图回答的是：改一个字节，提取会不会失败？头部区与正文区篡改对提取结果的影响——
改一个字节，提取即失败

6.5 动手实验：口令错误与像素篡改

实验 A：口令不匹配应失败；实验 B：改 1 像素后解码

```
import sys; sys.path.insert(0, "src")
import numpy as np; import image_io as IO
from ns5_core import embed_string, extract_string, get_image_hash

img = IO.load_as_gray("img/cover.png")
stego, rep, _ = embed_string(img, "secret", method="nsF5",
                             p=3, password="A")
print(extract_string(stego, method="nsF5", p=3, password="B"))
tampered = stego.copy(); tampered[0, 0] ^= 1 # 只改 1 个像素
print(get_image_hash(stego)[:16], "vs", get_image_hash(tampered)[:16])
print(extract_string(tampered, method="nsF5", p=3, password="A"))
```

避坑提醒 | 实验 B 的失败方式可能是 ValueError、乱码或截断消息，取决于被改像素落在哪块。

不要期待“总是抛异常”——篡改感知的工程表现就是：与哈希比对失败，且正文无法自同步。



图 6-5 口令门。这张图回答的是：口令错了，链路为什么会自治性破坏？正确 / 错误 / 空口令的提取结果与失败原因——口令错，链路自治性即破坏

6.6 小结与自我检查

固定路径 = 可预测 = 可被复制与针对；键控路径让隐藏位置由口令+内容决定；

头部放 128 位内容哈希，正文种子依赖该哈希 → 解码无需外部同步；

确定性置换（splitmix64 + Fisher-Yates）是自同步的数学保证；

键控 ≠ 完整性认证；强对抗场景需要密钥化 MAC。

想一想 | 如果两张不同的原图被嵌入了同一条消息，含密图的隐藏位置会一样吗？为什么？（提示：正文种子由什么决定？）

本章小结。固定路径是靶子，键控路径让位置由“口令 + 内容哈希”共同决定；头部池放内容哈希、正文池用哈希加口令派生，解码端两步重建、无需传藏宝图；splitmix64 + Fisher-Yates 保证同种子同排列；雪崩效应让篡改无所遁形——但也只到“感知”为止。

动手做 |

章末练习（提示与答案见附录 J）

P6.1（手算）splitmix64 从种子 0 出发：状态推进后的第一步 x 是多少（十六进制）？用 6.3 节的表核对。

P6.2（思考）攻击者拿到一张已嵌入的图与完整的算法源码，唯独没有口令。对照 6.2 的流程表，列出他每一步会卡在哪里；再假设他拿到了第二张“同一口令嵌入”的图，他能对比出什么？（这引出“多图同口令”的安全边界。）

P6.3（思考）为什么头部哈希不用口令做 MAC 在教学项目里“够用”，在真实取证产品里“不够”？各给出一条理由。

P6.4（上机）跑 6.5 节实验 A/B，再做一个变体：把含密图重新用画图软件打开另存为 PNG（像素不变），解码会成功吗？会的话，为什么哈希校验没有拦住它？

P6.5（上机·选做）用 hashlib 对同一段文本的两个“只差 1 个字符”的输入求 SHA-256，数一数有多少位不同；多跑几组，验证“约一半”的直觉。

第 7 章 · 机器学习基础（写给第一次学的人）（第 7–8 周）

本章导航 · 先看问题，再读正文

核心问题 模型到底是怎么“学”的？为什么“准确率”会骗人？

读完你能回答

- 用 sigmoid 把无界得分压成概率，并手算一步梯度下降；
- 解释交叉熵损失与梯度方向（为什么“把 w 调大”能降损失）；
- 说明过拟合与数据泄漏，复述“按源图分组”为什么是纪律；
- 读懂混淆矩阵、ROC 与 AUC，并解释为什么不能只看准确率。

本章为第一次学机器学习的人写：每个抽象都配一个可以手算的迷你例。

开场问题：给你 2898 行数据，每行 11 个数字，让你人工挑出哪些图藏了消息——你打算怎么挑？肉眼看每行 11 个数，看十行就晕了。但这正是机器学习要替你做的事，而且它做的每一步——找规律、定阈值、报置信度——你都将在本章亲手算一遍小例子。本章不讲神经网络，只讲清楚一件事：“学习”是怎么被还原成“在数据上最小化一个损失函数”的。这是读懂第 8 章全部实验的最小前置。

小节	回答的问题	你的产出
7.1 二分类视角	隐写检测在学习框架里长什么样？	能定义样本/特征/标签
7.2 数据	一行数据里的每列是什么？	知道 photo_id 为什么是命门
7.3 模型与损失	“学”具体是在算什么？	能手算一步梯度下降
7.4 过拟合与泄漏	交叉验证为什么必须按组切？	能复述真实泄漏事故
7.5 评估指标	AUC 之外还要看什么？	能解释 Youden 与低误报点

动手做 | 本章目标：建立“数据→特征→模型→评估”的完整框架；能讲清过拟合、交叉验证与数据泄漏；看懂 AUC 与阈值选择。

7.1 隐写检测，本质上是一个二分类问题

把整件事换一个视角：给定一张图，它要么是干净载体（label=0），要么藏过消息（label=1）。机器学习的任务，就是学一个函数 $f(\text{图像}) \rightarrow 0/1$ 或“含密概率”。这正是项目里监督式隐写检测做的事。

整个监督学习只有四样东西：数据、特征、模型、评估。下面按这个顺序讲，每个概念都立刻对应到项目代码。

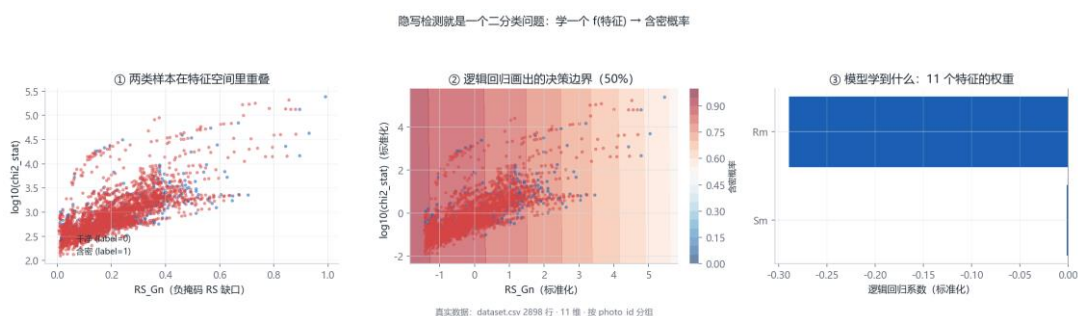


图 7-1 二分类视角：散点、决策边界与权重。这张图回答的是：两个类别在特征空间里天生重叠吗？特征空间重叠、逻辑回归决策边界与标准化系数——两个类别天生有重叠

7.2 数据：样本、特征与标签

v1 把每张图压缩成 11 个数字，叫特征向量；v1.4 又在其上扩展出 143 维 v2 特征（见 8.8）。先以 11 维教学：所有样本组成一张表 data/dataset.csv，目前有 2898 行 = 414 张照片 ×（1 个干净 + 6 个含密变体），其中干净 414 个、含密 2484 个。

数据集长什么样（前 3 列是 11 维特征的一部分）

```
Rm,Sm,Rn,Sn,RS_Gr,RS_Gn,chi2_pvalue,diff_entropy,lsb_diff_entropy,median_prefix_p,chi2_stat,label,photo_id,...
41316,4574,36608,5057,0.56,0.48,3.5e-278,1.99,0.986,2.7e-166,1719.3,0,0,...
40488,4907,35701,5342,0.54,0.46,2.0e-269,2.02,0.989,2.2e-155,1675.9,1,0,...
```

data/dataset.csv（真实前两行，数值截断显示）

注意 photo_id 这一列：同一张照片的干净版与各变体共享同一个 id。这个字段是防数据泄露的关键（v1.4 的 v2 数据集扩到 12 档并混入真实 JPEG 干净样本，分组思想不变）。

7.3 模型怎么“学”：损失、梯度与逻辑回归

先看最简单的可解释模型——逻辑回归：它对特征做线性加权 $z = w \cdot x + b$ ，再用 sigmoid 函数 $\sigma(z) = 1/(1+e^{-z})$ 压到 $0 \sim 1$ ，输出“含密概率”。

训练就是找一组权重 w ，使模型在所有样本上的损失最小。逻辑回归常用的损失是交叉熵：预测 1 而真值是 0，损失就大；预测越错，惩罚越陡。优化方法（如梯度下降）反复微调 w ，让损失一点一点降下来。

新手提示 | 你不必现在会推梯度公式。先记住三句话：损失 = 模型错得有多离谱；训练 = 找一组参数让损失最小；梯度下降 = 沿着“损失下降最快的方向”一次次挪动参数。项目用 scikit-learn 的 LogisticRegression，这些细节都被封装好了。

“训练就是调权重”值得用一个 4 样本的迷你数据集完整走一步。设一维特征 x 与标签 y ： $x = [0.2, 0.4, 0.6, 0.8]$ ， $y = [0, 0, 1, 1]$ （前两个干净、后两个含密）。初始 $w = 0$ ， $b = 0$ ，则每行的预测概率都是 $\sigma(0) = 0.5$ 。

量	计算	结果
初始损失（交叉熵）	$-\text{mean}(y \cdot \ln p + (1-y) \cdot \ln(1-p))$ ， p 均为 0.5	$\ln 2 \approx 0.693$
梯度 $\partial L / \partial w$	$\text{mean}((p-y) \cdot x) = (0.5 \cdot 0.2 + 0.5 \cdot 0.4 - 0.5 \cdot 0.6 - 0.5 \cdot 0.8) / 4$	-0.10
梯度 $\partial L / \partial b$	$\text{mean}(p-y) = (0.5+0.5-0.5-0.5) / 4$	0
一步更新（学习率 1）	$w \leftarrow w - 1 \times (-0.10)$	$w = 0.10$

读一读这步更新在说什么：梯度是负的，说明“把 w 往大调”能减少损失——直觉上， x 大的样本标签是 1，权重应当为正，模型正在从数据里“悟”出这条规律。重复“预测 → 算梯度 → 更新”几百次， w 会收敛到一个正数，模型学会“ x 越大越可能含密”。第 8 章的 11 维、143 维模型做的事一模一样，只是 x 从 1 个数变成 143 个数、“悟”的规律从一条变成一张交织的网。

想一想 |

上面那一步更新里，如果学习率取 100 会怎样？（ w 直接飞到 -10，损失反而爆炸。）学习率是“每步走多远”的步长，太大来回震荡、太小磨蹭不动。项目用的 LightGBM 不用梯度下降而用“逐树累加”，但“每一步都要让损失可验证地下降”这条纪律是同一套。

把迷你例用到的三个公式正式写出来。预测概率：

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = w \cdot x + b$$

交叉熵损失（对全部 N 个样本取平均）：

$$L(w, b) = -\frac{1}{N} \sum [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$$

对权重与偏置求偏导（链式法则展开后形式出奇地干净）：

$$\frac{\partial L}{\partial w} = \frac{1}{N} \sum (p_i - y_i) x_i, \quad \frac{\partial L}{\partial b} = \frac{1}{N} \sum (p_i - y_i)$$

梯度里的 $(p_i - y_i)$ 就是“预测偏差”：预测对了贡献 0，预测错了按错误方向推权重。7.3 节手算的 -0.10 与 0 正是这两个式子在 4 个样本上的取值——公式与算例严丝合缝。交叉熵的这个梯度形式是它被普遍采用的原因：不像平方损失那样在“自信地错”时梯度趋零。

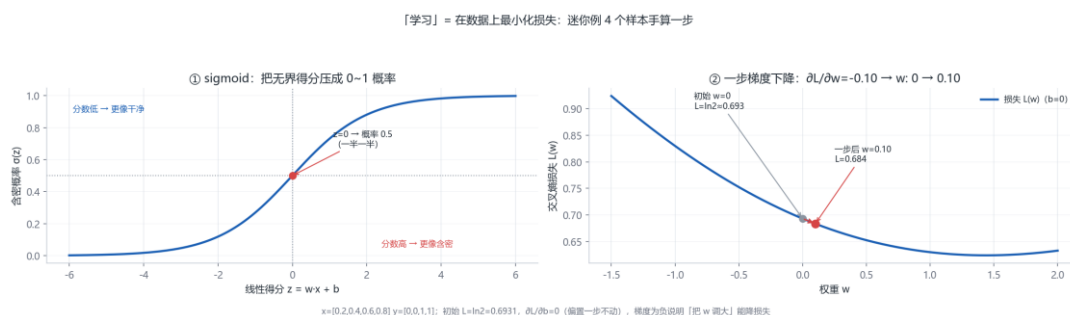


图 7-2 sigmoid 与一步梯度下降。这张图回答的是：模型是怎么“学”的？学习率太大会怎样？sigmoid 曲线与迷你例的一步梯度下降——学习率太大一步跨过谷底

7.4 过拟合、交叉验证与数据泄漏

“按 photo_id 分组切分”这条纪律值得用一个真实事故来讲——它不是教科书上的“hypothetical”，而是本项目 2026-09-14 审计里真实发生过的（docs/RESULTS.md 有据可查）：校园照片语料上的 143 维模型，某次评估报出 AUC 0.8946、弱密度档检出率 85.3%；审计发现 414 行 JPEG 干净样本的 photo_id 被错误地逐行编号（同一张照片占了 414 个 id），按源图分组形同虚设——修正后同一模型同一数据，AUC 掉到 0.7555、弱档检出掉到 44.2%。跌掉的 0.14 不是模型变笨了，而是**虚假的乐观被挤掉了**：泄漏让模型在测试时“认出了照片”，而不是“认出了隐写痕迹”。

怎么防？两条不变量写进了代码：其一，任何训练前校验都要断言“同一 photo_id 的全部样本只出现在一侧”；其二，置信区间按**源图**重采样而不是按图（同源样本高度相关，按图算会把有效样本量虚报 5 倍）。把这两条当成本章最重要的 takeaway——它们比任何模型技巧都更能决定你报告里数字的可信度。

避坑提醒 |

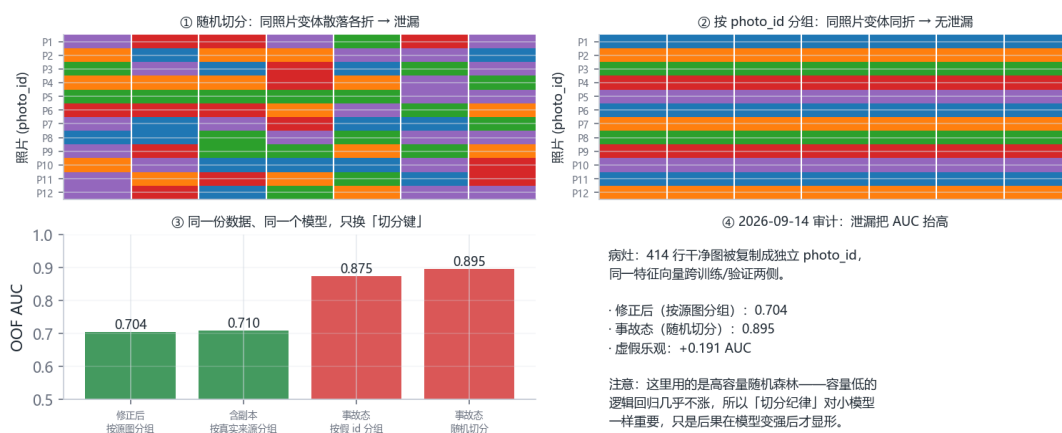
“随机切分”四个字在机器学习教材里默认无害，在隐写数据集上是定时炸弹。判断你的数据集需不需要按组切分，只问一个问题：**同一“源”会不会派生多个样本？**会——就必须按源分组。这个判据同样适用于医疗（同一病人多次拍片）、金融（同一客户多笔流水）等一切场景。

模型在“见过的数据”上表现好很容易，真正要测的是没见过数据——这叫泛化。如果模型把训练样本背下来，在测试集上就会崩，这就是过拟合。

防止过拟合的标准流程是交叉验证：把数据切成若干折，轮流用其中一折当验证集。但这里有一个隐蔽陷阱：同一张照片的 7 个样本高度相似，如果随机切分，训练集里可能出现某照片的“兄弟样本”，模型等于提前看到了测试照片——这叫数据泄漏，AUC 会虚高得离谱。

避坑提醒 | 项目用 GroupKFold：按照片 id 分组切分，保证同一张照片的所有变体要么全在训练集、要么全在验证集。src/train_model.py 的 cv_auc() 只做这一件事，却决定了整个实验的可信度——这是本项目最值得学习的机器学习工程细节之一。

过拟合与数据泄漏：随机切分让 AUC 虚高，按源图分组才是诚实的



本图为「机制复现」（11 维 RF）；手册 7.4 节事故原数字为 143d 模型 0.8946→0.7555，机制相同

图 7-3 过拟合与数据泄漏（分组 vs 随机）。这张图回答的是：不按源图分组，分数为什么是假的？随机 / 分组切分的折分配与“同源副本+独立 id”泄漏的 AUC 复现——不分组，分数就是假的

7.5 评估指标：别只看准确率

隐写检测的正负样本不平衡（干净少、含密多），且“误报一张干净图”和“漏掉一张含密图”代价不同，所以要用一组指标：

指标	回答的问题	直觉
准确率 accuracy	整体猜对比例	不平衡时容易骗人
精确率 precision	判为含密里有多少真含密	误报低→精确率高
召回率 recall	真含密里抓到多少	漏报低→召回率高
F1	精确率与召回率的调和平均	两者都想要时的折中
ROC / AUC	模型排序能力与阈值无关	AUC=0.5 等于瞎猜

模型输出的是概率，还要选一个阈值才能变成 0/1 判断。阈值越高越保守（误报少、漏报多）；阈值越低越激进（检出高、误报多）。项目训练时用 Youden 准则在“真阳率-假阳率”最大处选阈值，另外专门算一个“低误报点”阈值（假阳率 $\leq 10\%$ ），供严格档使用。

Youden 阈值（真实代码逻辑）

```
from sklearn.metrics import roc_curve
fpr, tpr, th = roc_curve(y_true, proba)
j = tpr - fpr                                # Youden J
best = float(th[np.argmax(j)])                # 离随机线最远的点
```

动手做 | 学完本节运行 `python src\train_model.py` 一次。你不需要读懂全部输出，先找到三行：CV-AUC、测试集 AUC、低误报点检出率。第 8 章将逐行解释它们的含义。

阈值一节再补两个项目里的真实设计决策。其一，Youden 点（真阳率-假阳率最大）是“默认操作点”，但隐写检测的真实场景里，**把一张干净照片判成含密**（冤枉好人）的代价往往高于漏掉一张弱嵌入图——所以模型另存一个“低误报点”阈值。其二，模型上线前在 1514 张**从未见过的真实干净照片**（校园 414 + DIV2K 100 + ALASKA#2 1000）上实测误报率并给出 Wilson 置信区间——在训练分布内测误报是自欺，OOD 测出来才算数（详见 8.8）。

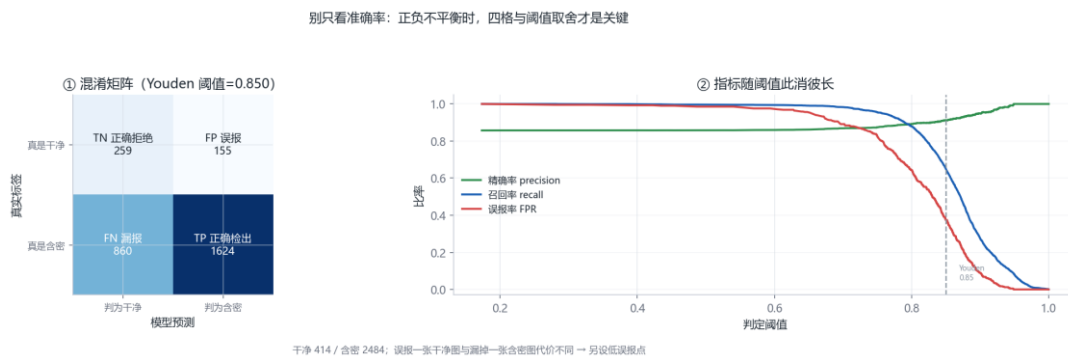


图 7-4 混淆矩阵与指标随阈值。这张图回答的是：阈值挪一下，指标会怎么变？Youden 阈值下的混淆矩阵与 precision / recall / FPR 曲线——阈值一挪，指标全变

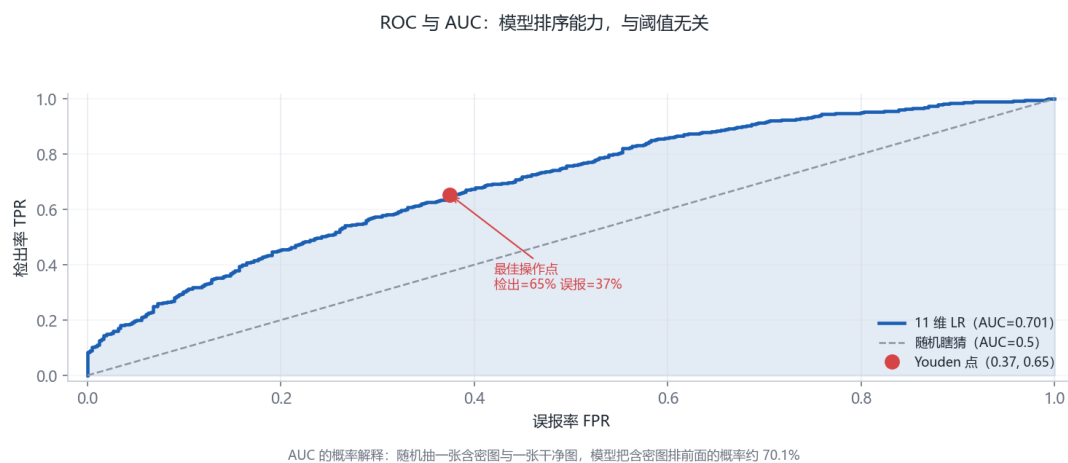


图 7-5 ROC 与 AUC（真实数据）。这张图回答的是：只看准确率，会怎样误判一个模型？按源图分组 OOF 的 ROC 曲线、AUC 与 Youden 操作点——只看一个准确率会误判模型

7.6 小结与自我检查

监督学习 = 用带标签样本学一个从特征到答案的函数；

随机切分在同源样本上会泄漏信息，GroupKFold 按组切分才是诚实的；

AUC 衡量排序能力，阈值决定实际操作点；

误报率与检出率是一对矛盾，Youden/低误报点是两种取舍。

想一想 | 如果一份论文的隐写检测 AUC=0.95 但没说怎么切分数据，你会先怀疑什么？把怀疑写下来，这就是你评估一切 ML 实验的起点。

本章小结。 监督学习四件套（数据/特征/模型/评估）在隐写场景的对应物：dataset.csv 的行、11/143 维特征、逻辑回归到 LightGBM、AUC 与按档检出率；交叉验证必须按 photo_id 分组，泄漏的代价是 0.14 的虚假 AUC；阈值是操作点，Youden 与低误报点是两种诚实的选择。

动手做 |

章末练习（提示与答案见附录 J）

P7.1（手算）把 7.3 节迷你例的学习率改为 10，重算一步更新后的 w 与新损失——观察“步长过大”的震荡。

P7.2（手算）4 个样本、2 折的交叉验证：写出“按 x 分组”与“按伪 photo_id 分组（同源样本同折）”两种切法，哪种更合理？为什么？

P7.3（思考）混淆矩阵四个格子里，“把干净图判成含密”对应哪一格？在取证场景下它的社会成本是什么？

P7.4（上机）用 sklearn 对 7.3 节的 4 样本例跑 LogisticRegression，核对 w 、 b 与手算方向一致。

P7.5（思考·选做）“模型 A 在校园语料 AUC 0.89，模型 B 在 BOSSbase 上 0.81，A 比 B 强”——这句话错在哪？用 RESULTS.md 的原话回答。

第 8 章 · 用机器学习做隐写检测（第 8–9 周）

本章导航 · 先看问题，再读正文

核心问题 为什么“深度学习直接看像素”在这个任务上会失败？

读完你能回答

- 说清 11 维特征与 143 维（含 SRM）的构成与“出身”；
- 解释裸 CNN 在 BOSSbase 上只有 0.5007 意味着什么；
- 读懂按档检出率，说明“强密度高、弱密度明显下降”；
- 说清 OOD 误报率为什么是部署前的最后一关。

“更好的模型”不是默认成立的——本章用一组真实数字把它拆开来。

开场问题：0.5007 与 0.9541，同一个任务、同一份数据，两个模型差了一整个太平洋——差在哪？这两个 AUC 都是本项目在 BOSSbase 基准上的真实实测值（详见 docs/RESULTS.md）：前者是“裸 CNN 直接看像素”，后者是“固定 SRM 高通滤波 + 小型卷积网络”。中间 0.45 的差距没有一行魔法——它全部来自**领域知识被放在了管线的哪个位置**。本章讲完，你应当能自己解释这张表：

模型	特征	AUC	一句话诊断
裸 CNN (xuNet 结构)	原始像素	0.5007	学到了内容，没学到痕迹
固定 SRM + CNN (yenet)	SRM 残差	0.9541	领域知识放在第一层，CNN 学高层
LGB-11d	手工 11 维	0.7128	小样本下可靠的教学基线
LGB-143d	SRM 90 + 其余 53 维	0.8062	默认部署版：容量与稳健的平衡
LGB-53d	可解释子集	0.7172	每一维都能讲出故事

小节	回答的问题	你的产出
8.1 为什么不用 CNN	路线选择的依据是什么？	能复述三条理由与两个 AUC
8.2 认识 11 特征	每个特征在度量什么？	能把特征名与第 3 章的统计量对上
8.3 数据设计	“1+6”变体为什么这样配？	能解释强中弱三档的用意
8.4 训练流水线	“诚实评估”由哪几层组成？	能画出训练/校准/测试三层图
8.5 读结果	AUC 与按档检出率怎么读？	能向他人解释一张 ROC 图
8.8 SRM 与双版本	v2 升级升在哪、坑在哪？	能讲出 OOD 部署的真实教训

动手做 | 本章目标：理解“为什么是 11 维统计特征而不是裸像素 CNN”；认识每个特征在测什么；跑通数据生成→训练→评估→GUI 判定全链路。

8.1 先回答一个反直觉的问题：为什么不用 CNN 直接看像素？

深度学习直接吃像素（“裸 CNN”）在这个任务上并不好用。项目在 414 张照片上试过多架构深度卷积网络，验证 $AUC \approx 0.50$ ，即完全无法泛化。原因值得细品：

隐写改动的是极弱的高频信号（LSB 层），信噪比远低于物体、纹理等视觉内容；

CNN 很容易学到“照片内容”而不是“隐藏痕迹”，一旦测试照片风格不同就失效；

真正独立的样本数量是照片数（几百张），而深度模型通常需要成千上万张独立源图。

于是项目选择“领域知识做特征 + 简单分类器”路线：手工设计能直接度量 LSB 统计足迹的特征，再用逻辑回归/树模型判别。v1 时代 11 维特征在小样本上的 AUC 约 0.75 ~ 0.79；v1.4 升级到 143 维 + LGB 双版本后达到 0.90+（见 8.8），但“特征法优于裸 CNN”的基本结论不变。

新手提示 | 这不是“深度学习无用”的结论，而是“在样本量约束下如何选建模策略”的方法论课：先用可解释的强特征验证信号是否存在，再考虑更复杂的模型。很多工程问题都遵循这个次序。

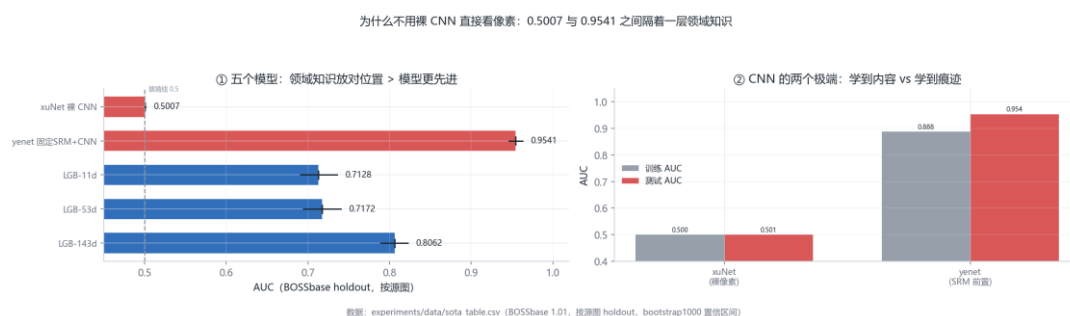


图 8-1 隐写检测的模型全景与路线之争。这张图回答的是：为什么“深度学习直接看像素”在这题上会失败？五个模型的 AUC 与置信区间，以及两个 CNN 的训练/测试落差——裸 CNN 为什么“稳定地不会”

8.2 认识 11 个特征

11 维特征不是 11 个孤立的数，而是第 3 章三套统计量的“变量名”。把名字与来历对上，每个特征就都有了性格（特征名与 featurize_v2.py 的 BASE_11 一致）：

特征名	出身	在度量什么
Rm / Sm	RS 分析	M/F 翻转后变平滑的组占比——位面结构的正反面
Rn / Sn	RS 分析（负掩码）	负翻转下的对照占比，用于扣除图像本底的对称性
RS_Gr / RS_Gn	RS 分析	(R-S)/N 的正负两版——3.4 节的 Gn 全家
chi2_pvalue / chi2_stat	卡方检验	偶奇配对被拉平的程度（p 高越可疑）
diff_entropy / lsb_diff_entropy	信息论	相邻像素差分的熵——纹理本底的“随机度基线”
median_prefix_p	卡方前缀族	20 个前缀段 p 值的中位数——局部随机化的探测器

注意它们的设计哲学：**每一个特征都自带“嵌入后会怎么变”的预测**——嵌入后 Rm 降、chi2_pvalue 升、lsb_diff_entropy 变化。机器学习在这里的角色不是“发明规律”，而是“把十几个

各有盲区的预测器加权成一个决策”。这也是 8.1 节“领域知识做特征”的含义：特征的解释性让模型出错时你知道去哪查。

新手提示 |

差分熵是什么？把相邻像素的差分（右邻减自身）画成直方图，它的 Shannon 熵就是 `diff_entropy`。平滑图的差分集中在 0 附近（熵低），纹理图的差分散得开（熵高）。嵌入会把 LSB 层的差分直方图“抬平”——所以 `lsb_diff_entropy` 是专门盯着 LSB 层的这一份。

v1 的 11 维特征全部由 C++ 库 `cpp/fsfeatures.dll` 计算（Python 绑定在 `src/fsfeatures.py`）；v2 的 143 维特征在 `src/featurize_v2.py` 中组装。按“测什么”可以分成三组：

特征	含义	嵌入后会怎样
Rm / Sm / Gr	正掩码下常规/奇异组数与缺口	Gr 下降（结构受损）
Rn / Sn / Gn	负掩码下常规/奇异组数与缺口	Gn 显著塌缩（核心信号）
chi2_stat / chi2_pvalue	卡方统计量及其 p 值	灰度对趋向均匀，p 上升
diff_entropy	灰度差分熵：图像本底粗糙度	用于识别天然噪声图
lsb_diff_entropy	LSB 位面差分熵	随机化后趋近 1
median_prefix_p	20 个前缀段卡方 p 的中位数	整体随机化时升高
回到项目看代码 打开 <code>src/steganalysis.py</code> ，找到 <code>diff_entropy()</code> 与 <code>lsb_diff_entropy()</code> ：它们用相邻 像素差值的香农熵估计“随机度 基线”。再打开 <code>src/fsfeatures.py</code> ，看 11 维特征 是如何排列并交给 C++ 的。	回到项目看代码 打开 <code>src/steganalysis.py</code> ，找到 <code>diff_entropy()</code> 与 <code>lsb_diff_entropy()</code> ：它们用相邻 像素差值的香农熵估计“随机度 基线”。再打开 <code>src/fsfeatures.py</code> ，看 11 维特征 是如何排列并交给 C++ 的。	回到项目看代码 打开 <code>src/steganalysis.py</code> ，找到 <code>diff_entropy()</code> 与 <code>lsb_diff_entropy()</code> ：它们用相邻 像素差值的香农熵估计“随机度 基线”。再打开 <code>src/fsfeatures.py</code> ，看 11 维特征 是如何排列并交给 C++ 的。

特征不是 11 个孤立的数：每个都自带「嵌入后会怎么变」的预测

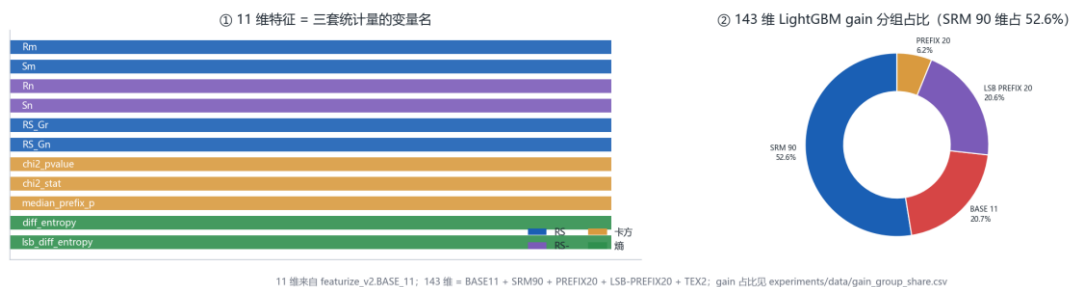


图 8-2 11 维特征的出身与 143 维的构成。这张图回答的是：11 维和 143 维特征各由什么组成？11 维特征按统计量家族着色，及 143 维的 gain 分组占比——先懂特征，再谈模型

8.3 数据怎么来：make_dataset.py 的“1+6”设计

src/make_dataset.py 对每张源照片生成 1 张干净图 + 6 个含密变体（v1.4 另支持 12 档 v2 变体，见 8.8）：

变体（方法, p, 密度）	强度	目的
clean	—	提供干净的负样本
nsF5, p=3, d=0.25	弱	贴近真实隐蔽通信的弱密度
nsF5, p=3, d=0.55	中	中等密度，检验一般强度
nsF5, p=2, d=0.35	中弱	p 越小效率越低，痕迹更强
nsF5, p=2, d=0.85	强	强嵌入，统计足迹明显
matrix, p=3, d=0.50	中	对比另一方法
matrix, p=2, d=0.80	强	普通矩阵编码的高密度
v1.4 再 +6 档	—	nsF5 p3 d0.40; matrix p3 d0.40/0.60; lsb d0.30/0.50/0.70

所有图统一缩放为 512×512 灰度；特征全部由 C++ 提取，含密变体由 C++ 嵌入路径生成，保证训练与推理一致。

8.4 训练脚本怎么读：train_model.py

python src\train_model.py 的流程是一条完整的“诚实评估流水线”：

读表并删除特征缺失的行；

按 photo_id 用 GroupShuffleSplit 预留 25% 作为最终测试集（本组照片绝不进训练池）；

训练池上做 5 折 GroupKFold 交叉验证，比较逻辑回归 / 随机森林 / 梯度提升 / XGBoost；

选出 CV-AUC 最高的模型，在训练池内再切 20% 作为校准集选阈值（Youden 与低误报点）；

用全部训练池重训，在 held-out 测试集上报 acc / balanced acc / AUC / 分类报告；

按 (method, p, density) 分组报告每档检出率；

保存模型与阈值到 models/stego_classifier.joblib（v1.4 起支持 LGB 网格调优、Stacking 与 53d 可解释版另存）。

避坑提醒 | 校准集用于挑阈值，测试集只许用一次。如果反复拿测试集调阈值，测试集就“脏”了。项目用三层结构——训练/校准/测试，是工业界也认可的严谨做法。

8.5 怎么读结果：ROC、AUC 与按档检出率

ROC 与 AUC：排序能力与阈值无关，操作点由你选

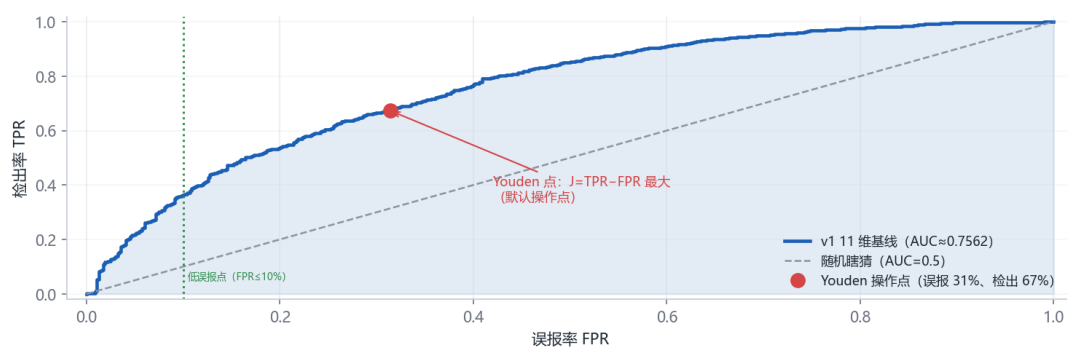


图 8-3 隐写检测 ROC 与 AUC (v1 基线)。这张图回答的是：v1 基线的 ROC 与操作点长什么样？v1 数据集 11 维基线的 ROC 曲线、Youden 操作点与低误报点（按照片分组留出外折）

ROC 曲线把阈值从 1 扫到 0，画出“检出率 vs 误报率”。AUC 是曲线下面积：0.5 是瞎猜。v1 基线上 0.75 量级说明“大部分时候能把含密图排在干净图前面”，但弱密度仍容易漏；v1.4 双版本模型（见 8.8）把 8-split 平均 AUC 做到 0.8980（143d）/ 0.8461（53d），口径见 docs/RESULTS.md。

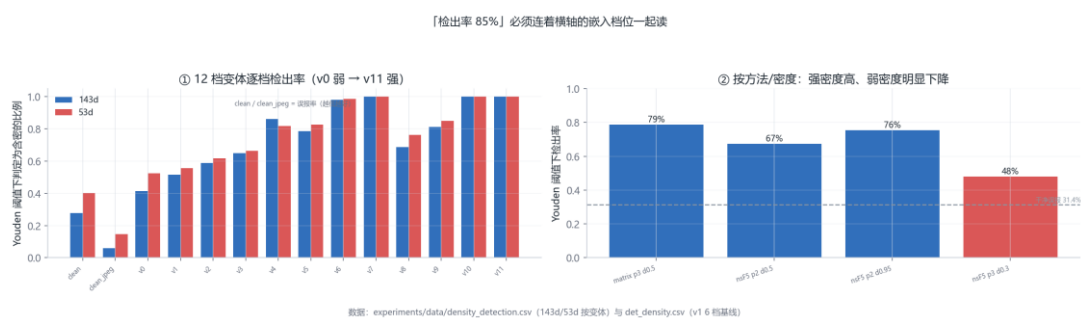


图 8-4 按档检出率（逐变体与按方法/密度）。这张图回答的是：强密度和弱密度的检出率差多少？v1 数据集 6 档基线的逐档检出率与按方法/密度汇总（Youden 阈值）：强密度高、弱密度明显下降

回到项目看代码 | 对比 experiments/data/density_grid.csv（第 5 章实验产出的真实统计）：lsb 各档几乎 100% 检出，matrix p2 d0.80 约 95%，而弱档 nsF5 p3 d0.25 只有约 47%、nsF5 p3 d0.55 约 55%。这说明“弱嵌入更难检测”不是玄学，而是统计足迹随密度衰减的直接结果。

AUC 有一个值得刻在脑子里的概率解释：**随机抽一张含密图和一张干净图，模型给含密图更高分的概率。**LGB-143d 在 BOSSbase 上 0.8062，读作“十次对比里约八次排对”；xuNet 的 0.5007 读作“跟抛硬币没区别”。这个解释让 AUC 不依赖任何阈值，也因此它回答不了“设了阈值之后误报多少”——那是 ROC 曲线上你选的那个点的责任。

按档检出率（图 8-2）则回答另一个更残酷的问题：**嵌入率降到多少时检测会失效。**v1 基线的曲线是“强密度 90%+、弱密度掉到六成”；v2 143 维把整条曲线上抬，但最弱档依然最难——这不是模型不行，而是信息论在设限：0.2% 像素改动的统计痕迹，本来就逼近自然图像自身的噪声地板。读实验报告时，先看横轴的嵌入档位再看数字，否则任何“检出率 85%”都是无意义的口号。

想一想 |

同一个 143d 模型，在校园语料上 AUC 0.8939、在 BOSSbase 上 0.8062——模型没变，数字差了 0.09。回到 7.4 节的“按组切分”与 RESULTS.md 的语料表，列出至少两条可能造成差异的语料属性（提示：图像内容风格、负样本构成、分辨率与 JPEG 历史）。

AUC 的概率解释写成式子（Mann-Whitney 统计量的归一化形式）：

$$AUC = P(s_1 > s_0) = \frac{1}{n_1 n_0} \sum_i \sum_j 1[s_{1i} > s_{0j}]$$

其中 s_1 是含密图的模型得分、 s_0 是干净图的得分，指示函数 $1[\cdot]$ 在括号内为真时取 1。于是 $AUC = 0.8062$ 读作：随机抽一张含密图与一张干净图，模型把含密图排在前面的概率约 80.6%。它绕过了阈值——这也正是它回答不了“误报率多少”的原因：误报率是曲线上你选的那个点的纵坐标，由 Youden 或低误报准则决定。



图 8-5 诚实评估流水线（训练/校准/测试三层）。这张图回答的是：一个诚实的评估流程要经过哪几步？从读表到分档报告的六步流水线与两条分组不变量——评估流程本身也要经得起审计

8.6 灵敏度：在 GUI 里如何工作

`src/ml_predict.py` 的 `MLPredictor` 预处理好图像（转灰度→ 512×512 ）后，按模型记录的特征列数自动走 $v1\ 11$ 维或 $v2\ 143$ 维特征提取，再输出含密概率。灵敏度三档通过切换阈值工作：

灵敏度	ML 阈值	效果
严格（低误报）	<code>threshold_low_fp</code>	干净图误判最少，弱含密更容易漏
均衡	Youden 阈值	检出与误报平衡，默认档
宽松（高检出）	$\text{Youden} \times 0.75$	弱密度检出更多，误报略升

GUI 的“分析”页会把启发式概率与 ML 概率并排显示，要求你交叉印证，而不是只看一个数——这是项目刻意保留的诚实设计。

8.7 动手实验：从训练到单图判定

完整链路（ $v1$ 数据集； $v2$ 与双版本命令见 8.8）

```
# 1) 生成数据集（可用自己的照片目录替代 F:\DCIM\Camera）
python src\make_dataset.py
```

```
# 2) 训练与评估
python src\train_model.py

# 3) 对单张图做 ML 判定
import sys; sys.path.insert(0, "src")
import numpy as np
from PIL import Image
from ns5_core import embed_string
from ml_predict import get_predictor
a = np.asarray(Image.open("img/cover.png").convert("L"))
s, _, _ = embed_string(a, "ML test", method="nsF5", p=3)
print(get_predictor().predict(s))
```

动手做 | 做一个“诚实性小实验”：先对一张照片做 ML 判定（应接近干净），再嵌入弱档消息（nsF5 p=3、密度 0.25 附近）重新判定。观察概率是否只微弱上升。这正是 README 里“弱密度需交叉印证”警告的现场体验。

8.8 v1.4.0 更新：SRM、143 维特征与双版本模型（2026-09）

8.1–8.7 描述的是 v1.0–v1.3 的路线：11 维特征 + LR/XGB，AUC 约 0.75–0.79。v1.4.0 沿三条线做了升级：SRM 高通滤波预处理、143 维 v2 特征与 12 档变体、以及最终的 LightGBM 双版本模型。下面按项目实际演进顺序讲，帮助你读懂 README 中那一长串实验表。

8.8.1 SRM 高通滤波：同源有收益，跨源反而亏

SRM（Spatial Rich Model）是 Fridrich 体系的一组高通滤波核。src/srm_filter.py 实现 30 个标准核（一阶差分 8 + 二阶 4 + 三阶 8 + 边缘 $3 \times 3 / 5 \times 5$ 共 8 + 方形 $3 \times 3 / 5 \times 5$ 共 2），对图像逐核滤波后取最大绝对响应、clip 到 ± 4 再缩放成单通道“增强图”，喂给原有特征器。CPU（numpy）与 GPU（torch）双实现，接入开关是 make_dataset.py --preprocess srm 与 GPU 的 use_srm=True。

指标	同源原图基线	同源 SRM 增强
5 折 CV-AUC	0.7594	0.7922
held-out 测试 AUC	0.7811	0.8085
弱档 nsF5 p3 d0.25 检出	51.9%	62.5%
nsF5 p2 d0.35 检出	76.0%	90.4%

指标	同源原图基线	同源 SRM 增强
低误报点含密检出	41.2%	50.3%
干净误报（低误报阈值）	9.6%	9.6%
避坑提醒（v1 时期对照，数字未在当前版本复现） 当年观察到：SRM 预处理的收益只在“同源”数据内出现。跨源合并（校园 + BOSSbase 全量）时，SRM 反而把 CPU 测试 AUC 从 0.741 拉到 0.704、GPU 验证 AUC 从 0.651 拉到 0.572——高通滤波在压掉图像内容的同时，也压平了不同相机/压缩源之间的可分性。因此默认模型采用未 SRM 的合并全量；SRM 单源校园模型另存为 campus_srm 专用。	避坑提醒（v1 时期对照，数字未在当前版本复现） 当年观察到：SRM 预处理的收益只在“同源”数据内出现。跨源合并（校园 + BOSSbase 全量）时，SRM 反而把 CPU 测试 AUC 从 0.741 拉到 0.704、GPU 验证 AUC 从 0.651 拉到 0.572——高通滤波在压掉图像内容的同时，也压平了不同相机/压缩源之间的可分性。因此默认模型采用未 SRM 的合并全量；SRM 单源校园模型另存为 campus_srm 专用。	避坑提醒（v1 时期对照，数字未在当前版本复现） 当年观察到：SRM 预处理的收益只在“同源”数据内出现。跨源合并（校园 + BOSSbase 全量）时，SRM 反而把 CPU 测试 AUC 从 0.741 拉到 0.704、GPU 验证 AUC 从 0.651 拉到 0.572——高通滤波在压掉图像内容的同时，也压平了不同相机/压缩源之间的可分性。因此默认模型采用未 SRM 的合并全量；SRM 单源校园模型另存为 campus_srm 专用。

8.8.2 v2 143 维特征与 12 档变体

src/featurize_v2.py 在 11 维基础上增加：30 个 SRM 残差的均值/绝对均值/标准差（90 维）、20 段整图前缀卡方 p、20 段 LSB 前缀卡方 p，以及 texture_noise 与 est_rate（2 维），合计 143 维。训练变体从 6 档扩展到 12 档：追加 nsF5 p3 d0.40、matrix p3 d0.40/0.60，以及 lsb d0.30/0.50/0.70。

严格 A/B（同一测试集照片分组、5 折 GroupKFold OOF）：v2 数据集本身带来 +0.019（同 11 维时 0.7868 vs 0.7678）；在 v2 数据集上再加 143 维又 +0.027（OOF 0.8143、held-out 0.8278）。累计约 +0.05 AUC。

8.8.3 真实 JPEG 干净图暴露的部署问题与修复

v2 LR 在训练分布内 AUC 很高，但部署到真实 JPEG 干净照片时几乎全判 1.0：SRM 残差对 JPEG 高频噪声过于敏感。修复办法是把 414 张真实 JPEG 干净图加进训练，得到 dataset_campus_v2_jpeg.csv，再做 LGB 网格调优。【2026-09-14 审计更正】这批 JPEG 干净行当初被赋予了独立 photo_id，而其特征与对应 clean 行逐位相同（是副本）：同一张源图的样本因

此可以跨训练/验证两侧，“按源图划分”名存实亡、指标被抬高；同一批 GPU 特征还把像素先除以 255 再过高通滤波，与推理端 CPU 特征相差约 30 倍。两处都已修复并重跑（修复方式见 experiments/add_jpeg_clean.py，细节见 docs/RESULTS.md 与 CHANGELOG 1.6.2）。现口径：143d held-out AUC 0.8939、8-split 平均 0.8980、弱档 nsF5 p3 d0.25 检出 50.0%；53d 为 0.8391 / 0.8461 / 41.4%。

8.8.4 双版本部署策略：143d 稳健 vs 53d 可解释

可解释性实验（2026-09-14 按修正后的特征口径重算）：143 维里 SRM 90 维贡献了 52.6% 的 LightGBM gain，BASE 11 占 20.7%、LSB PREFIX 20 占 20.6%、PREFIX 20 占 6.2%；消融显示去掉 SRM 90 维后 5 折 OOF AUC 从 0.9010 掉到 0.8513（约 +0.05 AUC）。也就是说修好尺度之后 SRM 不是“噪声特征”。于是项目保留两套模型：

维度	143d 默认（稳健）	53d 可解释
特征构成	BASE11 + SRM90 + PREFIX20 + LSB-PREFIX20 + TEX/EST2	去 SRM 后的 53 维
8-split 平均 AUC	0.8980	0.8461
Held-out AUC	0.8939	0.8391
弱档 nsF5 p3 d0.25	50.0%	41.4%
真实 JPEG 干净 OOD	9.58% (n=1514, 95%CI 8.20–11.16%)	28.86% (n=1514, 95%CI 26.64–31.20%)
推荐场景	通用部署 / 异构数据 / 真实图	教学 / 答辩 / 单图可解释（AUC 略低约 0.05）

MLPredictor() 默认加载 143d；传入 model_path 可切 53d。推理时会按模型记录的特征列数自动选择 11 维或 143 维特征提取。GUI 的模型单选框仍在规划中，当前通过 API 切换：

v1.4.0 双版本模型切换示例

```
from ml_predict import MLPredictor

# 默认：143d 稳健版 (models/stego_classifier.joblib)
pred_143 = MLPredictor()

# 53d 可解释版 (去 SRM 90 维, AUC 略低、但可逐维解释)
```

```
pred_53 = MLPredictor(
    model_path='models/stego_classifier_v2_jpeg_lgb_51d.joblib',
    clip_outliers=False)
```

```
r = pred_53.predict(img)
# {'probability': ..., 'verdict': ..., 'threshold': ...}
```

动手做 | 实验任务：用 `$env:DS_FILES = "dataset_campus_v2_jpeg.csv"` 重新训练，分别用 143d 与 53d 对“真实 JPEG 干净图”和“弱档 nsF5 p3 d0.25 嵌入图”做判定，把 AUC 与 OOD 行为写入实验报告。

避坑提醒 | v2/53d 模型只在训练分布附近可靠：训练含校园照片（PGM/BMP 灰度派生）与已加入的 JPEG 干净样本。对全新相机/压缩源，先做小规模 OOD 测试再谈部署。

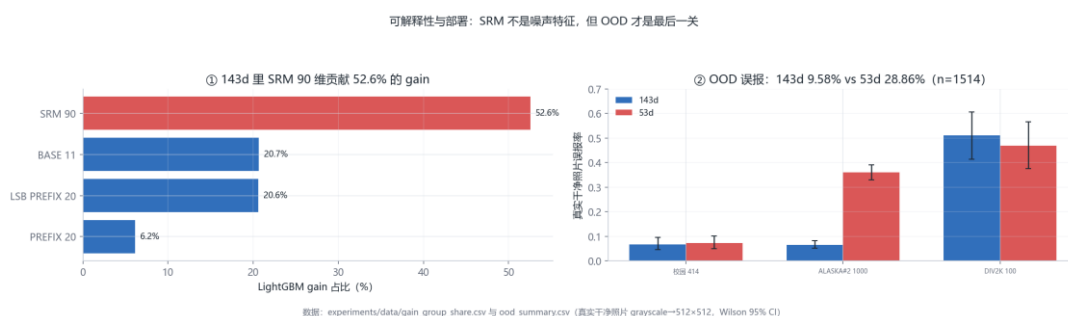


图 8-6 特征 gain 可解释性与 OOD 误报率。这张图回答的是：模型没见过的新分布，才是部署前的最后一关吗？

143d 的 gain 分组占比与按来源的 OOD 误报率（含置信区间）——模型没见过的新分布才是真边界

8.9 小结与自我检查

小样本 + 弱信号场景下，领域特征 + 简单模型通常比裸 CNN 更可靠；

11 个特征 = RS 家族 + 卡方家族 + 熵/随机度基线；

训练/校准/测试三层结构 + GroupKFold 是实验可信度的骨架；

ML 概率与启发式判读应交叉印证，弱密度检测有其物理上限。

v1.4 之后双版本模型在训练分布内已很强，但真实部署前仍必须做 OOD 验证：本项目在 1514 张真实干净照片（校园 414 + DIV2K 100 + ALASKA#2 1000）上实测，143d 误报 9.58%（95%CI 8.20–11.16%）、53d 28.86%（26.64–31.20%）；DIV2K 这类 2K 高清图是主要失分来源（143d 约 51%）。评估脚本 `experiments/ood_eval.py`，结果见 `docs/RESULTS.md` 第 5 节。

想一想 | 数据集中同一张照片有 7 个样本，为什么不能用“随机 80/20 切分”？请设计一个最小例子说明泄漏会让 AUC 虚高多少。

本章小结。路线之争的答案是“领域知识的位置”：裸 CNN 0.5007、固定 SRM 前置的 yenet 0.9541、手工特征 + LGB 0.71 ~ 0.81——知识放对地方比模型更先进更重要；11 维特征每个都有第 3 章的出身；训练/校准/测试三层 + 按源图分组是诚实的骨架；OOD 误报率是部署前的最后一关，1514 张真实干净照片是本项目交出的答卷。

动手做 |

章末练习（提示与答案见附录 J）

P8.1（思考）把 8.1 节主表里的五个模型按“领域知识含量”从高到低排序，再看 AUC 排序——两者一致吗？有没有反例，怎么解释？

P8.2（手算）一个 ROC 点：阈值取最高时检出率 0%、误报率 0%；取最低时 100%/100%。若模型完美，曲线长什么样？AUC 多少？

P8.3（上机）跑 8.7 节完整链路（数据生成→训练→单图判定），把你自己的照片目录喂进去，记录 AUC 与混淆矩阵；对比 README 的数字并解释差异来源。

P8.4（思考）8.8.3 的部署事故里，“真实 JPEG 干净照片被全判 1.0”的根因是 SRM 残差对 JPEG 噪声过敏。如果反过来把训练数据全部换成 JPEG 图、部署到 PNG 场景，你会预期什么问题？

P8.5（思考·选做）53d 可解释版牺牲 0.09 的 AUC 换“每一维可解释”。列举两个必须选 53d 的场景与两个必须选 143d 的场景。

第 9 章 · 工程化：C++、GPU、GUI 与测试（第 10 周）

本章导航 · 先看问题，再读正文

核心问题 同一份算法代码，凭什么说它“工程上可靠”？

读完你能回答

- 画出四层架构，并举例说明“缺失即回退”的降级路径；
- 解释**对拍**（differential testing），以及“先测量、再下沉”的热路径纪律；
- 说明 GPU 一致性的难点，与“只吃可并行的归约”这条设计原则；
- 列出测试金字塔的五层与六项冻结冒烟各自守护什么。

算法正确只是底线；本章把“工程可靠”拆成一张可以执行的清单。

开场问题：同一份算法代码，凭什么说它“工程上可靠”？ 算法正确（第 5 章的往返自检）只是底线。工程可靠性要回答三个更难的问题：慢的环节能不能快（性能）、快的环节会不会算错（一致性）、没有加速硬件的用户能不能用（降级）。本章的项目给出了教科书级的答案：C++ 对拍、GPU 批量化、DLL 缺失自动回退、六项冻结冒烟。读完本章，你应当能把“工程可靠”从一句口号拆成一张可以执行的清单。

小节	回答的问题	你的产出
9.1 分层	代码怎么切成四层？	能画出分层图并举例说明回退
9.2 C++ 加速	热路径怎么找、怎么保证不错？	能解释“对拍”（differential testing）
9.3 GPU 版	批量向量化换了什么？	能说出 GPU 一致性的难点
9.4 GUI	教学演示怎么设计？	知道 matrix_demo/scan_panel 的用途
9.5 测试与 CI	重构的安全网是什么？	能列出六项冻结冒烟

动手做 | 本章目标：看懂“算法”如何变成“软件”；理解为什么需要 C++ 与 GPU 加速、如何保证跨语言一致；会运行测试与 CI 流程。

9.1 系统架构：分层而不耦合



图 9-1 系统四层架构。这张图回答的是：代码为什么这么分层？缺一层会怎样？加速层 / 算法核心 / 功能层 / 界面层与缺失回退路径——分层而不耦合

加速层: `cpp/fsfeatures.dll` (特征)、`cpp/nsf5embed.dll` (嵌入/置乱)；

算法核心: `ns5_core.py`——汉明码、湿纸、哈希键控、嵌入/解码；

功能层: `steganalysis.py`、`efficiency.py`、`make_dataset.py`、`train_model.py`、`ml_predict.py`；

界面层: `gui.py + matrix_demo.py + scan_panel.py`, 所有功能一键可点。

分层的意义: 算法层不依赖 GUI, 可以在无界面环境 (服务器/CI) 里跑; 加速层 缺失时自动回退 Python, 保证功能永远可用。

9.2 C++ 加速：为什么是“热路径”，以及如何保证没错

Python 逐元素循环极慢，但隐写有两个典型热路径：确定性置换（把 1600 万 像素的顺序洗一遍）与特征提取（对每张图做 RS/卡方/熵统计）。把它们下沉到 C++ 动态库，性能可以提升几个数量级：置换加速随规模变化（数据见 `experiments/data/bench_permute.csv`，可用 ``python experiments/tools/gen_bench.py --only permute`` 复现；性能与机器相关）： 4096^2 （1600 万像素） $P_y \approx 15.6$ 秒 $\rightarrow$ C++ ≈ 0.24 秒（约 65 倍），同一份基准里小规模（6.5 万）最高约 240 倍。。

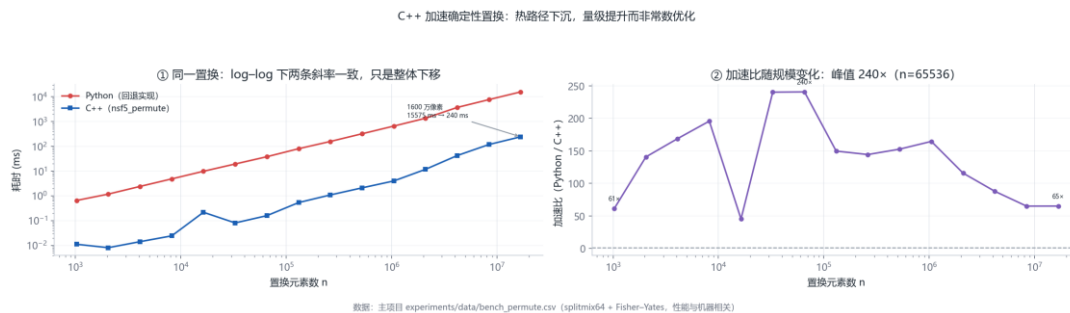


图 9-2 确定性置换性能。这张图回答的是：热路径下沉到 C++，实测快了多少？Python vs C++ 确定性置换耗时（log-log）与加速比曲线——热路径下沉的实测收益

性能提升的前提是“结果完全一样”。Python 调 DLL 用的是 ctypes，项目为此设计了自校验：

cppembed.py 用同一图像分别走 C++ 与 Python 路径嵌入，比较输出是否像素级一致；

fsfeatures.py 的 check_against_python() 逐特征核对 C++ 与 Python 结果；

DLL 缺失或加载失败时自动回退 Python 同算法，保证嵌入/解码两端序列恒定可逆。

避坑提醒 | 跨语言一致性是“可加速”的前提，也是调试的雷区：C++ 端位运算、取整、溢出行为与 Python/NumPy 不完全一致。遇到“有 DLL 能嵌、无 DLL 解不了”的诡异 bug，先跑 cppembed.selfcheck() 和 fsfeatures.check_against_python()，而不是去改算法。

“对拍”（differential testing）值得单独讲透，因为它是数值工程里最便宜的正确性武器：同一个函数写两份实现（一份 Python 参考实现、一份 C++ 提速实现），喂**完全相同的输入**，要求输出**逐位一致**。项目里确定性置换的对拍就是范例——固定种子、固定数组规模，Python 版与 DLL 版各自洗牌，diff 两个输出序列，必须一字不差。为什么这么较真？因为置换序列一旦有哪怕一位不同，嵌入与解码两端立刻失配（第 6 章的自同步全靠序列一致）。“提速”与“保真”之间的这座桥，就是对拍。

热路径怎么找也有纪律：**先测量，再下沉**。用 cProfile 跑一次大图嵌入，看时间花在哪——实测会发现时间集中在两处：纯 Python 的逐元素洗牌循环，与逐像素的特征统计。其余代码（哈希、编码、文件 IO）占比不到一成，下沉它们是白费力气。“优化前先测量”这六个字，是 CSAPP 从头到尾反复敲的黑板字。

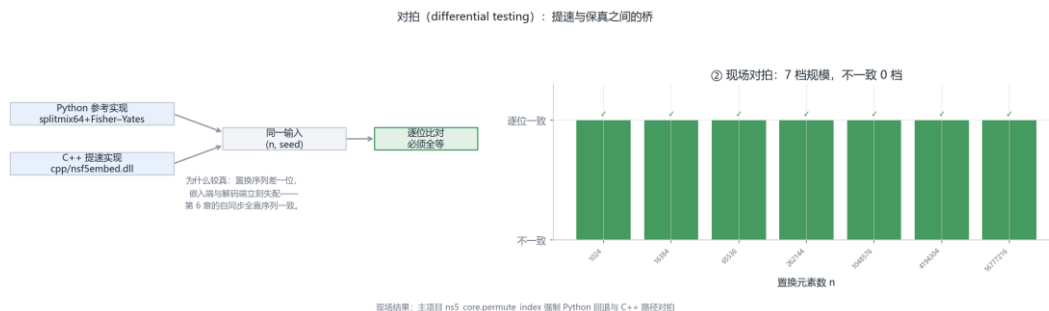


图 9-3 对拍：Python vs C++。这张图回答的是：提速之后，怎么保证 C++ 版没有算错？对拍机制与逐档一致性的现场结果——提速与保真之间靠“逐位一致”搭桥

9.3 GPU 版：把 11 维特征“批量向量化”

gpu/ 目录用 PyTorch 把特征计算改写成张量算子：一次读入一批 512×512 灰度图，在 GPU 上并行完成直方图、RS、卡方与熵计算，再落回 CPU。v1.4 新增 gpu/featurize_v2_gpu.py 支持 143 维 v2 特征（含 SRM 卷积）。README 实测 v1 特征 2070 张约 5 秒（ ≈ 410 张/秒），且与 CPU 参考实现逐位一致（浮点误差 $\sim 1e-7$ ）。

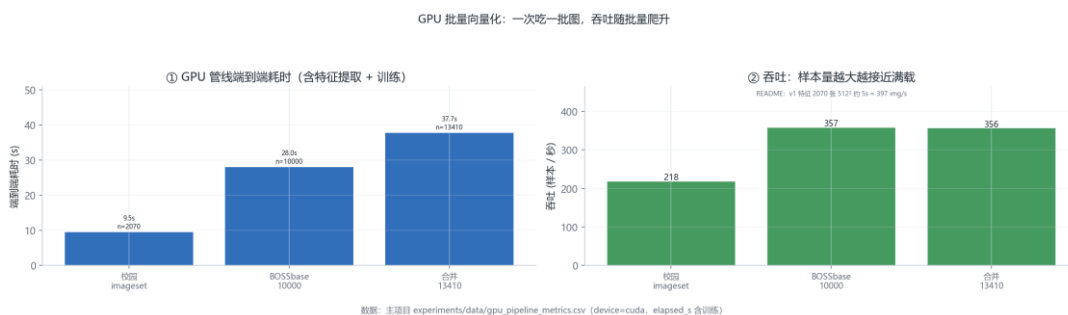


图 9-4 GPU 批量管线。这张图回答的是：GPU 批量化换来了多少吞吐？v1 11 维特征提取吞吐：CPU(C++) vs GPU(torch 批量) 与端到端耗时

回到项目看代码 | 读 gpu/featurize_gpu.py 的自检函数：它把 GPU 结果与 src/fsfeatures.py 的 CPU 结果逐元素比较。bit 级一致是这套双实现能并存的前提。再读 README 关于吞吐边界的讨论：小批量时主机↔设备拷贝开销可能让 GPU 反而不如 C++——这是诚实评测，不是“GPU 一定更快”的营销话术。

GPU 一致性比 CPU 对拍更难，原因是浮点：同一串加法在 GPU 上可能改变求和顺序，结果差在最后几位有效数字上。项目的对策是**把特征设计成对顺序不敏感的统计量**（直方图计数、均值、熵），并在管线里先跑 CPU 再跑 GPU、逐特征比对误差上限。这给出一条通用原则：**能**

在 GPU 上安全加速的，首先是“可交换、可并行的归约”；带复杂依赖的串行逻辑（比如湿纸求解）留在 CPU，不是技术不行，是设计对。

9.4 GUI：把研究工具变成可教学的应用

src/gui.py 的主界面围绕“嵌入→解码→分析”三步设计。真正出彩的是两个教学面板：

矩阵编码演示（matrix_demo.py）：随机/手点一个块，实时计算 s、m、d，高亮命中的列与被改系数，直观展示“至多改 1 位”；

载荷扫描（scan_panel.py）：拖动 payload 0→0.4，逐档重新嵌入并刷新卡方 p 值、RS 估计率与 ML 概率三条曲线，观察“藏得越满越容易被发现”。

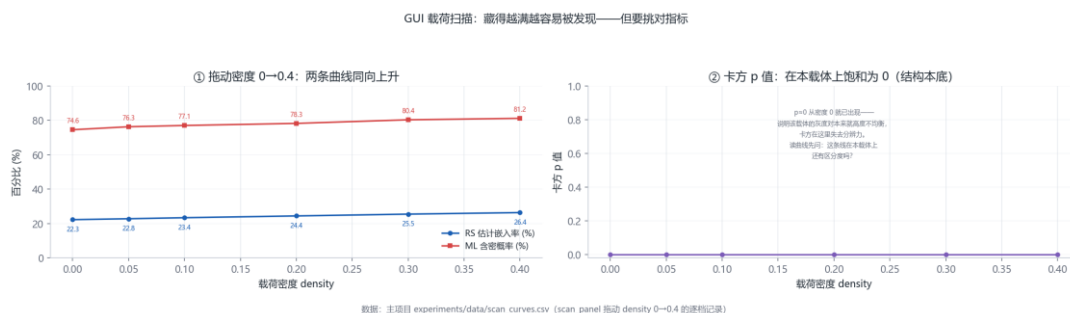


图 9-5 GUI 载荷扫描。这张图回答的是：嵌入密度变化时，各种可检测性指标怎么动？密度扫描下 RS 估计率、ML 概率与卡方 p 值的变化——统计可检测性随嵌入密度变化

避坑提醒 | 注意 GUI 文案里的严谨措辞：单张图没有“AUC”（AUC 需要一组正负样本），所以扫描面板用的是模型概率作为趋势示意。阅读软件输出时，要区分“单图概率”与“数据集指标”，这是容易被误用的一处。

9.5 测试与 CI：让重构不翻车

测试文件	验证内容	心智模型
test_core.py	汉明矩阵正确性、嵌入/解码往返、湿纸、口令错误	算法没坏
test_steg.py	盲分析能区分干净图与含密图	分析没坏
test_false_positive.py	大量干净图不误报（回归）	误报没失控

测试文件	验证内容	心智模型
test_gui.py	窗口能构建、载入与预览	界面没坏

.github/workflows/ci.yml 会在每次推送到 main 或提 PR 时自动跑核心测试、构建 wheel 与 sdist；推送 v* 标签时自动发 GitHub Release。这套“测试 + 打包 + 自动发布”是算法项目走向可复用工具的标准姿势。

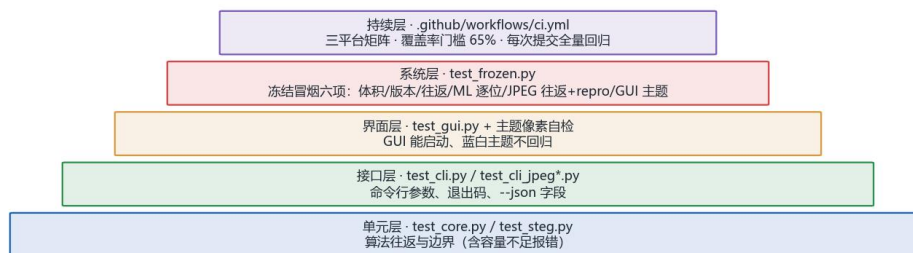
把项目的测试网按“离算法的距离”排成一张金字塔，每层都有真实文件对应：

层级	项目里的位置	守住什么
单元层	test_core.py / test_steg.py	算法函数的往返与边界（含容量不足报错）
接口层	test_cli.py / test_cli_jpeg*.py	命令行参数、退出码、--json 字段
界面层	test_gui.py + GUI 主题像素自检	GUI 能启动、蓝白主题不回归
系统层	test_frozen.py 冻结冒烟六项	体积 / 版本 / 往返 / ML 逐位 / JPEG 往返+repro / GUI 主题
持续层	GitHub Actions（覆盖率门槛 65%）	三平台矩阵 + 每次提交全量回归

六项冻结冒烟里最容易被低估的是第 4 项“ML 逐位一致”与第 5 项“JPEG 往返+repro”：它们防的不是代码写错，而是**打包时静默丢依赖**——DLL 或模型没进安装包，功能降级却无报错。让“降级”变成红灯而不是静默，是打包工程最痛的一课（v1.8/v1.9 两次踩坑后固化成冒烟项）。

测试金字塔：让「重构不翻车」变成机器替你守着的常态

越靠下越接近算法、越快报警；越靠上越接近交付、越能挡住「打包丢文件」。



文件与门槛取自主项目 README / CI 配置；六项冻结冒烟见 9.5 节

图 9-6 测试金字塔。这张图回答的是：重构不翻车的安全网由哪几层构成？单元→接口→界面→系统→持续五层

测试与各自的守护对象——重构不翻车的安全网

9.6 代码阅读路线图（按需选用）

入口：src/run_e2e.py（最短路径，串起所有模块）；

数据层：image_io.py → 数组怎么进出的；

嵌入层：ns5_core.py 从上到下读一遍（哈希、置换、汉明、湿纸、高层 API）；

分析层：steganalysis.py 的 analyze() 逆向追踪每个统计量；

ML 层：train_model.py 主流程 → featurize_v2.py / srm_filter.py（v1.4）→ ml_predict.py 双版本推理；

加速层：cppembed.py / fsfeatures.py 的 ctypes 与自校验；

界面层：gui.py 找按钮回调，顺藤摸瓜到算法函数。

想一想 | 为什么 ns5_core.py 的注释反复强调“改了置换算法会破坏可逆性”？请结合 v1.2.1 修复（DLL 缺失自动回退）想：如果 Python 与 C++ 排列不一致，会出现什么灾难性现象？

9.7 v1.4.0 更新：SRM、多源数据与模型工程（2026-09）

v1.4.0 新增/改动了若干工程模块，阅读新代码时先认目录：

新文件 / 模块	职责	学习要点
src/srm_filter.py	30 个标准 SRM 高通核, numpy/torch 双实现	核清单、归一化、增强图生成
src/featurize_v2.py	143 维 v2 特征组装与 CPU 实现	ALL_FEATURE_NAMES、featurize_v2()
gpu/featurize_v2_gpu.py	GPU 批量 143 维特征与一致性自检	extract_features_v2_gpu()
src/make_dataset.py 扩展	多进程 -j、SRM/feature-set/variants	按图并行、输出逐一一致
src/train_model.py 扩展	DS_FILES 多数据集、Stacking、LGB 调优	双版本模型保存与阈值
gpu/make_imageset.py 扩展	memmap 逐张落盘、--out/--id-offset	多源分工、避免 OOM

数据侧：项目建立了纯校园照片基准 data/campus_jpg（414 张，移除早期 DIP4E 教材图），并引入隐写分析事实标准 BOSSbase 1.01（1 万张 512² 灰度 PGM）做多源合并。2026-09-14 复核（可追到 experiments/data/gpu_pipeline_metrics.csv）：GPU 单跑校园 imageset AUC 0.7903（Youden 阈值 0.713、acc 0.802），BOSSbase 单跑 AUC 0.6438（0.798 / 0.655）；合并训练因仓库现有的 BOSSbase 图像集为 10000 样本（早期为 50000），改按 3410+10000=13410 样本重跑，AUC 0.6672。make_dataset.py 支持多进程（16 核约 5.6 倍加速），GPU 图像集用 memmap 逐张落盘避免大数组 OOM。

工程配套：许可证从 MIT 改为 Apache-2.0（新增 NOTICE 与第三方声明），README 与 pyproject 的版本/主页同步为 v1.4.0；C++ DLL 与 Python 的像素级/特征级一致性自检仍然保留。

动手做 | 跑一次 python gpu\featurize_v2_gpu.py 自检；然后按 README 用 data\campus_jpg 与 data\BOSSbase_1.01 各生成一源数据，对比“单源校园 / BOSSbase 单源 / 两源合并”三组模型的 AUC。这是 v1.4 最值得亲手复现的“数据域”实验。

本章小结。分层的意义是“每一层都可以单独替换与单独测试”；热路径靠测量找、靠对拍保真；GPU 只吃可并行的归约；测试金字塔从单元到冻结冒烟，把“重构不翻车”变成机器替你守着的常态。

动手做 |

章末练习（提示与答案见附录 J）

P9.1（上机）用 cProfile 跑一次大图（4096×4096）嵌入，找出耗时前三的函数，对照 9.2 节判断哪些值得下沉。

P9.2（上机）设计并执行一个对拍实验：固定种子分别调用 Python 与 C++ 的确定性置换，写出验证“逐位一致”的脚本思路。

P9.3（思考）为什么“DLL 缺失自动回退 Python”比“直接报错退出”更符合本项目定位？什么场景下反而应该报错退出？

P9.4（思考·选做）九层的测试金字塔里，哪一层在“改完算法参数”时最先报警？哪一层在“打包丢文件”时最先报警？

第 10 章 · 综合实践：从“读懂”到“做出来”（第 11–12 周）

本章导航 · 先看问题，再读正文

核心问题 读懂和做出来之间，隔着的是什么？

读完你能回答

- 用一个“小而**可证伪**”的标准给你的选题划定范围；
- 对账一份结论必须核对的四项：语料、协议、置信区间、是否可溯源；
- 设计一个**带对照**的消融实验（我改了什么、和谁比、在什么协议下比）；
- 按交付清单逐项打勾，并能脱稿回答答辩高频三问。

宁可把一个小问题答得干净利落，也不要在一个大问题上留一堆“可能、大概”。

开场问题：读懂和做出来之间，隔着的是什么？ 不是更多知识——12 周的知识已经够用——而是一个**可证伪的小问题**：它小到两周能做完，又大到做完后你能拿出一个别人可以复核的结论。CSAPP 的实验设计哲学是：宁可回答一个小问题答得干净利落，也不要在一个大问题上留下一堆“可能、大概”。本章三个方向（复现验证 / 功能扩展 / 教学再包装）都按这个标准组织，每个方向都配了最小可行里程碑。

小节	回答的问题	你的产出
10.1 方向 A	结论经得起重跑吗？	一份“复现+批判”报告
10.2 方向 B	能不能加一个真功能？	一段可讲解的代码与对比实验
10.3 方向 C	怎么讲给别人听？	一套演示页/动画/题库
10.5 答辩	评委最爱问什么？	脱稿回答任意三问
10.6 交付	最后一周交什么？	对照清单逐项打勾

动手做 | 本章目标：用两周时间完成一个可演示的改进实验。下面给出三个由易到难 的方向与验收清单，你也可以把它们组合起来。

10.1 方向 A：复现并批判性验证（最稳）

“批判性验证”的第一课是**对账**：把你要验证的结论与权威结果表（docs/RESULTS.md）逐字段核对——语料是什么、协议是按源图分组还是随机切分、数字有没有置信区间。摘一行主表做示范（BOSSbase、按源图 holdout）：

模型	特征	AUC	95% CI
LGB-11d	11 维	0.7128	[0.6913, 0.7360]
LGB-143d	143 维	0.8062	[0.7894, 0.8225]
LGB-53d	53 维	0.7172	[0.6943, 0.7406]
xuNet（裸像素 CNN）	原始像素	0.5007	[0.5004, 0.5012]
ynet（SRM+CNN）	30 SRM 固定核	0.9541	[0.9463, 0.9630]

拿到这张表，好的批判性问题会自己冒出来：143d 对 11d 的提升（+0.09）主要来自 SRM 特征还是数据规模？（8.8 节的 A/B 实验已回答一半——同数据下 143 维另有增益。）为什么裸 CNN 的置信区间那么窄却贴着 0.5？（它稳定地“不会”，稳定也是一种可复现。）把你冒出的问题写下来，挑一个能动手的。

选 README / docs/RESULTS.md 中的 1~2 个结论，自己重新跑并尝试“推翻”它：

复现码族图：理论 $\alpha(p)$ 是否与实测吻合？ p 取大后偏差从哪来？

复现检测实验：换你自己的照片训练，比较 v1 11 维基线（约 0.75–0.79）与 README 的 v1.4 双版本结果，观察照片风格对 AUC 的影响；

复现篡改实验：改 1 像素、改 1 行、改 1 块，解码失败率各是多少？

验收项	通过标准
实验日志	记录环境、命令、参数、原始输出
图表	至少 2 张自绘曲线/对比图，坐标与图例完整
结论	能区分“验证了论文”与“与论文不一致”并给原因
可复现	提供一键脚本或逐条命令，别人能重跑



图 10-1 可复核结论的解剖。这张图回答的是：一个可复核的数字必须携带哪些限定词？一个可复核数字必须携带的限定词，与复现验收四项——让结论经得起重跑

10.2 方向 B：功能扩展（最有成就感）

支持中文/UTF-8 消息：v1.8 起已在 `encode_string()` 内实现（长度头单位是字节数），可对照源码校对你的实现；

彩色通道扩展：目前仅用 R 通道，可研究 R/G/B 三通道如何分配容量与安全性；

新增特征：在 v1 11 维或 v2 143 维基础上加一个你认为能区分 clean/stego 的统计量，用 GroupKFold 评估 AUC 是否真的上升；

对比新算法：实现朴素 LSB 替换，与 nsF5 在相同载荷下比较 Gn 塌缩与检出率。

避坑提醒 | 每次扩展都要跑 `test_core.py` 与 `test_steg.py`，保证“嵌入→解码”往返仍然成立。修改编码方案时，解码端必须同步修改——这是初学者最常忘的。

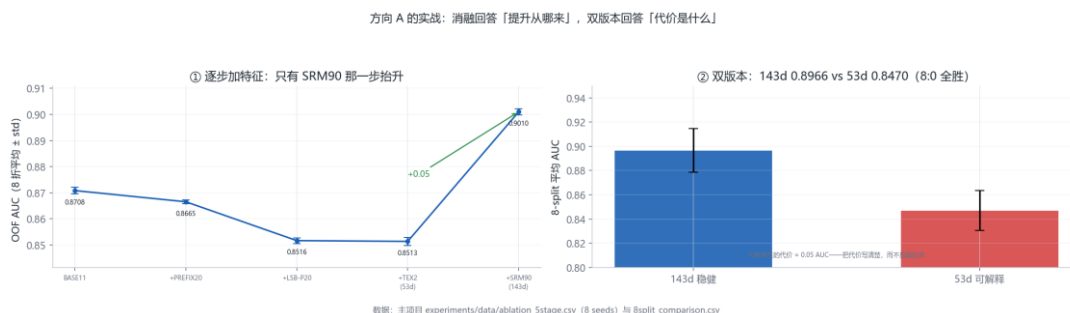


图 10-2 消融与双版本对比。这张图回答的是：新增一个特征，怎么证明它真的有用？143 维逐组增益消融与 143d/53d 双版本 8-split 对比——新增一个特征怎么证明它有用

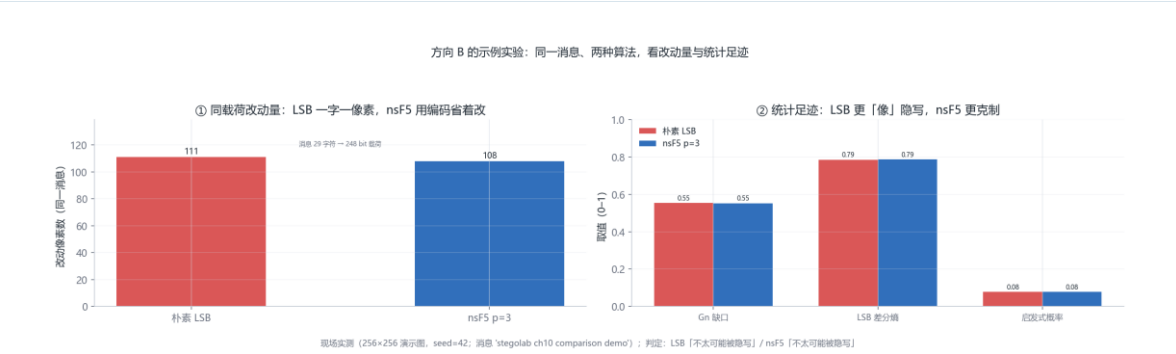


图 10-3 LSB vs nsF5 同载荷对比。这张图回答的是：同一个消息，两种算法各付什么代价？同载荷下朴素 LSB 与 nsF5 的改动量、统计足迹与启发式判定——同一个消息，两种代价

10.3 方向 C：教学演示再包装

- 把 GUI 里的矩阵编码演示扩展成一套可讲解的教材页/动画脚本，例如：
- 逐步动画：随机块 → 显示 H → 算 s → 给 m → 求 d → 高亮 → 校验；
- 题库模式：随机出题让你手算“该翻转哪一位”，自动判分；
- 对比模式：同一消息分别用 LSB、matrix p3、nsF5 p3 嵌入，并排显示改动数与 Gn。



图 10-4 三个方向。这张图回答的是：综合实践可以往哪三个方向做？方向 A 复现 / B 扩展 / C 教学的任务与交付——把读懂的东西讲给别人听

10.4 文档与代码对照阅读表

论文章节	对应代码	读它之前先掌握
第 2 章 技术基础	ns5_core.py / steganalysis.py	LSB、汉明码、湿纸、卡方/RS
第 3 章 需求分析	README 功能总览	隐写与隐写分析博弈观
第 4 章 总体设计	arch 图 + ns5_core.py	分层架构、哈希键控流程
第 4.5 节 加速	cppembed.py / fsfeatures.py	ctypes 与一致性自校验

论文章节	对应代码	读它之前先掌握
第 4.6 节 GPU	gpu/*.py	张量化、批量、吞吐边界
第 5 章 实验	run_e2e / make_dataset / train_model	GroupKFold、AUC、Youden

提示：本项目不再随仓库分发论文稿；对外数字一律以 docs/RESULTS.md（权威结果表）为准，它逐行标注了语料、协议与是否可溯源。下表按早期论文章节举例，阅读时请对照该表。

10.5 汇报/答辩常见问题与应答要点

答辩的加分项往往不在答案本身，而在**引用的口径**。同样一句“AUC 0.81”，说出“LGB-143d, BOSSbase 1.01, 按源图 holdout, 95% 置信区间 [0.79, 0.82], 另有 1514 张真实干净照片的 OOD 误报率实测”与只说一个数字，是两个档次。评委由此判断你有没有亲手跑过实验——因为只有跑过的人才会自然地地带出这些限定词。

高频问题	应答要点
LSB 隐写为什么能被发现？	相邻灰度对被拉平（卡方）+ LSB 结构塌缩（RS），先讲直觉再讲统计量
矩阵编码为什么“改得少”？	$n=2^p-1$ 个位置承载 p 位， H 列互异 $\rightarrow$ 任一伴随式差对应唯一位置
nsF5 相比 F5 好在哪里？	湿纸预标记湿点、只在干点解方程，无收缩无重嵌
湿纸编码怎么解方程？	GF(2) 线性方程 $H_{dry} \cdot y = d$ ，先试单列/双列，再高斯消元兜底
为什么不用裸 CNN？	小样本弱信号下学不到泛化特征；特征+简单模型更稳（AUC 实证）
AUC 0.756 说明什么？	排序能力尚可但弱密度难检；AUC 与“具体阈值下检出率”是两回事
哈希键控能防什么？	防位置可预测/复制；能感知普通篡改，但不是密钥化 MAC

高频问题	应答要点
C++/GPU 加速有没有风险？	有：必须做跨语言一致性校验，否则嵌入解码会不可逆

答辩与交付：把「跑过实验」写进你的限定词里	
① 答辩高频问题（答口径，不只答数字） Q: LSR 为什么能加速？ 答：数据量对数平方（N^2）→ LSR 线性复杂度（NS） Q: 数据量为什么是「数据量」？ 答：$m \times 2 \sim 1$ 个位置乘积 p 位，H 列互斥 Q: $m \times 2$ 在 T_5 附近？ 答：数据量对数平方，只在平方级方程，无数据量乘积 Q: 为什么不能做 CMF？ 答：小样本数据中不满足过拟合特征，AUC 实证 0.5007 Q: AUC 0.716 是什么？ 答：数据量对数平方可以证明数据量乘积，与数据量下比与数据量同乘 Q: C++/GPU 加速有没有风险？ 答：必须做跨语言一致性校验，否则不可逆	② 最后一周交付清单 ☐ 1. 改进代码 diff（每处能讲清） ☐ 2. 实验脚本 + 原始输出（不挑数） ☐ 3. 2~3 张图表（效率/ROC/密度/篡改任选） ☐ 4. 3~5 页报告（问题→方法→结果→局限） ☐ 5. 5 分钟演示（嵌入到分析一条龙） ☐ 6. 脱稿回答答辩者任意三问 每一道题在「别人可以复核你的结论」服务。

按手册 10.5-10.6 整理

图 10-5 答辩与交付清单。这张图回答的是：答辩时哪些点最容易失分？答辩高频问题应答要点与最后一周交付清单——加分项在引用的口径

10.6 交付清单（最后一周）

改进代码 diff（自己能讲清每处改动）；

实验脚本 + 原始输出（不加工、不挑数）；

2~3 张图表（效率/ROC/密度检出/篡改任选）；

3~5 页报告：问题→方法→结果→局限→下一步；

5 分钟演示：从嵌入到分析的一条龙运行；

能脱稿回答 10.5 表中的任意三个问题。

本章小结。选题标准是“小而可证伪”；方向 A 练对账与复现、方向 B 练功能与对比、方向 C 练表达与教学；交付清单的每一项都在为“别人可以复核你的结论”服务——这也是本手册从第 1 章贯穿到最后的一条主线。

动手做 |

章末练习（提示与答案见附录 J）

P10.1（写作）从 10.1 的对账表里挑一个你最想质疑的数字，写出：怀疑的理由、复现步骤、判定标准。一页以内。

P10.2（设计）为方向 B 的“新增一个特征”写出实验设计：假设、数据、协议（记得按源图分组）、判定标准（AUC 提升多少才算数）。

P10.3 (演练) 对照 10.5 的答辩表自问自答三题, 录音回放——自己听一遍胜过排练十遍。

第 11 章 · 边界、伦理与继续前进（穿插学习）

本章导航 · 先看问题，再读正文

核心问题 一个既能“藏”又能“查”的工具，发明者该对哪一半负责？

读完你能回答

- 用技术事实（改动率、检测能力、攻击模型）判断三个真实情景是否正当；
- 复述三条项目已知局限，以及各自的“当时可接受”理由与升级姿势；
- 说出 143d 与 53d 在**精度与稳健**上的取舍；
- 给出三条难度递增的继续前进之路。

技术本身不做善恶选择，做选择的是使用场景——本章给的是判断力，不是答案。

开场问题：一个既能“藏”又能“查”的工具，发明者该对哪一半负责？ 答案是：对两者的**边界**负责。隐写保护隐私与言论，也可能掩护恶意通信；隐写分析支撑取证与审计，也可能被用于窥探。技术本身不做善恶选择，做选择的是使用场景——所以本章的任务不是给你答案，而是给你**一套判断场景的练习**与一张诚实的局限清单。看懂边界，才算真正读懂这个项目。

11.1 法律与伦理边界

想一想 |

用第 1–9 章学到的知识判断下面三个情景，并说出理由：

- ① 给自己拍摄的照片嵌入不可见的水印与溯源哈希，发布到网上——正当吗？
- ② 帮朋友把考试答案用隐写嵌进图片，在考场上通过手机传给同学——正当吗？
- ③ 用项目的 analyze/ML 检测器审计自家服务器收到的上传图片，排查隐蔽外传——正当吗？

（参考判断：①是版权与溯源的正当用途；②规避考试诚信制度，且工具的“隐蔽性”被用于欺骗特定受众；③是防御性取证，且审计对象是自己有权管理的系统。争议永远存在于细节——比如③若扩大到审查员工的私人相册，答案就变了。）

隐写是双刃剑：它支撑版权水印、媒体溯源、隐蔽认证等正当应用，也可能被用于 恶意隐蔽通信。学习与研究的正确姿势是：

- 只在自有数据、公开数据集或经授权场景做实验；
- 把工具用于“检测与取证”而非“协助隐藏”；
- 发表结果时说明局限与误报风险，不夸大检测能力；
- 不把他人的隐私图像当作练习载体（照片数据要脱敏、授权）。

避坑提醒 | 这个项目的隐写分析部分（卡方/RS/ML）正是“盾”的用途：监管与取证需要它来发现可能的隐蔽信道。研究攻击面之前，先想清楚 你的实验数据与使用目的。



图 11-1 双刃剑与情景判定。这张图回答的是：同一件工具，什么场景算正当、什么场景算不当？隐藏与检测两侧的正当 / 不当用途及判定依据——技术不做善恶选择，场景才做

11.2 项目已知局限（看懂边界才算真正读懂）

局限清单里有三条值得追问“为什么当初这么设计”，因为它们展示了工程决策的真实形状：

局限	为什么（当时）可接受	升级的正确姿势
彩色图只用 R 通道	教学口径单一变量；三通道要重设计容量分配与键控语义	逐通道独立键控或联合伴随式（预留为扩展实验）
哈希键控而非 MAC	教学项目攻击者模型弱；MAC 需要密钥管理与协商	HMAC-SHA256，密钥经口令派生（KDF）
像素域不抗 JPEG	像素域是教学主战场，压缩域已由 yccstego 承担	两域统一 API 已就位（--jpeg / GUI 域切换）

看懂这张表的读法：每一行都不是“做错了”，而是“在当时的约束下正确，且升级路径明确”。真实项目与玩具项目的区别，就在于有没有把“没做的事”写得和“做了的事”一样清楚。

载荷支持 UTF-8 文本（中文可直接嵌入）；更实际的限制是仅支持文本、不支持文件；

彩色图只使用 R 通道，容量利用率低；

内容是哈希键控而非密钥化 MAC，无法防“重嵌并更新头部”的强攻击者；

v1 数据集的独立源图仍有限（414 张校园 + BOSSbase 后扩到 1 万张）；弱密度与跨源仍是检出难点，任何新模型都要先做 OOD 测试；

启发式分析输出的是“隐写倾向概率”，不是严格统计意义上的真实概率；

像素域 nsF5 是有损格式的天敌——JPEG 域需要 yccstego 那样的 DCT 系数实现（姊妹项目，独立仓库与 PyPI 包：<https://github.com/YukinoShita-lin/yccstego>）。

v2/143 维模型对训练分布外的真实 JPEG 干净图曾“过激”，修复依赖把 JPEG 干净样本加入训练；任何模型都要先做 OOD 测试再部署——本项目现口径是 1514 张真实干净照片上 143d 误报 9.58%、53d 28.86%；

SRM 只在同源数据上提升检测，跨异构源反而下降，预处理收益有严格适用范围；

双版本模型各有取舍：143d 更准（8-split 平均 AUC 0.8980、弱档检出 50.0%、真实干净照片 OOD 误报 9.58%）；53d 去掉 SRM 后每一维都能解释，代价是 AUC 约低 0.05（0.8461 / 41.4% / 28.86%）。GUI 切换仍在规划，当前用 API 指定 model_path；

项目许可证已改为 Apache-2.0 并含 NOTICE，分发/引用时需保留第三方声明。



图 11-2 已知局限与升级路径。这张图回答的是：项目没做的事，该怎么写才诚实？三条局限的「当时可接受」理由与升级姿势——把没做的事写得和做了的事一样清楚

真实干净照片上的误报率：模型没见过的新分布，才是部署前的最后一关

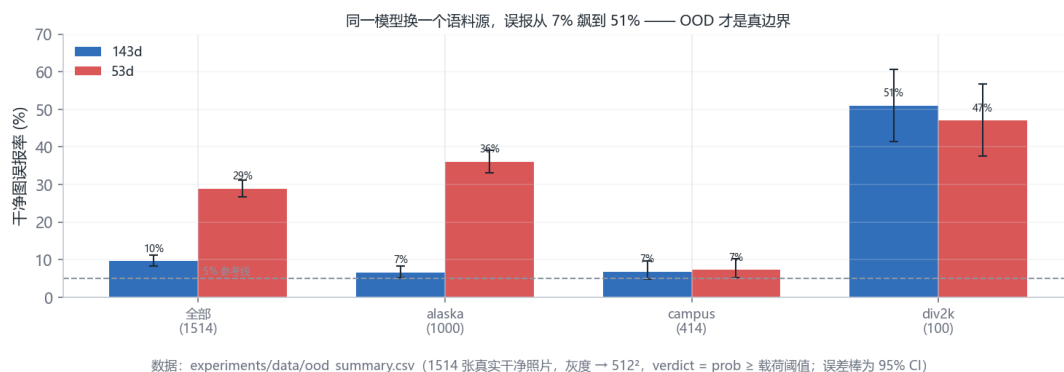


图 11-3 OOD 误报率分源。这张图回答的是：换一个语料源，误报率会飙升多少？143d / 53d 在全部与分源真实干净照片上的误报率与 95% CI——部署前的最后一关

11.3 现代信息隐藏与隐写分析地图

方向	代表思想/方法	与本科生的距离
内容自适应嵌入	HUGO / WOW / UNIWARD：在纹理复杂处嵌入	需要较深最优化背景
深度学习隐写	编码器-解码器端到端训练隐藏模型	可做入门复现
深度学习隐写分析	SRNet / 大规模 CNN 检测	需要大算力与大数据
JPEG 域隐写	yccstego: Y 通道量化 DCT 系数 nsF5 (姊妹项目, 独立仓库: https://github.com/Yukinoshita-lin/yccstego) (姊妹项目, 独立仓库: https://github.com/Yukinoshita-lin/yccstego)	本项目即可扩展
可逆/鲁棒水印	兼顾不可见性、鲁棒性与容量	工程性强, 适合就业方向

如果你的兴趣被点燃，推荐的下一步：把本手册第 5 章“湿纸”读进原始论文 (Fridrich et al.)，再用姊妹项目 yccstego (<https://github.com/Yukinoshita-lin/yccstego>) 对照学习 JPEG 量化域，最后挑一个现代算法做文献综述——附录 E 给了具体入口。



图 11-4 现代信息隐藏地图。这张图回答的是：继续深入该往哪个方向走？五个现代方向、代表方法与与本科生的距离——继续前进该往哪走

双版本取舍：精度、可解释性与误报，三个指标不能同时最优

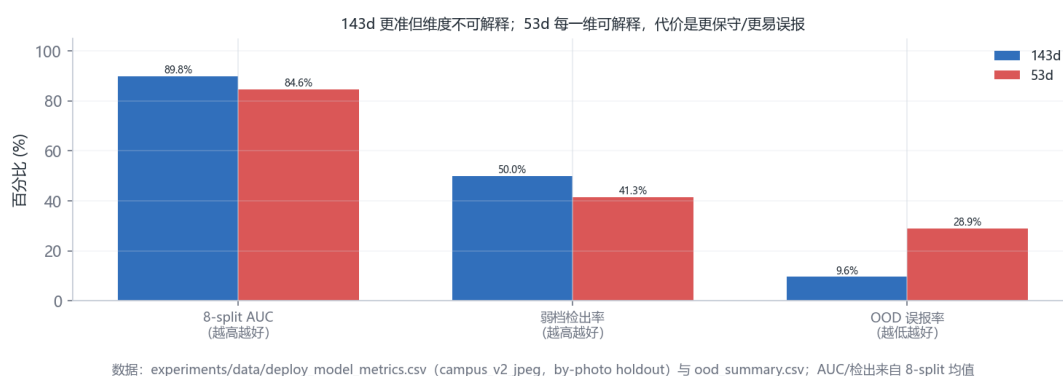


图 11-5 双版本模型取舍。这张图回答的是：精度、可解释性与误报，能同时最优吗？143d / 53d 在 AUC、弱档检出率与 OOD 误报率上的取舍——精度与稳健不可兼得时怎么选

11.4 结语

刚开始时你也许以为隐写就是“把字藏进图片”，现在你应该能讲出：它为什么依赖 载体冗余、为什么直接改 LSB 会留下统计指纹、汉明码如何用最少的改动传递信息、湿纸编码如何让嵌入“无收缩”，以及机器学习如何诚实地说出“我有多确定”。这些知识串起来的，不只是 F:\Steganography 这一个项目，而是信息安全、概率统计与机器学习交汇处的一种思维方式。

动手做 | 最后做一个收尾练习：不看任何资料，用 500 字向一位同学介绍“nsF5 为什么比朴素 LSB 更好”。写完后打开 README 对照，查漏补缺——能写清，才算真正学完。



图 11-6 结语与三条路。这张图回答的是：全书的主线是什么？全书主线回顾与难度递增的三条继续前进之路——从“图像是数字”到“工具有边界”

结语。从第 1 章“图像是数字”到第 11 章“工具有边界”，这本手册其实只教了一件事：**任何一个结论都必须经得起“往下剥一层”与“亲手算一遍”**。继续前进的路有三条，难度递增：其一，把第 4、5 章的算法读回原始论文（Westfeld 2001、Fridrich 2005），体会“教科书简化”与“论文原貌”的差距；其二，读 yccstego 的源码——它 2026-10-04 修复的四个编解码 bug（issue #1）是一份活生的调试教材；其三，给自己设计一个 OOD 实验：拿模型从没见过的新语料测一次误报率，体会“训练分布”四个字的重量。

动手做 |

章末练习（提示与答案见附录 J）

P11.1（写作）为 11.1 节的三个情景各写一段 100 字的判断书，要求引用具体的技术事实（改动率、检测能力、攻击模型）而非情绪。

P11.2（思考）从 11.2 的局限表里挑一条，设计一个两周的可执行升级方案（含验收标准）。

P11.3（上机·选做）把项目 README、CHANGELOG 与 docs/RESULTS.md 通读一遍，列出三处“文档与代码互相印证”的地方——这是读懂一个陌生开源项目最快的方法。

第 12 章 · 批判性思维（复盘指南）

本章导航 · 先看问题，再读正文

核心问题 做完了项目，怎么保证自己没有被骗到？

读完你能回答

- 复述源图泄漏事故的全过程（事故 → 定位 → 修复 → 结论被推翻），并说出「以实验为准」到底指什么；
- 用五条质疑清单审一个结论，每条都能落回「质疑 → 验证 → 结论」的闭环；
- 把一个「空想」改写成合格方案：**可执行、可证伪、有对照**；
- 把「结论」与「证据」分开看，知道一个反例就足以推翻结论；
- 在报告里写出「已知局限」与「未解决的问题」，并复述实验日志的五个要素。

本章不教新算法。它教的是：怎样让已经做出来的东西，经得起别人重跑一遍。

开场问题：一个结论要怎样才敢说出口？ 前十一章你学会了“怎么把东西藏进去”和“怎么把它查出来”，第 10 章你还亲手做了一个综合实践。但做完不等于做对——**一个实验最容易骗到的人，是做实验的自己。** 本章是一份复盘指南，八个要点，全部拿本项目真实发生过事故当教材：一次源图泄漏让 AUC 虚高了 0.14，一条“深度学习当然更强”的叙事被一个 0.5007 打脸，一次“我觉得有用”被改写成有对照、有推翻条件的消融实验。读完本章，你应当能把“我读懂了”升级成“我敢让别人重跑一遍”。

小节	回答的问题	你的产出
12.1 以实验为准	实验是怎么骗人的？	能讲清源图泄漏事故的全过程
12.2 质疑设计	拿到一个结论该问什么？	会用五条质疑清单
12.3 给出方案	怎么把质疑变成方案？	能写出可执行/可证伪/有对照的方案
12.4 结论与证据	结论和证据怎么分开？	知道一个反例就能推翻结论
12.5 承认不知道	“我不知道”为什么是能力？	能写“已知局限”与“未解决问题”
12.6 警惕叙事	哪些“当然”是陷阱？	能举出项目里的三个反例

小节	回答的问题	你的产出
12.7 复现第一课	复现不出来先怀疑谁？	记住环境→理解→书的排查顺序
12.8 实验日志	结论凭什么可追溯？	能写出日志五要素

12.1 一切以实验结果为准：实验是怎么骗人的

这一节的核心不是“实验很重要”，而是“**实验是怎么骗人的**”。先看一个真实发生的数字冲突：同一个 143 维模型，在 BOSSbase 语料上是 **AUC 0.8062**，在自建的校园语料上却是 **0.8939**。两个数字差得很远，但这不是矛盾——**语料属性不同**。BOSSbase 是领域标准基准、难度高；校园语料是自建语料、含真实 JPEG 干净图、比 BOSSbase 容易。把两个语料的数字放在一起比，比出来的不是“模型变强了”，而是“你换了考卷”。

更严重的一类骗局，发生在**实验设计本身**。项目里真实发生过一次事故：校园语料的 `clean\_jpeg` 行错用了独立的 photo_id——414 行副本各占一个编号，于是“按源图划分”的纪律被悄悄破坏：同一张源图的干净版与含密版，被分到了训练集和测试集的两侧。模型只要认出“这是哪张源图”，就能顺带猜出标签。结果是 **143d 的 AUC 从 0.7555 被抬到 0.8946**、弱档检出率从 **44.2% 虚高到 85.3%**。这不是模型变强了，是**实验设计出错了**。

环节	发生了什么	关键动作
事故	clean_jpeg 414 行副本各占一个 photo_id	同源副本跨了训练/测试两侧
定位	「按源图划分」不变量校验失败	按源图重采样、重算置信区间
修复	生成器与训练器都在训练前拒绝破坏不变量的语料	把不变量写成可执行校验
推翻	143d AUC 0.8946 → 0.7555；弱档检出 85.3% → 44.2%	旧结论作废，重跑后重新写文档

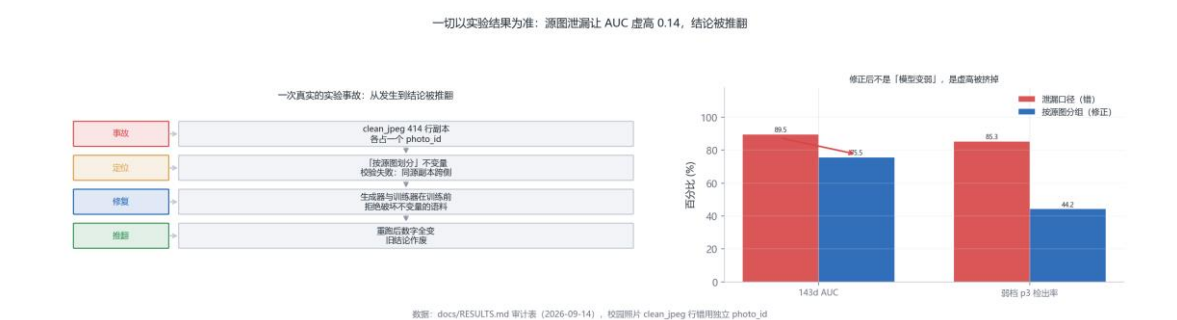


图 12-1 源图泄漏事故与结论被推翻。这张图回答的是：一次实验设计错误是如何让 AUC 虚高 0.14、并最终推翻旧结论的。左：事故→定位→修复→推翻四个环节；右：旧口径与修正后的并排对照

动手做 |

打开 StegoLab 的 `tools/run\_ch12.py` 跑一遍。它用一个**合成语料**现场演示泄漏机制：标签只由“源图身份”决定，特征里带着源图指纹。随机按图划分时 AUC 约 **0.74**，按源图分组后回落到约 **0.52**——虚高约 **+0.22**。你会在自己的终端里，亲眼看到“划分方式”如何凭空造出 0.2 的分数。这不是模型变强，是设计出错。

所以，**结论必须带上限定词**：语料、协议、置信区间、是否可溯源。没有这些限定词的数字，不能作为结论引用。项目里把这条纪律做成了一个可执行的检查：权威结果表 `docs/RESULTS.md` 由脚本 `experiments/build\_results\_table.py` 生成，每个数字都必须能追到「脚本 + CSV」，追不到的一律进第 7 节并标注原因。

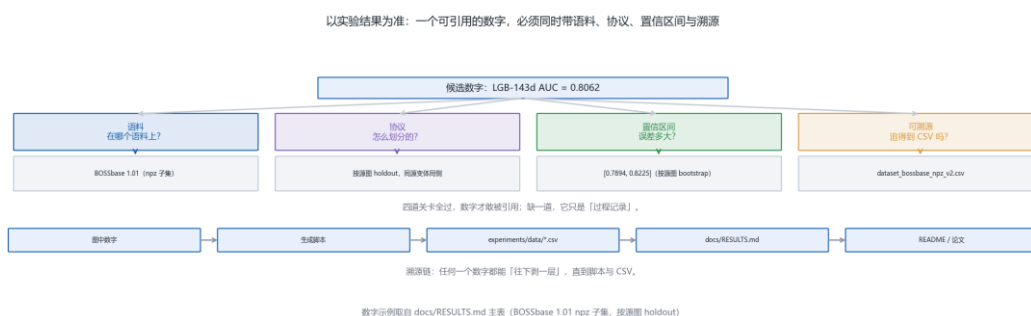


图 12-2 可引用数字的四道关卡。这张图回答的是：什么样的数字才敢被引用——语料、协议、置信区间、可溯源缺一不可。上半：四道关卡与各自的追问；下半：从图中数字一直追到脚本与 CSV 的溯源链

避坑提醒 |

最危险的不是“没有数字”，而是“**数字很漂亮但没有限定词**”。“AUC 0.89”这句话单独看毫无意义——它没说在哪个语料、什么划分、误差多大、能不能溯源。答辩时若被追问一句“这个 0.89 是在什么协议下测的”，答不上来，前面所有工作都会被扣分。

12.2 敢于质疑设计，但要建立在思考之上

质疑不是抬杠。抬杠停在“我觉得不对”，质疑必须落回“**我实际去验证了，结果是……**”。项目里现成的质疑对象很多，下面这张清单把每一条都配上了真实的验证动作与结果——你看到的不是观点，而是“质疑 → 验证 → 结论”的完整闭环。



图 12-3 质疑清单与验证闭环。这张图回答的是：拿到一个结论时该问哪五个问题，以及每个问题的验证动作与实测结果。五条清单：数据来源、对照组公平性、相关还是因果、有没有更简单的解释、指标是否回答了想问的问题

质疑	验证动作	项目里的实测结果
这个结论是在什么数据上得到的？换一批还成立吗？	换语料复跑同一模型	BOSSbase 0.8062 vs 校园 0.8939 —— 语料不同，数字不可比
这个实验的对照组设置公平吗？	查是否共用同一份按源图划分	CNN 与 LGB 基线共用划分，才可比
有没有把“相关”说成“因果”？	看有没有做消融	SRM 消融 $\Delta = +0.0497$ —— 有因果证据，不是相关
有没有更简单的解释被忽略了？	先查训练是否收敛	裸 CNN 训练集 AUC 0.5005（未收敛）—— 不是“CNN 弱”
这个指标能回答我想问的问题吗？	看弱档检出率，而不是只看 AUC	AUC 高 ≠ 弱密度可用（弱档检出会明显下降）

想一想 |

“裸 CNN 在 BOSSbase 上只有 0.5007”常被引用为“深度学习不如手工特征”的证据。但请你先问那个更简单的问题：**这个模型训练收敛了吗？**查一下就知道——它的训练集 AUC 只有 0.5005，与随机猜测无异。一个没收敛的模型输掉比赛，说明的是“这次训练失败了”，不是“这条路走不通”。把“相关”说成“因果”，往往就差这一问。

12.3 质疑之后，要敢于并善于给出自己的方案

只会质疑的人停在原地；会做研究的人要往前走一步——**给出自己的方案**。这里要分清“空想”和“方案”：

空想是：“我觉得这个算法不好。”方案是：“如果把 X 换成 Y，在 Z 数据上应该会有 W 的变化，因为……，我设计了如下实验来验证。”差别不在于语气，而在于**可执行、可证伪、有对照**这三条硬指标。

“善于”的含义	具体要求	反例（不合格）
可执行	能在有限时间、有限算力内做完	“重新训练一个十亿参数的模型”
可证伪	能提前写出“什么结果会推翻我的假设”	“如果效果不好就再调调”
有对照	说清改了什么、和谁比、在什么协议下比	“我改了一下，感觉更好了”

项目里有一个从“质疑”走到“方案”的完整范例：起点是一句质疑——“SRM 特征到底有没有用？”终点不是一句感觉，而是一个**消融实验**：把 143 维里的 SRM 组去掉（退回 53 维），在校园语料上按源图 GroupKFold(5) 跑 8 个 seed。结果 **OOF AUC 从 0.9010 掉到 0.8513** ($\Delta = -0.0497$)，而且 8/8 个 seed 全胜。这比说“我觉得 SRM 有用”强一百倍：它有对照、有协议、有推翻条件。

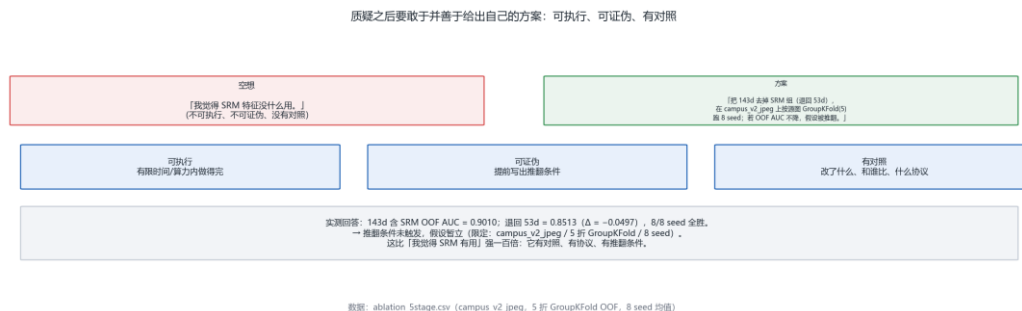


图 12-4 从空想到方案。这张图回答的是：怎么把一句“我觉得”改写成可执行、可证伪、有对照的方案，以及 SRM 消融这个真实范例。上半：空想与方案并排对照 + 三条硬指标；下半：消融实验的实测回答与推翻条件

动手做 |

给自己的选题写一个“方案卡”，必须填满四栏：**假设**（一句话）、**对照**（和谁比、什么协议）、**推翻条件**（什么结果会让你认输）、**成本**（要跑多久、要不要 GPU）。填不满四栏的，都还是空想。项目里 SRM 消融这张卡是这么填的：假设“SRM 组提升 OOF AUC”；对照“143d vs 去掉 SRM 的 53d，同一份 5 折 GroupKFold”；推翻条件“若去掉 SRM 后 AUC 不降 ($\Delta \leq 0$) 则假设被推翻”；成本“8 seed × 5 折，分钟级”。

12.4 区分“结论”与“证据”

读论文和写报告时，最容易混为一谈的两样东西，是**结论**和**支撑它的证据**。结论是一句话：“SRM 特征有效。”证据是另一句话：“在固定协议下，加入 SRM 后 OOF AUC +0.0497，且 8/8 seed 全胜。”一个结论可以有很多条证据，也可以被**一个反例推翻**。这一节的任务，是教你把这两样东西永远分开看。



图 12-5 结论与证据分离。这张图回答的是：一条结论下面挂着哪些证据，以及一个反例为什么足以推翻它。上半：结论“SRM 特征有效”与四条证据；下半：反例如何迫使结论加上限定语

举例：如果将来在 BOSSbase 上发现“SRM 同源有效、**跨源反而有害**”，那么“SRM 特征有效”这句话就必须加上限定语——“在校园语料的按源图协议下有效”。这不是打脸，这是科学：**结论的适用范围，由证据的边界决定**。所以读任何一篇论文，先找证据（在什么数据、什么协议、多少 seed），再决定信不信结论。

新手提示 |

一个可操作的训练：读到任何结论句，就地在旁边列出“它至少需要哪几条证据才成立”。列不出来的，说明你还没读懂它；列出来发现证据不够的，说明作者说得太满。附录 K 的每一张表都在做这件事——**结论放正文，证据放附录，两者一一对应**。

12.5 承认“我不知道”是一种能力

项目里最难能可贵的部分之一，是那份**更正记录**。它展示了：承认自己错过、承认数字不可信、承认有些问题还没解决，并不丢人，反而让整个项目更可信。第 12.1 节那场源图泄漏事故，最后没有从文档里“删掉重写”，而是原样保留在审计记录里——因为**一个被诚实记录的失败，比十个含糊的成功更有价值**。

“不知道”不是终点，是下一个实验的起点。具体到写作，就是两条硬要求：在实验报告里，**必须有一节写“已知局限”与“未解决的问题”**；在答辩里，**承认局限比假装完美更能赢得信任**。项目第 11 章列出的三条已知局限（彩色图只用 R 通道、哈希键控而非 MAC、像素域不抗 JPEG），每一条都写清了“当时为什么可接受”与“升级的正确姿势”——这就是标准的写法。

避坑提醒 |

“已知局限”不是自我批评的清单，而是**边界的说明书**。写清“我在什么条件下不保证成立”，恰恰是在为“我在什么条件下保证成立”背书。反过来，一个声称“哪里都能用”的工具，恰恰是最可疑的——它要么没测过边界，要么没敢写。

12.6 警惕“看起来合理的叙事”

很多错误的结论，不是因为数据造假，而是因为**叙事太顺了**。“深度学习当然比手工特征强”、“更多特征当然更好”、“训练集上更好，测试集当然也更好”——这三句话听起来都理所当

然，但在项目里，它们各自都有一个现成的反例。批判性思维的一部分，就是对“听起来理所当然”保持警觉。



图 12-6 看起来合理的叙事与反例。这张图回答的是：三条“理所当然”的叙事分别被项目里的哪个反例打破。上：叙事；中：项目反例；右：教训——强不强取决于预处理与收敛、加特征要先证明有用、划分错了分数就是假的

听起来理所当然	项目里的反例	教训
“深度学习当然比手工特征强”	裸 CNN 0.5007（训练 AUC 0.5005，未收敛）；固定 SRM + 小 CNN 0.9541；手工 LGB-143d 0.8062	强不强，取决于预处理与是否收敛
“更多特征当然更好”	11d 0.8708 → 31d 0.8665 → 51d 0.8516 → 53d 0.8513 → 143d 0.9010（前四步不升反降）	加特征先要证明它有用（消融）
“训练集更好，测试集当然也更好”	源图泄漏：训练/测试共享源图指纹，AUC 0.8946 是虚高，修正后 0.7555	划分错了，分数就是假的

想一想 |

第二行那张曲线值得盯着看一会儿：从 11 维一路加到 53 维，AUC 不是上升而是**下降**的（0.8708 → 0.8513），直到加上 SRM 那一组才跳到 0.9010。如果当初只凭“更多特征更好”的直觉行事，你会在前四步里越走越差却浑然不觉。**直觉负责提出假设，实验负责裁决**——把这两件事分开，就是本章的全部要义。

12.7 把“复现”当作批判性思维的第一课

复现不只是“跑通代码”，而是**用自己的数据、自己的协议重新验证**。项目配套的 StegoLab 就是为这件事准备的：手册里每一个关键数字，都有一条对应的复现路径——配图有 `figure\_manifest.json` 登记来源与 SHA-256，实验有 `figures/chXX\_results.csv` 落盘。

如果复现不出来，请严格按这个顺序排查：**先怀疑环境、再怀疑理解、最后才是怀疑书**。环境（版本、字体、依赖、随机种子）→ 理解（口径、协议、单位是否一致）→ 书（手册是否写错）。这个顺序不是客气，是概率：绝大多数“复现不出来”，前两个原因就占完了。一个能诚实说“我复现不出来，原因可能是……”的人，比一个只说“我读懂了”的人可靠得多。

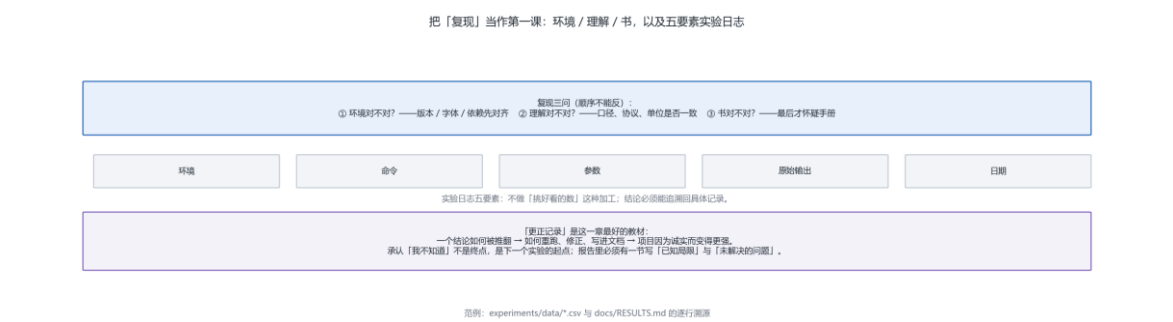


图 12-7 复现与实验日志。这张图回答的是：复现不出来时按什么顺序怀疑，以及一份合格的实验日志要记哪五个要素。上半：复现三问（环境→理解→书）；中部：日志五要素；下半：更正记录为什么是最好的教材

12.8 实验日志是批判性思维的载体

批判性思维不是一种态度，而是一种**可以被检查的记录习惯**。每次实验，记下五样东西：**环境、命令、参数、原始输出、日期**。不做“挑好看的数”这种加工；结论要能追溯回具体的实验记录。项目里这一点已经做得很扎实：`experiments/data/` 下的 CSV 是原始落盘，`docs/RESULTS.md` 是逐行溯源，附录 K 又把手册引用的数字与它们一一对上——**任何一句话都能往下剥一层，直到脚本与 csv**。

```
# 一份合格实验记录的骨架（项目真实做法）

环境： python 3.10 / numpy 1.26 / matplotlib 3.8 / 字体 Microsoft YaHei
命令： python tools/run_ch12.py --out figures/ch12_results.csv
参数： seed=20261005, n_src=120, k=5, d=32, ridge_lam=0.1
输出： figures/ch12_results.csv (逐行 case/detail/value/ok/note)
日期： 2026-10-05

# 结论只允许引用「输出」里的数字，且必须带上语料/协议/CI/溯源四项限定词。
```

回到项目看代码 |

本章的可运行实验在 `F:\StegoLab\chapters\ch12\_critical.py` (配图 + 体检逻辑), 入口 `tools/run\_ch12.py`。它会现场做五件事: ① 限定词体检 (逐行检查权威结果表的四限定词是否齐备); ② 泄漏复算 (合成语料上随机划分 vs 按源图分组的 AUC 差); ③ 消融复核 (0.9010 vs 0.8513、8/8 全胜); ④ 叙事反例核对 (裸 CNN 0.5007 与固定 SRM 0.9541); ⑤ 可证伪性检查。结果落在 `figures/ch12\_results.csv`, 本章所有数字都可与它逐项对账。

12.9 章末练习：给一个结论做体检

本章的练习不是“算一个数”，而是“**审一句话**”。下面三条结论，请各写一份不超过 200 字的体检报告，指出它缺了什么限定词、可能被怎样误读、以及你会怎么补做实验来支撑或推翻它。

结论（待体检）	提示：先问自己什么
“我们的模型 AUC 达到 0.89，效果很好。”	语料？协议？置信区间？可溯源？——0.8939 是校园语料、按源图 holdout 的结果，换个语料还成立吗？
“加入 SRM 特征后效果更好。”	对照是谁？协议是什么？增益多大？几个 seed？有没有推翻条件？
“检测器在测试集上准确率 95%，可以部署了。”	测试集是怎么划分的？有没有源图泄漏？真实干净照片上的误报率是多少（OOD）？

动手做 |

把上面三条结论分别改写成“带限定词的版本”，再和附录 K 的真实数字对一遍。改写完成的标志是：**任何一句话被追问“在什么数据、什么协议下”时，你都能立刻答出来，并指出它落在哪个 csv 的哪一行。**做到这一点，你就把第 12 章学完了。

本章小结。批判性思维不是让你怀疑一切，而是让你**知道每一句话的适用范围**。以实验为准，因为实验会骗人；敢于质疑，但要落回验证；质疑之后要给出方案，方案要可执行、可证伪、有对照；把结论和证据分开看；承认“我不知道”；警惕“听起来理所当然”；把复现当作第一

课；用实验日志把一切钉在纸上。—— 做完了项目再读这一章，你会发现它不是额外负担，而是让你前面十一周的成果**真正站得住**的那块地基。

附录 A · 术语中英对照

按出现顺序整理，方便随时查阅。掌握粗体项即可覆盖本手册绝大部分内容。

中文	英文	一句话解释
隐写术	Steganography	把秘密信息藏进正常载体，隐藏“通信行为本身”
隐写分析	Steganalysis	通过统计/学习判断载体是否藏有秘密
载体 / 含密图	cover / stego	嵌入前的原始图像 / 嵌入后的图像
最低有效位	LSB	像素二进制的最低位，翻转后视觉几乎不变
位平面	bit plane	按二进制位拆出的二值图层，共 8 层
冗余	redundancy	载体中可被修改而不易察觉的部分
嵌入容量	capacity / payload	一张图最多能承载的秘密比特数
相对载荷	relative payload	嵌入比特数与可嵌位置数之比
嵌入效率	embedding efficiency	平均每次修改所携带的比特数 α
矩阵嵌入	matrix embedding	用分组编码在更少改动中嵌入更多位
二元汉明码	binary Hamming code	能纠正/定位 1 位错误的线性码 $[n,k,3]$
校验矩阵	parity-check matrix H	$p \times n$ 矩阵，列互异，用于算伴随式
伴随式	syndrome	$s = H \cdot x \pmod{2}$ ，块的“现状摘要”
GF(2)	Galois field of 2	只含 0/1、按异或运算的有限域
收缩	shrinkage	减幅时系数变 0 导致块作废的现象

中文	英文	一句话解释
湿纸编码	wet paper coding	湿点不动、只在干点解方程完成嵌入
干点 / 湿点	dry / wet positions	可修改的位置 / 碰不得的位置
F5 / nsF5	F5 / nsF5 algorithm	JPEG 矩阵嵌入；nsF5 用湿纸消除收缩
减幅	coefficient shrinking	让系数绝对值减小 1 的修改方式
SHA-256	SHA-256	输出 256 位摘要的密码学哈希函数
键控	keying	由口令/内容密钥决定隐藏位置等行为
确定性置换	deterministic permutation	同种子必得同排列的洗牌算法
splitmix64	splitmix64	项目使用的 64 位伪随机数生成器
Fisher–Yates	Fisher–Yates shuffle	等概率洗牌的标准算法
自同步	self-synchronization	解码端无需外部参数即能找到正文
篡改感知	tamper perception	图像被改动后能够被察觉
盲隐写分析	blind steganalysis	无原始载体时的统计检测
卡方检验	chi-square test	检验相邻灰度对是否被拉平
p 值	p-value	观测与假设吻合的概率（此处高 p 反而可疑）
RS 分析	RS analysis	通过正负掩码扰动统计规则/奇异组
常规/奇异组	regular / singular	扰动后判别值变大 / 变小的像素组
差分熵	difference entropy	相邻像素差分布的香农熵，衡量纹理随机度

中文	英文	一句话解释
香农熵	Shannon entropy	信息量/不确定度的度量，单位 bit
监督学习	supervised learning	用带标签样本训练模型
特征向量	feature vector	描述样本的数值列表（本项目 11 维）
标签	label	样本的正确答案（0=干净，1=含密）
二分类	binary classification	输出属于两类之一的预测
逻辑回归	logistic regression	线性加权 + sigmoid 的可解释分类器
sigmoid	sigmoid	把任意实数压到 0 ~ 1 的 S 形函数
交叉熵损失	cross-entropy loss	分类常用的损失函数
梯度下降	gradient descent	沿损失下降方向迭代更新参数
过拟合	overfitting	背下训练样本而失去泛化能力
交叉验证	cross-validation	轮流用不同折验证模型的流程
数据泄漏	data leakage	验证信息混入训练导致指标虚高
GroupKFold	GroupKFold	按组切分的交叉验证（同照片样本同组）
混淆矩阵	confusion matrix	真实 vs 预测的四格计数表
精确率/召回率	precision / recall	判对比例 / 抓全比例
ROC / AUC	ROC curve / AUC	全阈值下的检出-误报曲线及其面积
Youden 准则	Youden's J	在 tpr-fpr 最大处选阈值
误报 / 漏报	false positive / negative	把干净判成含密 / 把含密判成干净
ctypes	ctypes	Python 调用 C/C++ 动态库的接口

中文	英文	一句话解释
DLL	dynamic-link library	Windows 动态链接库
CUDA / 批处理	CUDA / batching	GPU 并行计算 / 多样本同时处理
一致性校验	consistency check	C++/GPU 与 Python 结果逐位比对
ASCII / UTF-8	ASCII / UTF-8	英文字符编码 / 通用字符编码 (含中文)
空间富模型	SRM / Spatial Rich Model	一组高通滤波残差核，用于突出嵌入噪声
残差图	residual map	高通滤波后保留的“噪声残差”
LightGBM	LightGBM / LGB	梯度提升树实现 (v1.4 双版本模型使用的分类器)
特征组	feature group	同一来源的一批特征 (BASE/SRM/PREFIX/LSB-PREFIX 等)
分布外	out-of-distribution (OOD)	与训练分布不同的输入 (如真实 JPEG 干净图)
BOSSbase	BOSSbase 1.01	隐写分析标准基准库: 1 万张 512 ² 灰度 PGM
PGM	PGM	无损灰度图像格式 (BOSSbase 载体格式)
Stacking	stacking / meta-learner	用次级模型组合多个基模型预测的集成方法
内存映射	memmap	大数组分批落盘、按需读写，避免一次性 OOM
校准集	calibration set	单独用于选择阈值的样本子集
网格调优	grid tuning	在超参数网格上搜索并比较模型
双版本模型	dual-version model	143d 稳健版 + 53d 可解释版并存部署策略

附录 B · 常用命令速查

以下命令均在项目根目录 F:\Steganography 下执行。

目的	命令
安装核心依赖	<code>pip install numpy pillow</code>
安装全部依赖 (ML/绘图)	<code>pip install -e .</code> 或 <code>pip install numpy pillow matplotlib scikit-learn joblib pandas</code>
核心算法自测	<code>python src\test_core.py</code>
隐写分析自测	<code>python src\test_steg.py</code>
误报回归测试	<code>python src\test_false_positive.py</code>
GUI 冒烟测试	<code>python src\test_gui.py</code>
端到端验证	<code>python src\run_e2e.py</code>
启动图形界面	<code>python src\gui.py</code>
生成码族/效率图	<code>python -c "import sys; sys.path.insert(0,'src'); from efficiency import plot_code_family_and_efficiency; print(plot_code_family_and_efficiency())"</code>
生成特征数据集	<code>python src\make_dataset.py data\campus.jpg --out campus</code>
训练/评估分类器	<code>\$env:DS_FILES = "dataset.csv"; python src\train_model.py</code>
单图 ML 判定	<code>python -c "import sys; sys.path.insert(0,'src'); from ml_predict import get_predictor; import numpy as np; from PIL import Image; print(get_predictor().predict(np.asarray(Image.open('img\cover.png').convert('L'))))"</code>
C++/Python 一致性自检	<code>python -c "import sys; sys.path.insert(0,'src'); import cppembed; cppembed.selfcheck()"</code>
GPU 数据集生成	<code>python gpu\make_imageset.py</code>
GPU 特征自检	<code>python gpu\featurize_gpu.py</code>
GPU 训练	<code>python gpu\train_ml_gpu.py</code>
GPU 单图检测	<code>python gpu\predict_gpu.py img\cover.png</code>
构建发布包	<code>python -m build</code>
安装 wheel	<code>pip install dist\nsf5stego-1.9.0-py3-none-any.whl</code> (或直接 <code>pip install nsf5stego</code>)

目的	命令
生成 v2 数据集 (12 档 / 143 维)	<code>python src\make_dataset.py --out campus_v2 --feature-set v2 --variants all</code>
生成 SRM 增强数据集 (同源)	<code>python src\make_dataset.py data\campus_jpg --out campus_srm --preprocess srm -j 16</code>
指定 v2+JPEG 数据集训练	<code>\$env:DS_FILES = "dataset_campus_v2_jpeg.csv"; python src\train_model.py</code>
GPU v2 特征自检	<code>python gpu\featurize_v2_gpu.py</code>
BOSSbase 多源生成 + GPU 训练	<code>py gpu\make_imageset.py data\BOSSbase_1.01 --out bossbase (再运行 python gpu\train_ml_gpu.py)</code>
切换 53d 可解释模型 (API)	<code>python -c "import sys; sys.path.insert(0,'src'); from ml_predict import MLPredictor; import numpy as np; from PIL import Image; print(MLPredictor(model_path='models/stego_classifier_v2_jpeg_lgb_51d.joblib', clip_outliers=False).predict(np.asarray(Image.open('img/cover.png').convert('L'))))"</code>

附录 C · 12 周打卡检查表

每完成一项就在日期列写下当天日期。两周内完成一章不丢人，跳过动手实验才丢。

周	主题	必须完成的动作	日期
1	图像与二进制	运行位运算示例；说出 LSB 定义	
2	Python 环境	pip 装依赖；跑通 run_e2e.py；保存一张灰度图	
3	LSB 与盲分析	嵌入→分析实验；看懂 test_steg.py 的输出	
4	卡方/RS 原理	给同学讲清 Gn 塌缩；跑一遍 test_false_positive.py	
5	矩阵编码/F5	手算一个 p=3 例子；用 matrix_demo 验证	
6	nsF5/湿纸	读 ns5_core.py 湿纸段；跑 test_wet_paper_needed	
7	哈希键控	口令错误实验；篡改 1 像素实验；写出键控边界	
8	ML 基础	跑通 v1 训练；能解释特征/泄漏 /GroupKFold/AUC	
9	ML 隐写检测	对比 11d/143d/53d 判定；解释 SRM 与双版本策略	
10	工程化	跑 cppembed.selfcheck 与 featurize_v2_gpu 自检；读 SRM/多源管线	

周	主题	必须完成的动作	日期
11	综合项目	选定方向并完成 70% 代码/实验	
12	汇报	完成报告与演示；脱稿 回答 10.5 的三个问题	

附录 D · 概念 → 代码定位表

学习后期“忘了某概念在哪个文件”，查这张表最快。

概念	文件	优先阅读内容
消息编码/长度头	src/ns5_core.py	encode_string / decode_string
图像哈希/种子	src/ns5_core.py	derive_seed / get_image_hash
确定性置换	src/ns5_core.py	permute_index
汉明码/伴随式	src/ns5_core.py	build_hamming / syndrome
矩阵嵌入	src/ns5_core.py	MatrixEmbedding._embed
湿纸求解	src/ns5_core.py	solve_wet_paper / gauss_solve_GF2
像素域 nsF5	src/ns5_core.py	nsF5Pixel._embed
高层嵌入/解码	src/ns5_core.py	embed_string / extract_string
卡方统计	src/steganalysis.py	chi2_stats / chi2_sf
RS 统计	src/steganalysis.py	rs_metrics
综合判读	src/steganalysis.py	analyze
特征顺序	src/fsfeatures.py / ml_predict.py	FEAT_KEYS
训练流水线	src/train_model.py	main()
数据集生成	src/make_dataset.py	main()
C++ 嵌入封装	src/cppembed.py	embed_string / selfcheck
GPU 特征	gpu/featurize_gpu.py	批量算子与 check_vs_cpu
GUI 回调	src/gui.py	按钮事件 → 核心函数
v2 特征计算	src/featurize_v2.py	ALL_FEATURE_NAMES / featurize_v2()
SRM 预处理	src/srm_filter.py	srm_residuals_np / preprocess_batch_torch
模型推理（双版本）	src/ml_predict.py	MLPredictor(model_path=..., clip_outliers=...)

概念	文件	优先阅读内容
GPU v2 特征	gpu/featurize_v2_gpu.py	extract_features_v2_gpu() 与自检

附录 E · 推荐资源

书籍（中文先入门）

《机器学习》（周志华，清华大学出版社）——第 2 章“模型评估与选择”务必精读；

《统计学习方法》（李航）——逻辑回归与感知机的经典推导；

《数字图像处理》（Gonzalez & Woods）——图像处理经典教材：位平面、直方图与频域基础（早期实验用过其中教材图，v1.4 数据源已改为纯校园照片与 BOSSbase）；

《深入浅出信息隐藏》类中文教材可作为扩展阅读。

核心论文（按阅读顺序）

Westfeld, “F5—A Steganographic Algorithm”, 2001——F5 与矩阵嵌入出处；

Westfeld & Pfitzmann, “Attacks on Steganographic Systems”, 1999——卡方检验；

Fridrich, Goljan & Du, “Reliable Detection of LSB Steganography in Color and Grayscale Images”, 2001——RS 分析；

Fridrich, Goljan, Lisonek & Soukal, “Writing on Wet Paper”, 2005——湿纸编码；

Pevný, Filler & Bas, “Using High-Dimensional Image Statistics to Perform Unsigned Steganalysis”——现代特征法（SPAM）入口。

在线资料与工具

项目 README 与权威结果表 docs/RESULTS.md（SRM/143d/53d 与多源实验的现口径，附语料与协议标注）；GitHub：Yukinoshita-lin/nsf5-steganography；

scikit-learn 官方文档：LogisticRegression、GroupKFold、roc_curve；

PyTorch 官方教程（GPU 版依赖）；

GitHub Actions 文档：了解 CI 中如何跑测试与发版。

动手做 | 到这里，手册主体结束。回到导读 0.4 的能力清单，逐条给自己打分：全部能打勾，就给自己发一张“nsF5 入门结业证”吧。

附录 H · 命令行工具与安装版 (v1.9.0)

本手册正文以“从 Python 代码出发”讲解算法; v1.8 起, 项目同时提供不写一行代码就能完成完整流程的两种形态: 图形界面 (见第 2 章与第 10 章) 与命令行工具 nsf5stego。本章速查这两种形态的安装与用法, 命令在 Windows PowerShell 与 Linux/macOS 终端下均可运行。

H.1 三种安装方式

PyPI (推荐 pip 用户): `pip install nsf5stego` —— 自带两个部署模型、命令行入口与图形界面;
Windows 安装版: 到 GitHub Releases 下载 `nsf5stego-setup-1.9.0.exe`, 简体中文向导、每用户免管理员、开始菜单快捷方式、可选加入 PATH; 免安装便携包: 解压 `nsf5stego-portable-1.9.0-win64.zip` 即用。三者算法完全一致, 按使用场景任选。

H.2 命令行三步 (与 GUI 一一对应)

嵌入 (把文本写进图片):

```
nsf5stego embed cover.png -m "要隐藏的文本" -p 口令 -o stego.png
```

解码 (从含密图还原文本, 方法/p/口令须与嵌入一致):

```
nsf5stego extract stego.png -p 口令
```

盲隐写分析 (卡方 + RS + 机器学习判定; 支持多图批量):

```
nsf5stego analyze stego.png --json
```

文本按 UTF-8 编码, 中文可直接嵌入; analyze 支持 --json 机器可读输出与通配符批量, 退出码 0/1/2 区分 成功/失败/参数错误, 便于脚本化考核。

H.3 新手引导路径

首次打开图形界面会自动载入一张演示封面, 界面顶部的引导条会随操作推进, 始终提示“下一步点哪里”: 点「嵌入并保存」(观察改动像素数与保存路径) → 勾选「查看差异」放大看哪些像

素变了 → 点「解码提取」验证完整还原。整个闭环一分钟内可完成, 完成后再回到正文对应章节理解背后的原理。

H.4 与手册正文的对应

命令行的实现入口是 `src/cli.py`, 算法核心仍在 `ns5_core.py`: 嵌入/解码原理见第 1–4 章, 盲隐写分析见第 5 章, 机器学习检测见第 8–9 章。用 CLI 完成实验后回到对应章节阅读源码, 学习路径与纯 GUI 用户完全一致。

附录 J · 章末练习提示与答案

本附录只给“提示 + 关键数字”，完整推理请先自己做——对过答案才算练过。题号与各章章末练习对应。

J.1 第 1 章

P1.1: $156 = 10011100_2$, $LSB = 0$ 。清零后置位: $156 \& 0xFE = 156$, 再 $| 1 \rightarrow 157$; 异或: $156 \wedge 1 = 157$ 。两法结果相同。

P1.2: 嵌入后 [133, 132, 131, 129, 134, 130, 129, 135], 变化量 [+1, -1, +1, 0, 0, -1, +1, 0] (与 3.2 节的表逐位一致)。

P1.3: 全 0 图嵌入随机比特后, 全部像素在 0/1 之间被改写——直方图从“集中在 0”变成“0 与 1 各半”, 值对 (0,1) 被彻底拉平; 视觉上仍是全黑 (值 1 $\approx$ 黑), 但卡方统计量会瞬间飙升。视觉冗余在, 统计冗余没了。

P1.4: MSB 平面严重偏斜 (正整数 MSB=1 的占比高), 越低的位平面越接近 50%——LSB 平面就是那张“最像抛硬币”的。

J.2 第 1½ 章

P1½.1: 量化 $DC = \text{round}(-224 / 5) = -45$; 反量化 $-45 \times 5 = -225$; 重建平均亮度 $= -225 \div 8 + 128 = 99.875$, 与原值 100 差 0.125——这就是“有损”的具体尺寸。

P1½.2: $+3 \rightarrow +2$ 是减幅 (幅值 -1、保符号、LSB 翻转); $+3 \rightarrow +4$ 虽然也翻转了 LSB, 但它是增幅——把能量往高频推, 违反 nsF5“最小改动”的纪律, 并且会在系数直方图上留下“高频增强”的可检测特征。隐写修改的第一原则: 能用最小的、方向确定的改动达成, 就不要引入新的统计痕迹。

P1½.3: 质量越高 $\rightarrow$ 量化步长越小 $\rightarrow$ 取整归零的系数越少 $\rightarrow$ 非零 AC (载体) 越多 $\rightarrow$ 容量越大。反过来, 低质量 JPEG 本身就是一次“大扫除”。

P1½.4: 像素域容量 ≈ 8 KB、改动 108 像素; JPEG 域容量 2 205 bit、改动 117 个系数——对照 1.5.3 节的表。

J.3 第 2 章

P2.1: $250 + 10$ 在 uint8 里回绕为 4; $3 - 5$ 下溢为 254 (模 256)。

P2.2: $b = a.copy()$ 或 $b = np.array(a)$; 副本的 $b.base$ is None, 视图的 $b.base$ is a 。

P2.3: 比特数 = 字节数 $\times 8$, $126 \times 8 = 1008$ (恰为 504 的两倍, 因为长度头只有 16 位不随消息翻倍); 改动数不翻倍——它由消息位与像素 LSB 的“命中率”随机决定, 期望翻倍、实际有波动。

P2.4: $img[:, ::2]$ 是水平隔列抽样, 画面变瘦——但它仍是原数组的视图, 修改它会改原图。

J.4 第 3 章

P3.1: 对和不变 [16, 12, 18, 14], 期望 [8, 6, 9, 7], 偏差 [+2, 0, 0, 0], $\chi^2 = 4/8 = 0.5$, $df = 3$, $\chi^2_{sf}(0.5, 3) \approx 0.919$ —— p 值极高, 四对里三对被完全拉平, “疑似随机化”。

P3.2: [128,128,131,131]: 原始 $f = 3$; F 翻转 $\rightarrow$ [127,127,132,132], $f = 5$; M 翻转 $\rightarrow$ [129,129,130,130], $f = 1$ 。[200,201,202,203]: 原始 $f = 3$; F $\rightarrow$ [199,202,201,204], $f = 7$; M $\rightarrow$ [201,200,203,202], $f = 5$ 。第一组在 M 下更平滑 (贡献 R_m 一侧), 第二组两种翻转都更粗糙。

P3.3: 未嵌入时全部频数压在灰度 0, (0,1) 对的 χ^2 巨大、 p 值 ≈ 0 (判“干净”); 嵌入随机比特后 (0,1) 对瞬间拉平, p 值 ≈ 1 (判“可疑”) ——本底越不平衡, 嵌入后的卡方信号越强。盲区: 嵌入全 0 比特时一个像素都不用改, 无痕。

P3.4: 满容量嵌入几乎改写全部 LSB, 偶奇配对彻底拉平、位面彻底随机化——prob 与 G_n 塌缩幅度都会到极值。“藏得越多越安全”在视觉上成立、在统计上恰恰相反。

P3.5: 卡方的配对拉平依然发生但集中于中间亮度段; RS 的结构塌缩因“避开纹理区”而减弱——这正是自适应隐写 (如 HUGO/S-UNIWARD 一族) 的设计动机, 8.8 节的 SRM 特征就是冲着它们去的。

J.5 第 4 章

P4.1: $s = H_1 \oplus H_2 \oplus H_4 \oplus H_7 = [0, 0, 0]$; $d = s \oplus m = [0, 1, 1] \rightarrow H_6 \rightarrow$ 翻转第 6 位, 码字 [1, 1, 0, 1, 0, 1, 1]。错第 5 位的纠错演练: $s(y) = [0, 1, 1] \oplus H_5 = [1, 1, 0]$; 若已知目标 m , 则 $s(y) \oplus m = [1, 1, 0] = H_3$, 定位第 3 位。注意“纠错需要知道 m 或假设 s 应为 0”——隐写提取端不知道 m , 所以不做纠错, 这也是隐写与信道编码的目标差异。

P4.2: $p=2$ 时 H 列为 $[1,0]$ 、 $[0,1]$ 、 $[1,1]$, $\alpha = 2 \times 4/3 = 8/3$ 。600 比特: $p=2$ 需 300 块 $\times 3$ 位 = 900 个位置, 平均改 $300 \times 3/4 = 225$ 位; $p=3$ 需 200 块 $\times 7 = 1400$ 个位置, 平均改 $200 \times 7/8 = 175$ 位。 $p=3$ 用更多位置换更少改动——容量富余时总是划算。

P4.3: 成立的前提是“载体 LSB 近似随机”。全 0 消息时 m 恒为 0, 免改条件变成 $s = 0$, 概率仍是 $1/2^p$ ——真正的提醒是: 低熵消息可被“已知消息攻击”利用 (攻击者猜出消息后反推嵌入位置), 第 6 章的键控因此重要。

P4.4: 改动数近似翻倍——伴随式由“载体 LSB 与消息”共同决定, 消息翻倍后块数翻倍、每块的改动概率不变。

P4.5: 16 384 比特在 512×512 (262 144 位置) 里 $p=3$ 需 5 462 块、 $p=6$ 需 2 731 块, 容量都够。追求最少改动选大 p , 追求定位粒度与稳健选小 p ——本项目默认 $p=3$ 是折中点。

J.6 第 5 章

P5.1: $m=[0,1]$ 时 $d = [0,1] \oplus [0,1] = [0,0]$, 零向量不在 H 的列里 $\rightarrow$ 免改; $m=[1,1]$ 时 $d = [1,0]$, 干位里位置 1 与位置 4 的列都命中——求解器按确定性次序取第一个 (相同输入必得相同方案)。

P5.2: $xv = [+3, 0, +1, +6, +2, +5]$, 湿点 = 位置 2 (0) 与位置 3 (+1); 翻转位置 4: $134 \rightarrow 133$ ($+6 \rightarrow +5$, 合法减幅)。

P5.3: 湿/干划分只在**嵌入端**约束“允许改哪里”; 提取端直接按同一次序读全体载体的 LSB (干点翻转后 $|c|$ 仍 ≥ 1 , 不会从载体集合里消失)。若湿点改由口令的伪随机数决定, 嵌入会改动它自己划出的湿点之外的位置——提取端无法知道“哪些位可信”, 自同步即失效。内容派生的妙处正在于: 嵌入只动干点, 干点翻转后图像内容变了, 但“哪些位置被翻转过”由提取端从消息本身校验, 无需重算湿点。

P5.4: 平滑图 (如渐变演示图) 的像素大量落在 127/128/129, 湿点密度高——nsF5 绕开它们、矩阵编码不管, 后者会撞雷重试, 改动数反超。结论: 比较算法必须固定测试图, “谁好”是相对于图像统计而言的。

P5.5: 全部载体都是湿点时干点为空, 方程一般无解——项目抛 CapacityError (对照 test_capacity_error 的 16×16 全 128 用例), 提示换大图/高质量/短消息。

J.7 第 6 章

P6.1: 第一步状态推进后 $x = 9e3779b97f4a7c15$ (黄金常数本身), 输出 16 294 208 416 658 607 535。

P6.2: 卡点有三处——① 头部池需要 seed0 (口令派生), 错口令读出的“哈希”是乱码; ② 乱码哈希派生的 seedB 把正文池洗成完全不同的顺序; ③ 即便碰巧读回比特, 长度头与内容哈希校验都会失败。拿到同口令的第二张图, 攻击者可以对比两图的头部池位置是否相同 (seed0 相同) ——这是“多图同口令”要换主密钥的原因。

P6.3: 教学项目里攻击者模型是“拿到图与源码的普通对手”, 内容哈希足以让口令错误与图像篡改稳定失败; 真实取证产品面对能重嵌整张图的强攻击者, 必须用密钥化 MAC 把“能算哈希的人”限定为持钥者。

P6.4: 会成功——另存 PNG 若像素不变, LSB 与头部比特全部完好。哈希感知没有被触发的理由: 像素域锚定的是**原始封面**的字节哈希, 语义是“repro/实验档案比对原图”, 而不是“比对含密图”; JPEG 域则有更精巧的设计——锚是色度哈希, 嵌入只动亮度 Y, 色度未变则校验通过。分层理解, 各安其位。

P6.5: 实测约 47%~53% 的位翻转 (如 6.4 节的 121/256) ——多次实验取平均会非常接近 128/256。

J.8 第 7 章

P7.1: $\eta=10$: $w = 0 - 10 \times (-0.1) = 1.0$, 新损失约 0.349 (仍在下降); $\eta=100$: $w = 10$, 干净样本的预测被推到接近 1, 损失升到约 1.54——步长过大, 一步跨过谷底。特征尺度越大, 同样的学习率越危险。

P7.2: 按伪 photo_id 分组 (同源样本同折) 更合理——否则“兄弟样本”同时出现在训练与验证两侧, 验证分数虚高。

P7.3: 假阳性 (FP) ——把干净图判成含密。取证场景下意味着对无辜者的怀疑, 社会成本高于漏掉一张弱嵌入图。

P7.4: sklearn 的 LogisticRegression 默认带正则与不同的优化器, 方向应与手算一致 (w 为正), 数值不必相同——这正好让你体会“同一模型, 不同求解器”的差异。

P7.5: 错在拿不同语料的数字比高低 (RESULTS.md: 不同语料的数字不可比较) ——0.89 与 0.81 反映的是语料难度, 不是模型强弱。

J.9 第 8 章

P8.1: 按领域知识含量排序: yenet (固定 SRM 核前置) > 143d (SRM 统计特征) > 53d/11d (手工统计) > xuNet (无领域知识)。AUC 排序基本一致 ($0.9541 > 0.8062 > 0.7172 \approx 0.7128 > 0.5007$) ——但 143d 低于 yenet 提醒我们: “知识放对位置”之外, 端到端架构的表达能力也是变量; 两者不是同一台机器上的两个旋钮。

P8.2: 完美模型的 ROC 是“左上角折线”: 检出率 100% 时误报率 0%, $AUC = 1.0$; 随机模型的 ROC 是对角线, $AUC = 0.5$ 。

P8.3: 差异来源: 你的照片风格 (内容复杂度)、拍摄设备 (噪声本底)、分组是否严格、样本量——任何一项都足以让 AUC 偏离 README。

P8.4: 对称的问题会出现——模型把 PNG 特有的“无压缩噪声”当成异常, 对真实 JPEG 部署场景产生新的误报/漏报偏移。训练分布与部署分布对齐, 是比调参更优先的事。

P8.5: 必选 53d: 向非技术听众/评审解释判定依据、需要审计每维含义的合规场景; 必选 143d: 在线服务批量筛查 (精度优先)、对抗强嵌入变体的研究基线。

J.10 第 9 章

P9.1: 预期前三名是确定性置换、特征统计的内层循环与文件 IO——前两者是 9.2 节的下沉对象, 第三者可用缓冲读写优化。

P9.2: 固定种子与输入 → 分别调用两版置换 → 逐位 diff (`np.array_equal`) → 重复多组种子与规模, 全部一致才放行。

P9.3: 本项目的定位包含“无加速环境可用” (服务器/CI/教学机), 回退保证功能与结果一致; 若性能是硬需求 (如在线服务), 降级应改为“明确报错+要求安装”, 避免静默变慢。

P9.4: 改算法参数最先惊动单元层 (`test_core` 的往返断言); 打包丢文件最先惊动系统层 (冻结冒烟的第 4/5 项)。

J.11 第 10 章

P10.1: 好的怀疑点示范——“143d 对 11d 的增益有多少来自 SRM、多少来自数据扩容?”复现步骤: 固定照片分组, 分别训 11d 与 143d; 判定标准: 同协议下 AUC 差 ≥ 0.02 且置信区间不重叠。

P10.2: 假设要可证伪 (“新特征使 AUC 提升 ≥ 0.02 ”), 协议必须按源图分组, 判定标准写进实验脚本。

P10.3: 录音回放时重点听: 有没有说清数字的语料与协议、有没有把“相关”说成“因果”。

J.12 第 11 章

P11.1: 判断书的关键是引用技术事实——例如②的要点是“隐写的隐蔽性被用于欺骗特定受众”, 而不是“隐写是坏技术”。

P11.2: 升级方案示范 (哈希键控 $\rightarrow$ HMAC): 口令经 PBKDF2 派生 HMAC 密钥; 头部与正文的关键字段全部改为 HMAC 覆盖; 验收标准 = 原有往返测试全绿 + 新增“篡改字段必失败”用例。

P11.3: 示范位置——README 的 α 公式与 efficiency.py 的实现; RESULTS.md 的 0.8062 与 train_model.py 的评估输出; GUI 的 0.2% 提示与 _embed 的统计报告。

附录 K · 真实数据速查（供实验对账）

本附录汇总全手册引用的真实数字，全部可由项目脚本复现（出处括注）。权威来源：
docs/RESULTS.md（由 experiments/build_results_table.py 生成，勿手改）与各脚本实测输出。

K.1 嵌入实测（256×256 演示图，2026-10-04）

指标	像素域 (p=3)	JPEG 域 (p=3, q85)
载体总数	65 536 像素	5 150 个非零 AC
嵌入 240 bit 改动	108 像素 (0.2%) (240 bit = 28 字节消息 + 16 位长度头)	117 个系数
理论容量	≈ 3.4 KB 文本 (p=3)	2 205 bit
保真	视觉无差异；PSNR ≈ 76 dB（实测）	视觉无差异（JPEG 压缩噪声本底）

K.2 满容量效率扫描（随机噪声图，compute_comparison(6) 实测）

p	理论 α	实测 α	改动像素
1	2.00	2.01	32 502
2	2.67	2.62	16 655
3	3.43	3.38	8 263
4	4.27	4.18	4 149
5	5.16	5.08	2 054
6	6.10	5.90	1 035
504 bit (p=3)	3.43	2.68	188

口径提醒（勿混比）。 上表前六行是**满容量填充**（把每个 p 的可用位置全部填满）的实测，不是某条固定消息的结果；最后一行另列 run_e2e.py 里那条 63 字节（504 bit）演示消息的实测：同一张图、p=3，只需改动 188 像素。两个数字分母不同——第 2 章 run_e2e 输出里的“改动 188 像素”是后者，第 4 章效率段的 8 263 是前者。

K.3 检测主表（BOSSbase 1.01，按源图 holdout，摘自 docs/RESULTS.md）

模型	特征	AUC	95% CI
LGB-11d	11 维	0.7128	[0.6913, 0.7360]
LGB-143d	143 维	0.8062	[0.7894, 0.8225]
LGB-53d	53 维	0.7172	[0.6943, 0.7406]
xuNet	原始像素	0.5007	[0.5004, 0.5012]
ynet	30 SRM 固定核 + conv	0.9541	[0.9463, 0.9630]

K.4 语料与协议要点

语料	规模	一句话
bossbase_npz	2 000 源图 × 5 变体 = 10 000 样本	领域标准基准
campus_v2_jpeg	414 张校园照片 × 14 = 5 796 样本	自建语料，含真实 JPEG 干净图
ood_real_jpeg	1 514 张真实干净照片	部署前误报率评估（校园 +DIV2K+ALASKA#2）

协议红线：一切训练/评估按**源图分组**；置信区间按源图重采样；不同语料的数字不可比。历史上因违反这三条产生过 0.8946 → 0.7555 的 AUC 虚高事故（2026-09-14 审计，详见 RESULTS.md 第 1 节）。

K.5 篡改与哈希实测

项目	实测值
SHA-256 雪崩（差 1 字符）	256 位中翻转 121 位 ≈ 47%
JPEG 域改动系数 / 载体	117 / 5 150（240 bit, p=3, q85）
JPEG 域色度哈希锚点	只覆盖 Cb/Cr，不被 Y 通道嵌入触碰

项目	实测值
冻结冒烟	六项：体积/版本/往返/ML 逐位/JPEG 往返 +repro/GUI 主题

K.6 LSB 嵌入率扫描（第 3 章，ch03_results.csv）

第 3 章 3.6 节扫描实验的逐档数据（256×256 演示图，随机比特，2026-10-04）。四列指纹同向变化：藏得越多，卡方统计量越低、RS 缺口 Gn 越小、LSB 差分熵越接近 1——视觉上仍无差别，统计上却一步步走向“可疑”。完整 15 列见 StegoLab 的 figures/ch03_results.csv。

嵌入率	载荷比特	改动像素	PSNR (dB)	卡方统计量	RS_Gn	LSB 差分熵
0.0%	0	0	—	34 829	0.555	0.785
10.0%	6 554	3 272	61.2	30 535	0.515	0.812
20.0%	13 107	6 488	58.2	25 995	0.471	0.842
30.0%	19 661	9 834	56.4	21 272	0.425	0.872
40.0%	26 214	13 131	55.1	16 668	0.375	0.901
50.0%	32 768	16 451	54.1	12 228	0.320	0.928
60.0%	39 322	19 719	53.4	8 398	0.261	0.951
70.0%	45 875	23 056	52.7	5 018	0.198	0.971
80.0%	52 429	26 326	52.1	2 353	0.134	0.986
90.0%	58 982	29 590	51.6	679	0.070	0.996
100.0%	65 536	32 911	51.1	72	-0.004	1.000

附录 L · 配套工具 ImageMatrix 使用指南

本附录是第 1 章配套工具 ImageMatrix 的完整说明。它是同作者发布的独立开源项目 (github.com/YukinoShita-lin/ImageMatrix, Apache-2.0 许可), 与本手册学习的 nsF5 项目**没有任何依赖关系, 也不含隐写功能**; 它唯一的用途, 是把“图像 = 数字矩阵”这件事从抽象变成你屏幕上看得见、改得动的东西。

L.1 它是什么, 为什么配它

一句话: **图片 ↔ 数值矩阵 (R/G/B/灰度) 双向转换器**——纯 C/C++ 编写、不依赖任何第三方库, 自带 Win32 图形界面, 也提供命令行模式。

能力	说明
打开图片	BMP / PNG / JPEG / GIF / TIFF, 支持把文件直接拖进窗口
导出 TXT 矩阵	五种排布: CHANNELS / RGB / RGBA / GRAY / HEX
导入 TXT 矩阵	把改过的矩阵文本还原成图片; 缺宽高时弹窗询问宽度, 自动纠错
另存为图片	PNG / JPEG / BMP / GIF / TIFF
像素取值	状态栏实时显示鼠标所指像素的 R/G/B/灰度 与 #RRGGBB

把它放进第 1 章, 是因为它精确地补上了这本手册开头唯一的“实物缺口”: 原文让你“打开项目看代码”, 而看代码需要先会 Python (第 2 章才教); ImageMatrix 让你在**学任何代码之前**, 就能亲手验证“图像就是一张整数表格”。它是 CSAPP“先摸实物”纪律在本手册里的第一件教具。

L.2 获取与安装

方式	做法	适合谁
一键安装包	在仓库 Releases 下载 ImageMatrix_Setup.exe, 双击安装 (无需管理员权限)	只想动手做实验的同学

方式	做法	适合谁
源码编译	用 MinGW-w64 运行 build.bat, 得到 build\imagematrix.exe 与 imagematrix-cli.exe	想顺手读一份纯 C++ 工程的同学
直接运行	编译产物是静态链接的单文件, 拷到别的 Windows 机器也能直接双击运行	实验室机房、无网络环境

安装包会把程序释放到 %LOCALAPPDATA%\Programs\ImageMatrix, 并自动创建开始菜单与桌面快捷方式; 在「设置 → 应用」里可一键卸载。

L.3 界面与核心操作



ImageMatrix 主界面。上方一排是“打开图片 / 导出为 TXT / 导入 TXT / 另存为图片”，以及“矩阵格式”下拉框与“写注释头”开关；中间画布支持滚轮缩放与拖动平移。第一次没有素材时，点“生成测试图案”即可得到一张彩条 + 灰阶 + 渐变的测试图。（截图来自 ImageMatrix 仓库）

- 1. 打开图片：点“打开图片...”，或把图片直接拖进窗口；
- 2. 查看矩阵：点“查看矩阵文本”，窗口内直接预览当前图片的数值矩阵，可复制到剪贴板；

- 3. 导出：在“矩阵格式”里选一种排布，点“导出为 TXT...”，选择保存位置；
- 4. 导入：点“导入 TXT...”，程序按矩阵把图片还原出来（没有 # width 头时会弹窗询问宽度）；
- 5. 保存：点“另存为图片...”，把还原结果存成 PNG / JPEG / BMP 等。

快捷键：Ctrl+O 打开、Ctrl+S 导出 TXT、Ctrl+T 导入 TXT、Ctrl+E 另存为图片；滚轮以光标为中心缩放，左键拖动平移，放大 8 倍以上显示像素网格。

L.4 五种矩阵格式速查

format	每行内容	典型用途
GRAY	一行 = 图像的一行（width 个灰度值）	做第 1.2 / 1.4 节的 LSB 实验（只关心亮度）
CHANNELS	R 矩阵 → G 矩阵 → B 矩阵 → GRAY 矩阵	看清“彩色图 = 三张表叠起来”
RGB	一行 = 一个像素：R, G, B	逐像素看颜色
RGBA	一行 = 一个像素：R, G, B, A	带透明度（A）的图
HEX	一行 = 一个像素：RRGGBB	体会十六进制记法（1.2 节）

导出的文件带 `#` 注释头，声明 width / height / format / max 与灰度公式，读回时靠它准确还原；灰度公式与手册一致： $Y = \text{round}(0.299 * R + 0.587 * G + 0.114 * B)$ （ITU-R BT.601）。

L.5 命令行（批处理与自检）

```
build\imagematrix-cli.exe to-txt -i photo.png -o photo.txt -f GRAY
build\imagematrix-cli.exe to-image -i photo.txt -o back.png
build\imagematrix-cli.exe info -i photo.png
build\imagematrix-cli.exe gen -o pattern.png -w 320 -h 240
build\imagematrix-cli.exe selftest :: 20 项自检
```

参数：`-f RGB|RGBA|GRAY|CHANNELS|HEX`、`--no-header`、`--width N`、`-q 92`（JPEG 质量）。命令行与 GUI 导出的 TXT 经仓库测试验证 **SHA256 完全一致**——这正是第 9 章“一致性优先”的工程纪律，只不过这里用在了一个更小的程序上。

L.6 与手册各章的对应

手册位置	用 ImageMatrix 做什么
1.1 图像即矩阵	导出 GRAY / CHANNELS，亲眼看到“图 = 数字表格”
1.2 二进制与 LSB	在 TXT 里把 200 改成 201，零代码体验 LSB 翻转
1.4 位平面	把一列像素写成二进制，观察高位 / 低位的规律差异
1.5 cover / stego	用记事本手工造出第一对载体与含密图
2.2 用 NumPy 思考图像	先用它看现象，再用 NumPy 复现同一个数组
3.2 最小 LSB 隐写	把手工动作换成代码，比较“手工 vs 代码”的差别
9.5 测试与 CI	参考它的 selftest 与 GUI 冒烟测试（GUI 与 CLI 输出逐字节一致）

L.7 一个完整的动手流程

动手做 |

从一张照片到一对 cover / stego（全程零代码）

- ① 打开一张真实照片（或点“生成测试图案”）；
- ② 格式选 GRAY，导出为 cover.txt，再另存一份 stego.txt；
- ③ 用记事本打开 stego.txt，按约定位置把若干像素改成目标奇偶（藏 1 → 奇数，藏 0 → 偶数）；
- ④ 导入 stego.txt，另存为 stego.png；
- ⑤ 把原图与 stego.png 并排看：肉眼无差别。

进阶：把第 3 章的 Python LSB 脚本跑一遍，导出它生成的 output/stego_*.png，再用 ImageMatrix 打开——你会看到代码与手工做的是同一件事。

最后提醒一句边界：ImageMatrix 帮你**看见**数字，不帮你**隐藏**数字。真正的 nsF5 隐写——汉明矩阵编码、减幅、湿纸编码、口令键控——仍然要在第 4、5、6 章里，跟着 F:\Steganography 的代码一步步学。它只是把你领到门口的那块垫脚石。